

Introduction to High-Performance Machine Learning

Neural Network Pruning and Sparsity

Lecture 8 04/20/2026

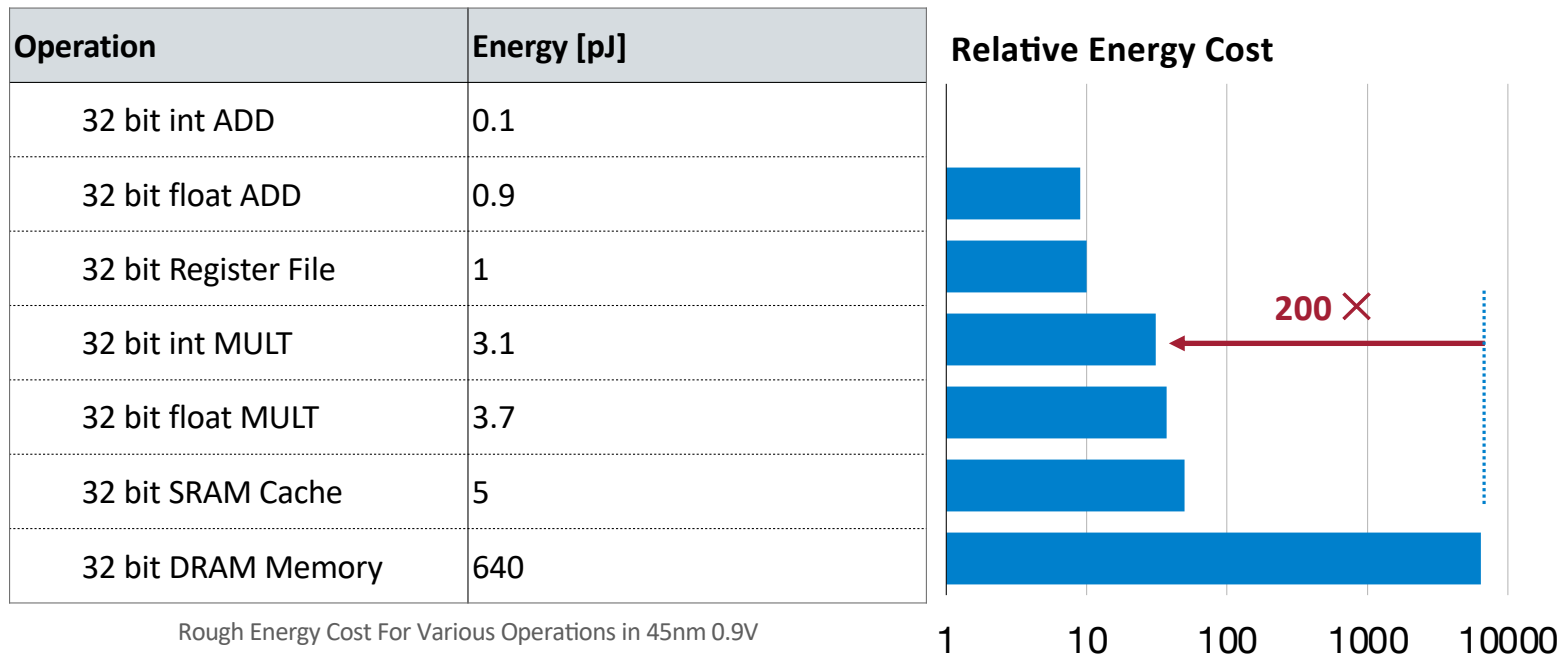
Dr. Kaoutar El Maghraoui



Neural Network Pruning

Memory is Expensive

Data Movement → More Memory Reference → More Energy



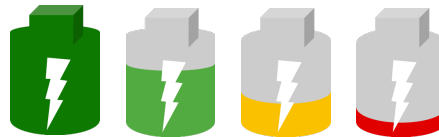
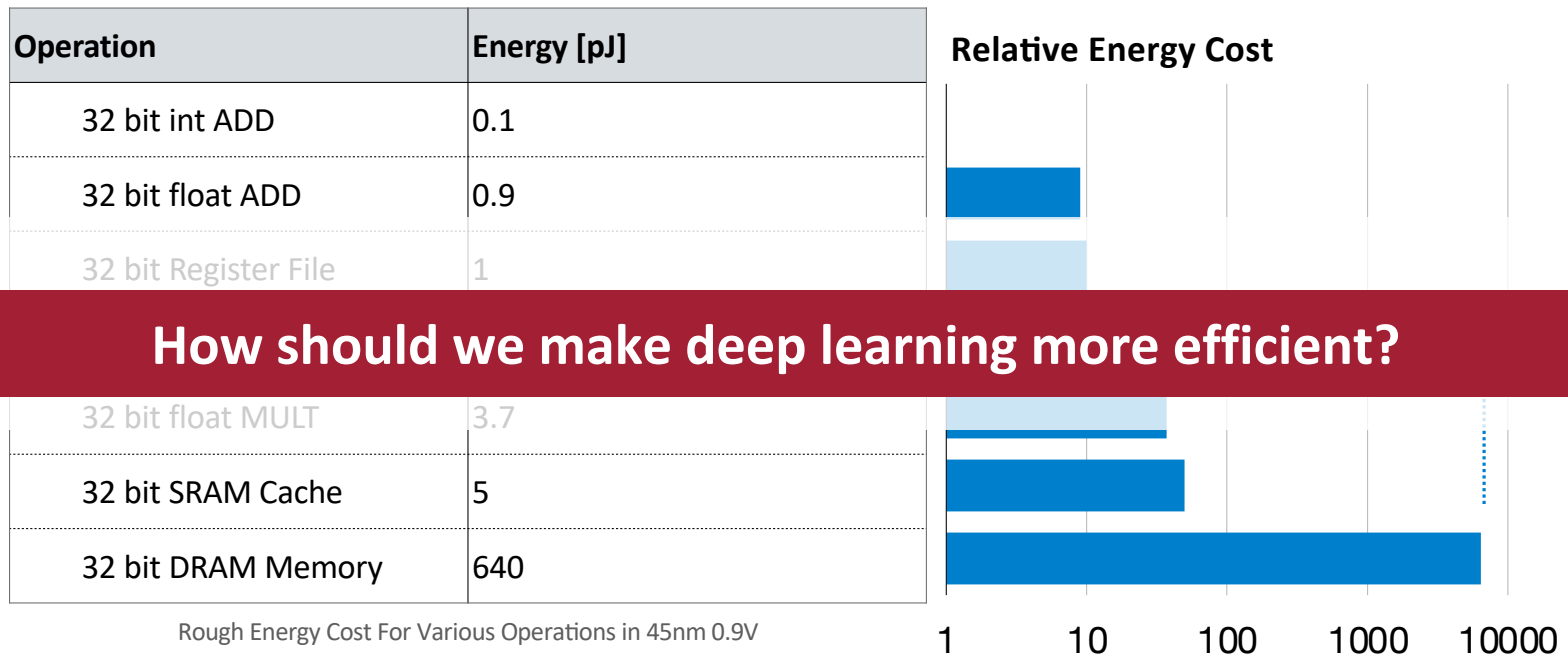
$$1 \text{ } \text{[Image of a RAM module]} = 200 \text{ } \times +$$

[This image](#) is in the public domain

Computing's Energy Problem (and What We Can Do About it) [Horowitz, M., IEEE ISSCC 2014]

Memory is Expensive

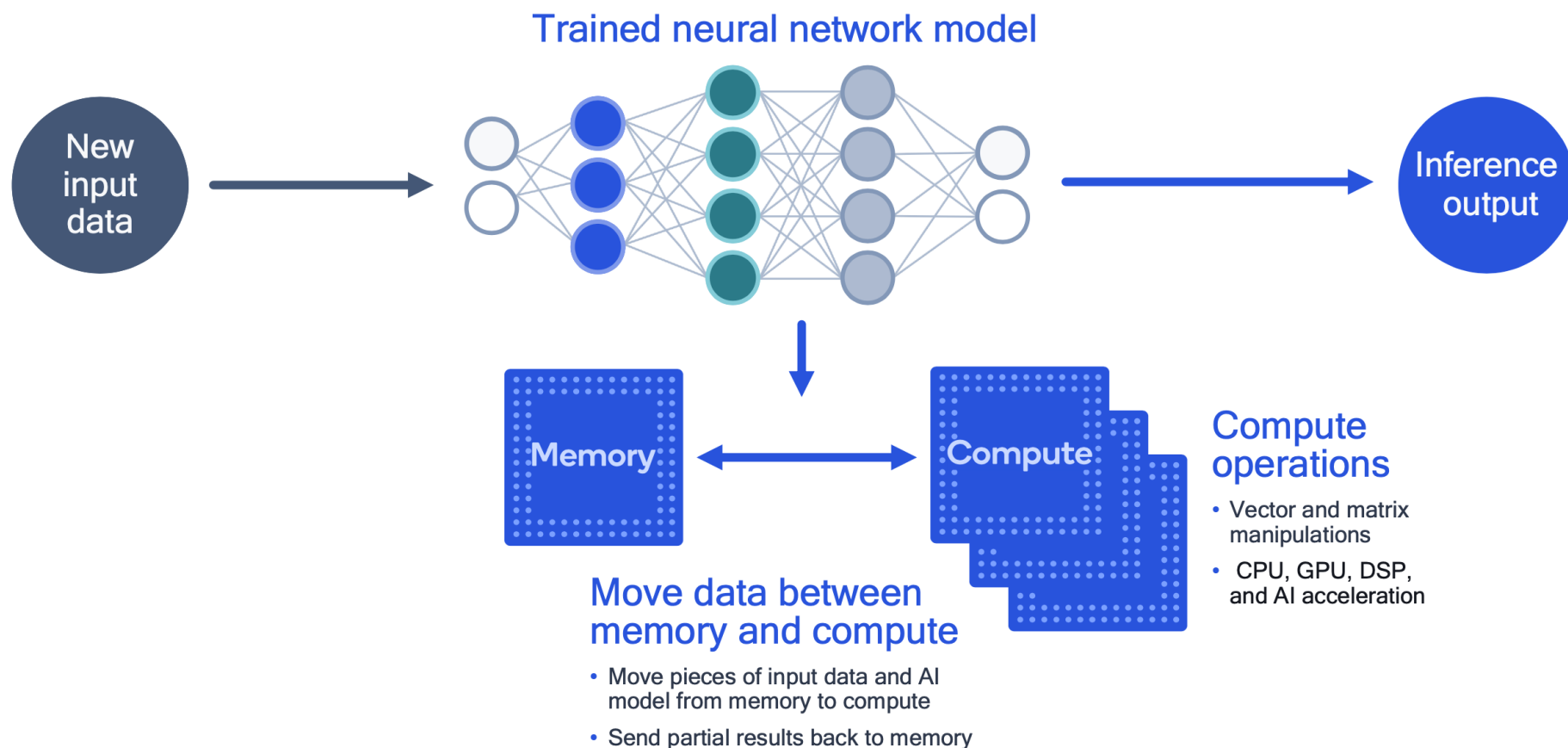
Data Movement → More Memory Reference → More Energy

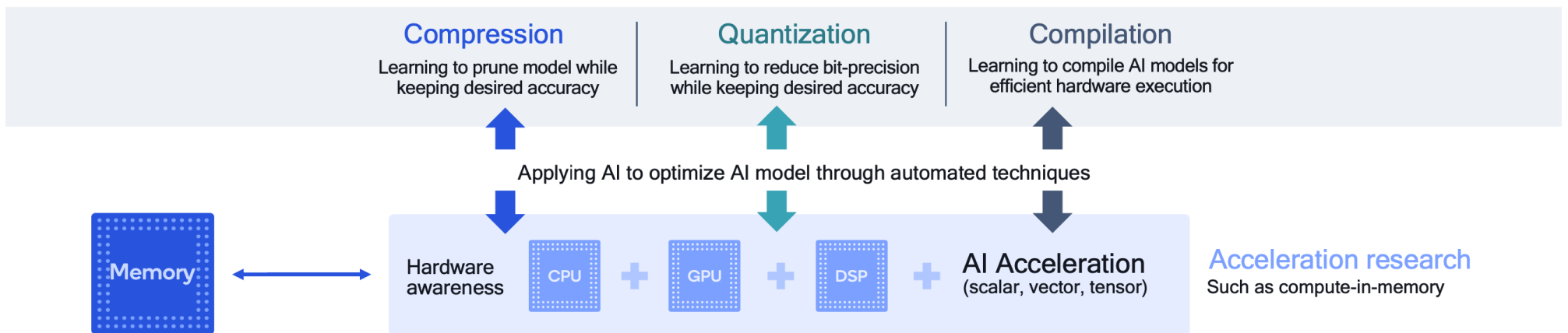
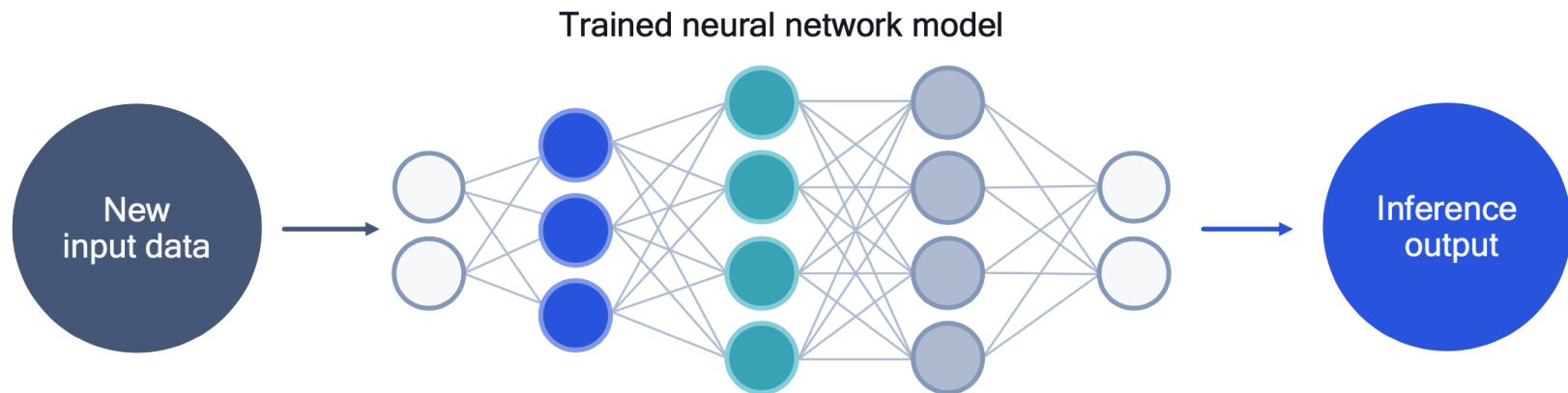


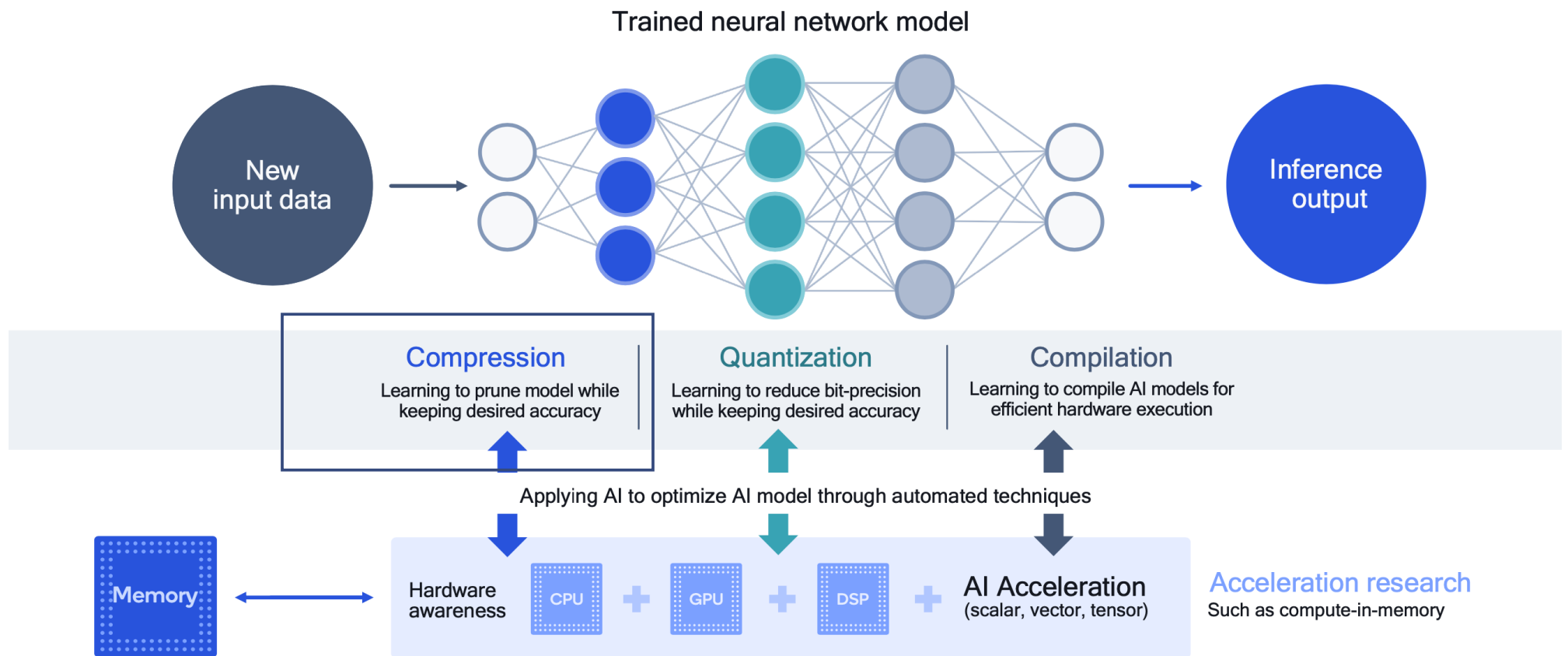
Battery images are in the public domain
[Image 1](#), [image 2](#), [image 2](#), [image 4](#)

Computing's Energy Problem (and What We Can Do About it) [Horowitz, M., IEEE ISSCC 2014]

Advancing AI research to increase power efficiency



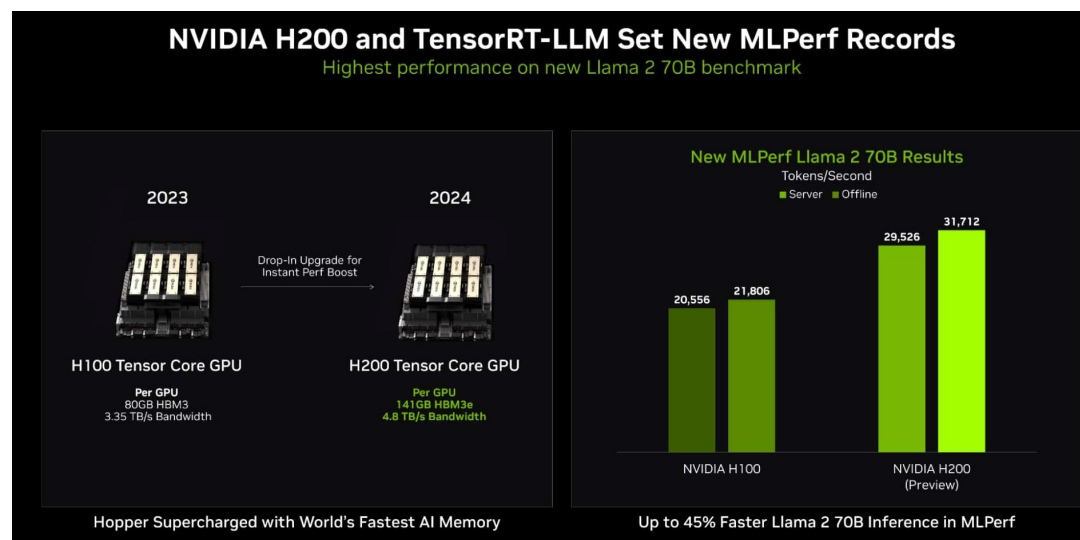




Pruning Large Language Models –MLPerf Open Division Competition

- **Structured Sparsity:**
 - Technique to reduce calculations, first introduced with A100 Tensor Core GPUs.
 - Achieved up to **33% speedup** in Llama 2 inference on H100 GPUs.
- **Pruning for Model Simplification:**
 - Simplifies AI models, enhancing inference throughput.
 - Resulted in **up to 40% speedup** on large language models (LLMs).
- **DeepCache Optimization:**
 - Reduces math needed for inference in Stable Diffusion XL.
 - Accelerated performance by **74%**.
- **Hardware:** All benchmarks conducted on NVIDIA H100 Tensor Core GPUs.
- More recent MLPerf rounds (2024-2025) have featured Llama 2/3 on H200 and Blackwell GPUs with even higher throughput numbers.

HPML



<https://blogs.nvidia.com/blog/tensorrt-llm-inference-mlperf/>

NVIDIA's open division MLPerf submission on Llama 2 70B

Achieved **2.5x speedup in inference** while retaining **99% of the original model's accuracy** by applying advanced pruning techniques. Two forms of pruning:

- **Depth Pruning:**
 - Reduced the model's layer count from **80 layers to 32 layers**.
 - By decreasing the number of layers, the model's overall computational complexity is significantly reduced, resulting in faster inference speeds.
- **Width Pruning:**
 - Reduced the number of intermediate dimensions (width) in the model from **28,762 dimensions to 14,336 dimensions**.
 - Width pruning decreases the size of individual layers, which further cuts down on the number of calculations required per inference pass, thus improving speed.

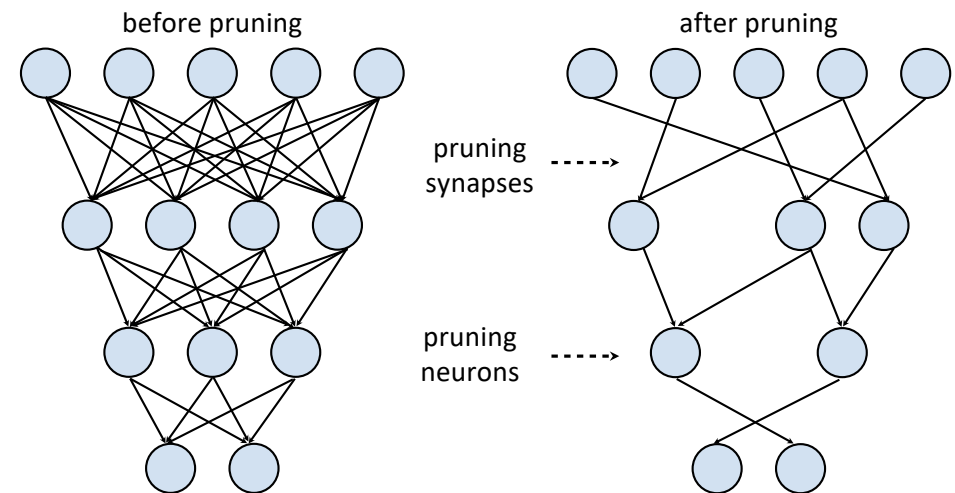


	Closed Division	Open Division	Speedup
Offline samples/sec	4488	11189	2.5x

Llama 2 70B performance metrics for both closed division and open division.
Measured on a single NVIDIA H200 GPU.

Agenda

1. Introduce **neural network pruning** which can reduce the parameter counts of neural networks by more than 90%, decreasing the storage requirements and improving the computation efficiency of neural networks.
2. Go through all steps of pruning and introduce different **granularities** and **criteria** of neural network pruning.



Benefits of Sparsity

- **Reduced Computational Demands:**
 - Optimized for efficiency, ideal for edge devices and mobile apps.
- **Model Compression:**
 - Smaller models with fewer storage and memory requirements, easing deployment and data transfer.
- **Faster Inference Times:**
 - Quicker predictions essential for real-time applications like autonomous vehicles and healthcare diagnostics.
- **Energy Efficiency:**
 - Lower power consumption during inference, contributing to sustainable machine learning practices.

Neural Network Pruning

- **Introduction to Pruning**

- What is pruning?
- How should we formulate pruning?

- **Determine the Pruning Granularity**

- In what pattern should we prune the neural network?

- **Determine the Pruning Criterion**

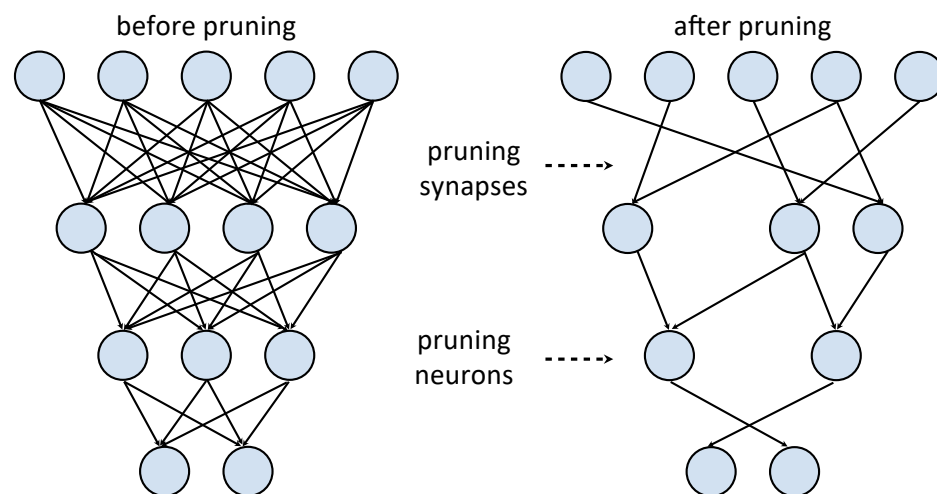
- What synapses/neurons should we prune?

- **Determine the Pruning Ratio**

- What should target sparsity be for each layer?

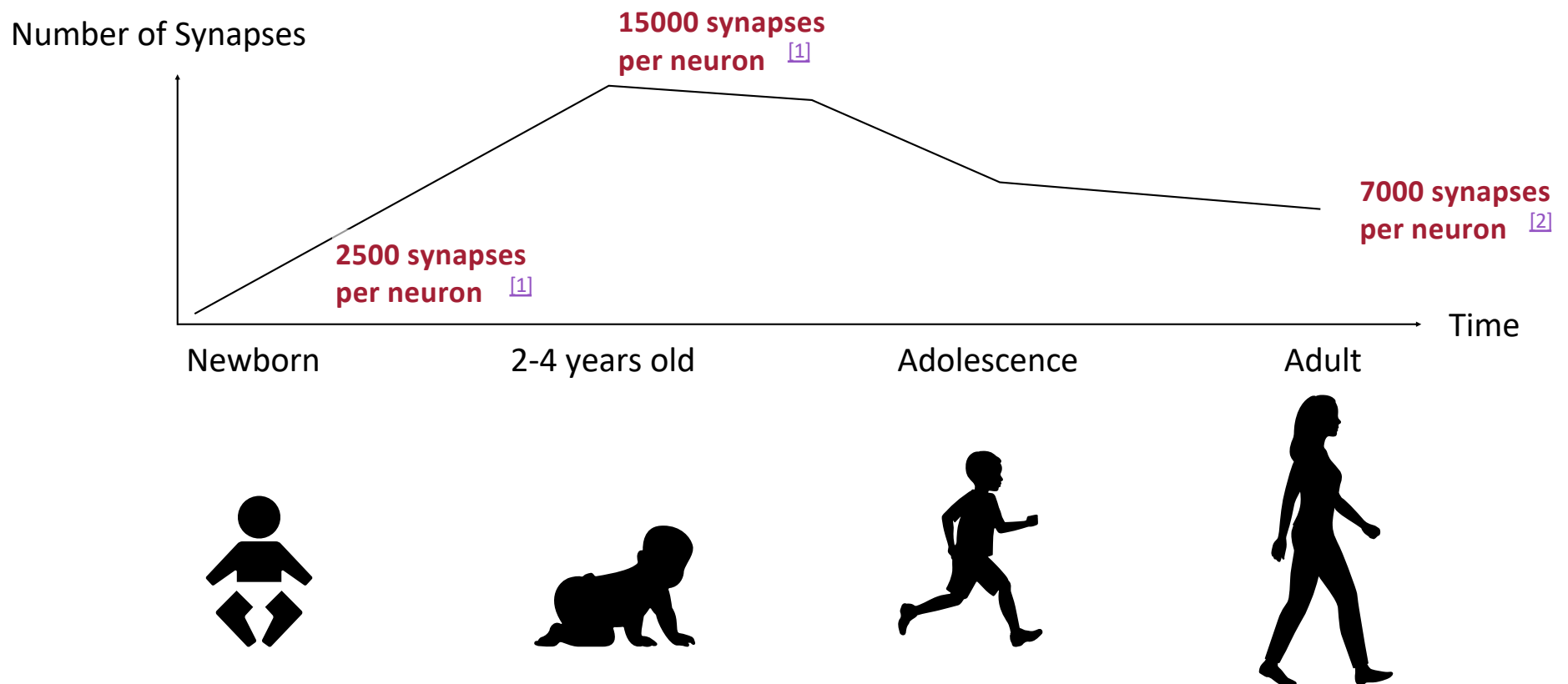
- **Fine-tune/Train Pruned Neural Network**

- How should we improve performance of pruned models?



- Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

Pruning Happens in Human Brain



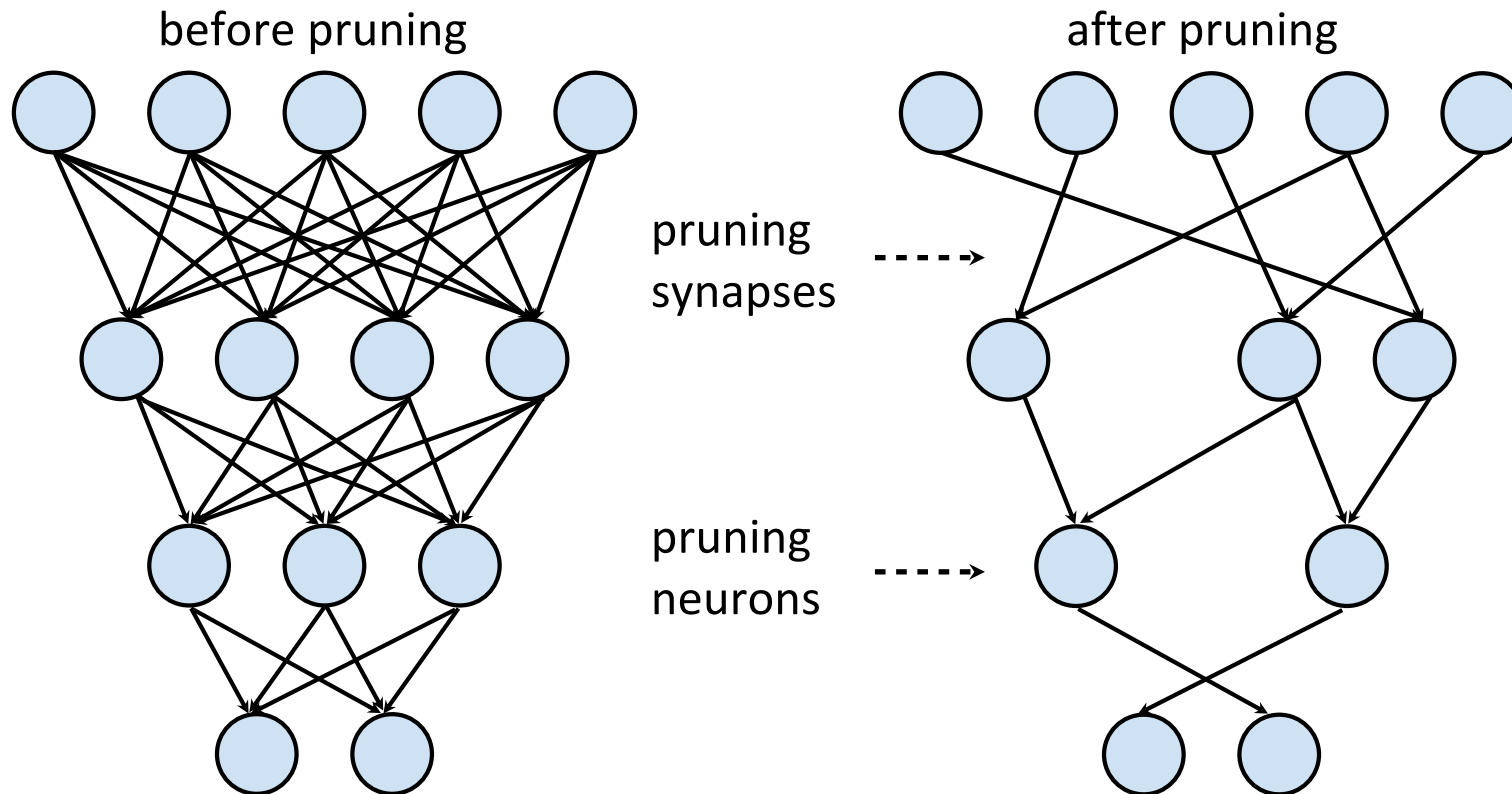
Do We Have Brain to Spare? [Drachman DA, Neurology 2004]
Peter Huttenlocher (1931–2013) [Walsh, C. A., Nature 2013]

<https://efficientml.ai>

Data Source: [1](#), [2](#)
Slide Inspiration: [Alila Medical Media](#)

Neural Network Pruning

- Make neural network smaller by removing synapses and neurons

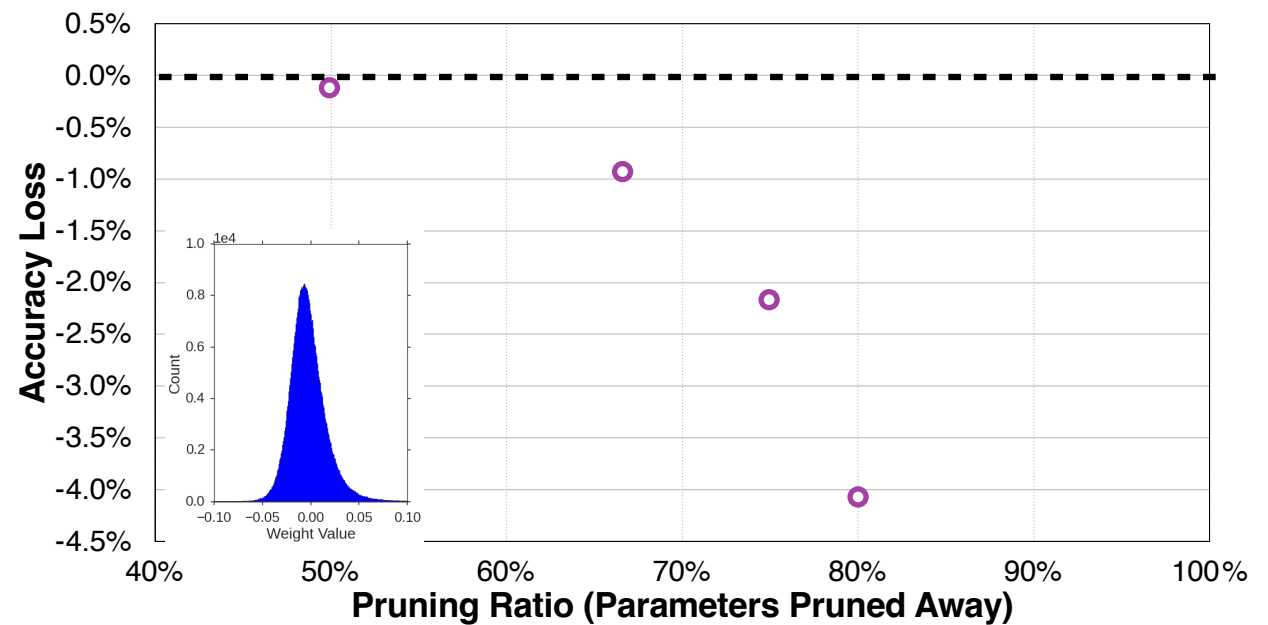
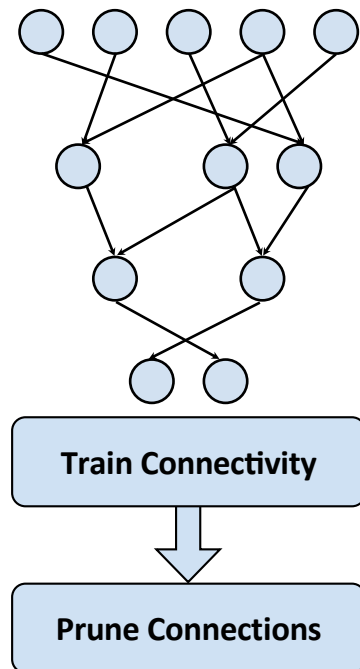


Optimal Brain Damage [LeCun *et al.*, NeurIPS 1989]

- Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

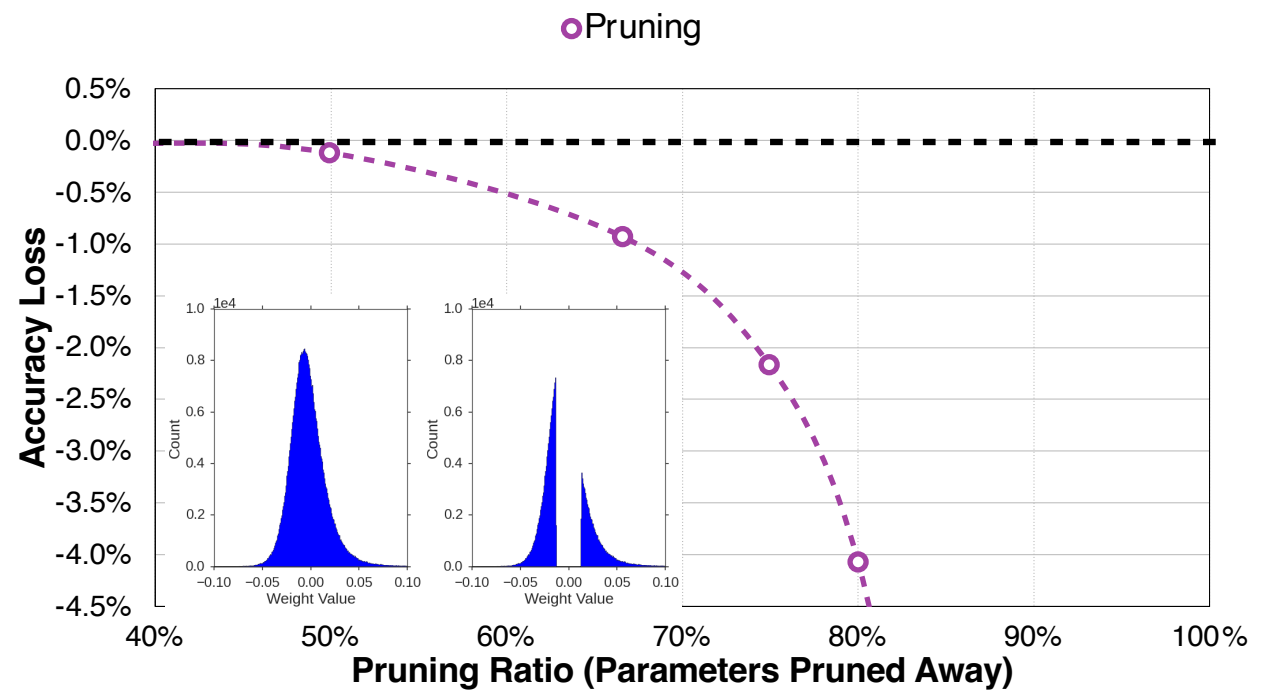
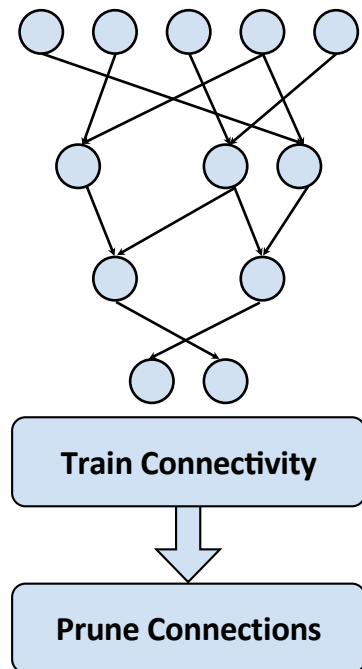
Neural Network Pruning

Make neural network smaller by removing synapses and neurons



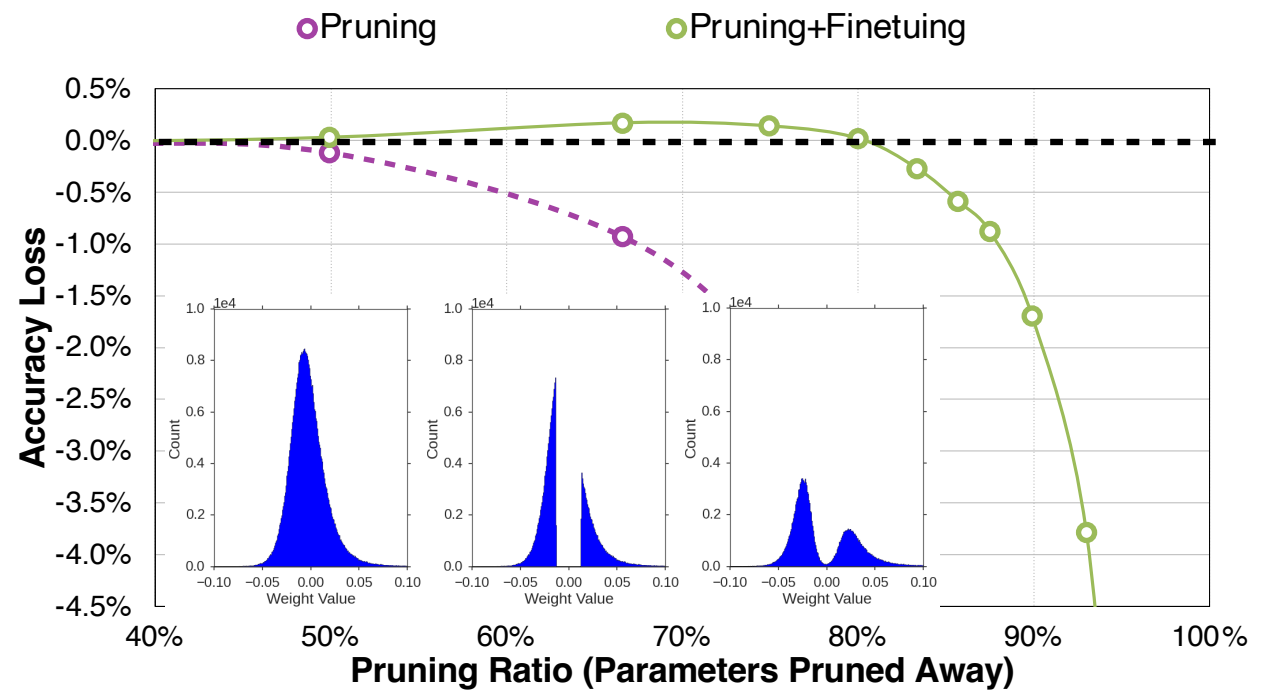
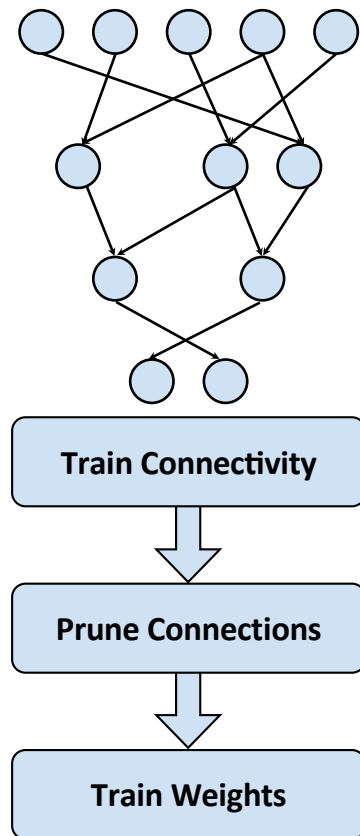
Neural Network Pruning

Make neural network smaller by removing synapses and neurons



Neural Network Pruning

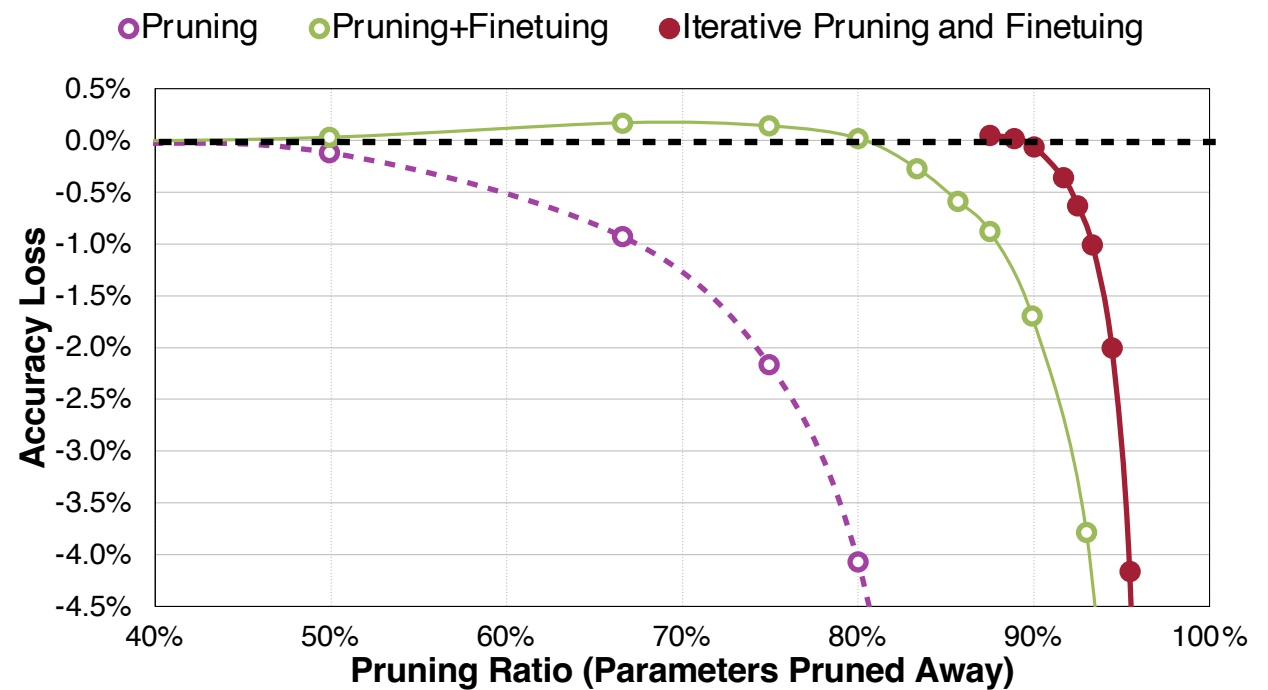
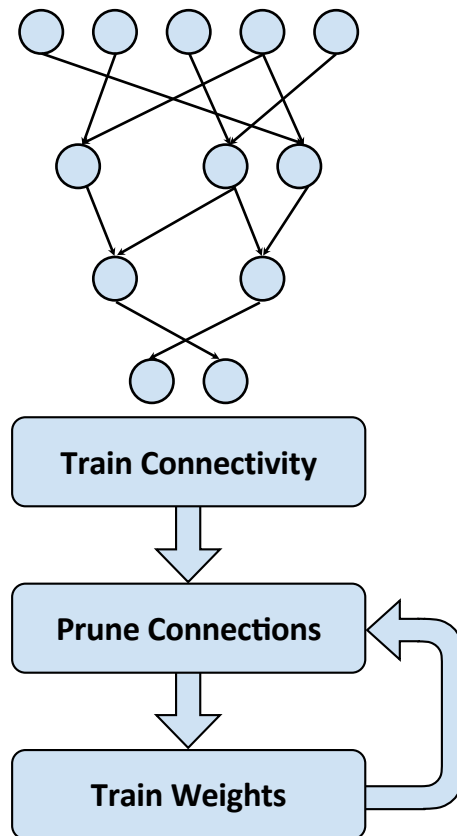
Make neural network smaller by removing synapses and neurons



Learning Both Weights and Connections for Efficient Neural Network [Han et al., NeurIPS 2015]

Neural Network Pruning

Make neural network smaller by removing synapses and neurons



Learning Both Weights and Connections for Efficient Neural Network [Han et al., NeurIPS 2015]

Neural Network Pruning

Make neural network smaller by removing synapses and neurons

Neural Network	#Parameters			MACs
	Before Pruning	After Pruning	Reduction	Reduction
AlexNet	61 M	6.7 M	9 ×	3 ×
VGG-16	138 M	10.3 M	12 ×	5 ×
GoogleNet	7 M	2.0 M	3.5 ×	5 ×
ResNet50	26 M	7.47 M	3.4 ×	6.3 ×
SqueezeNet	1 M	0.38 M	3.2 ×	3.5 ×

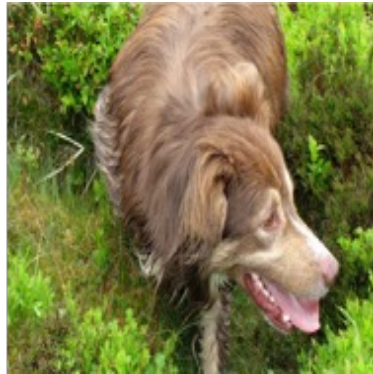
Neural Network Pruning

Pruning the NeuralTalk LSTM does not hurt image caption quality.



Baseline: a basketball player in a white uniform is playing with a **ball** .

Pruned 90%: a basketball player in a white uniform is playing with a **basketball**.



Baseline: a brown dog is running through a grassy **field**.

Pruned 90%: a brown dog is running through a grassy **area**.



Baseline: a man **is riding a surfboard on a wave**.

Pruned 90%: a man **in a wetsuit is riding a wave on a beach**.

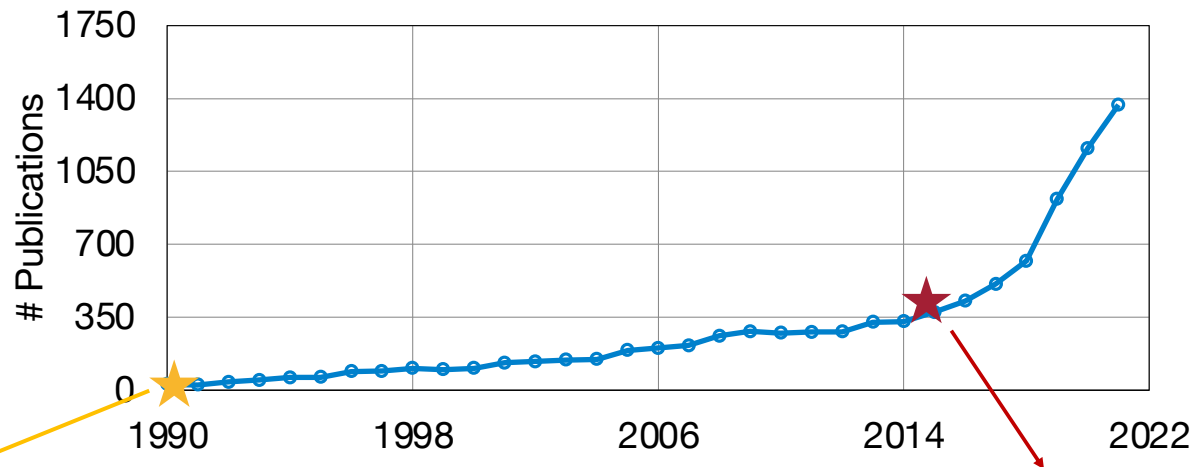


Baseline: a soccer player in red is running in the field.

Pruned 95%: a man **in a red shirt and black and white black shirt** is running through a field.

Neural Network Pruning

Make neural network smaller by removing synapses and neurons



598 Le Cun, Denker and Solla

Optimal Brain Damage

Yann Le Cun, John S. Denker and Sara A. Solla
AT&T Bell Laboratories, Holmdel, N. J. 07733

Learning both Weights and Connections for Efficient Neural Networks

Song Han
Stanford University
songhan@stanford.edu

Jeff Pool
NVIDIA
jpool@nvidia.com

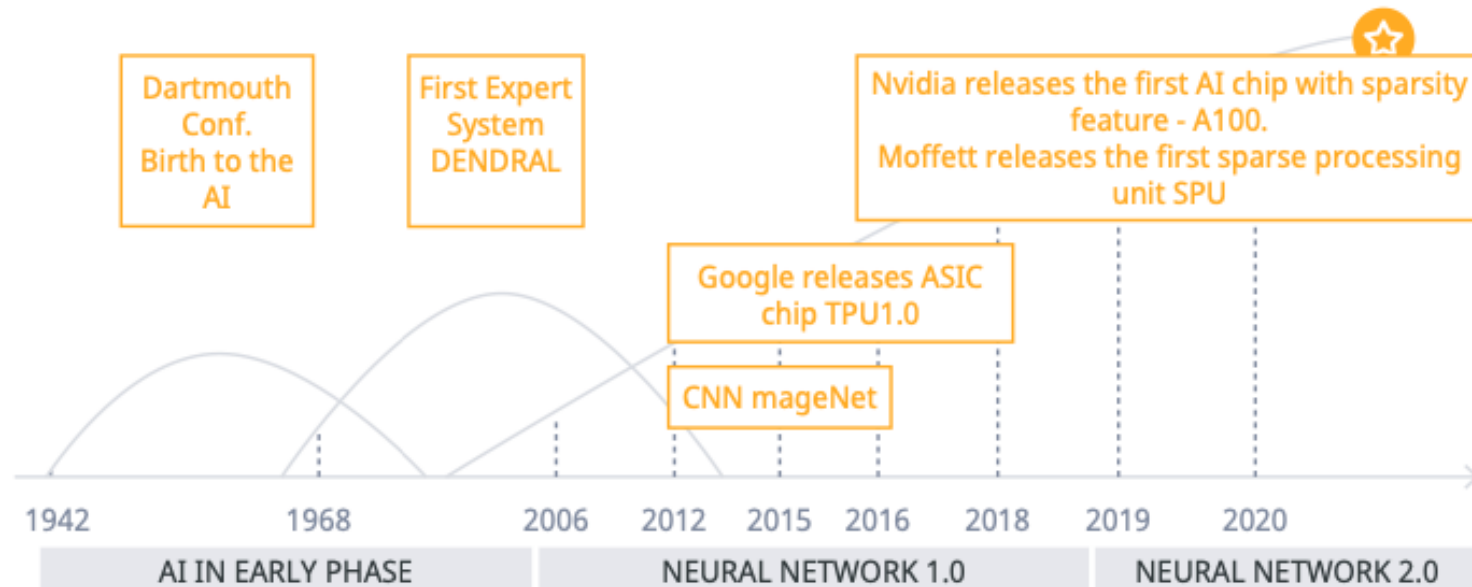
John Tran
NVIDIA
johntran@nvidia.com

William J. Dally
Stanford University
NVIDIA
dally@stanford.edu

Source: [Dimension.ai](https://dimension.ai)

Sparsity in the Industry

A milestone in AI 2.0: On May 15, 2020, Nvidia launched A100 with 2× Sparsity



The commercialization of sparsity has started,
and it will continue to lead the future

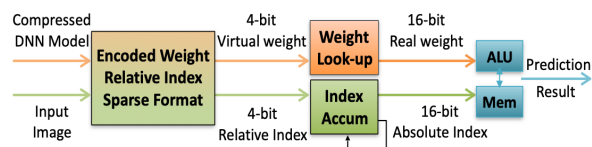
https://www.kisacoresearch.com/sites/default/files/documents/moffett_ai_introduction_to_sparse_computing_whitepaper.pdf

Pruning in the Industry

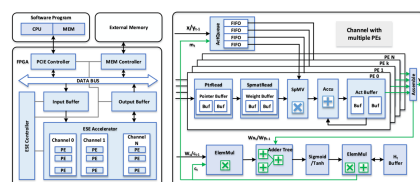
Hardware support for sparsity



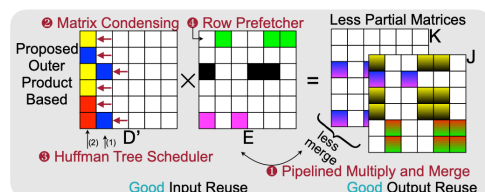
NVIDIA



EIE [Han *et al.*, ISCA 2016]

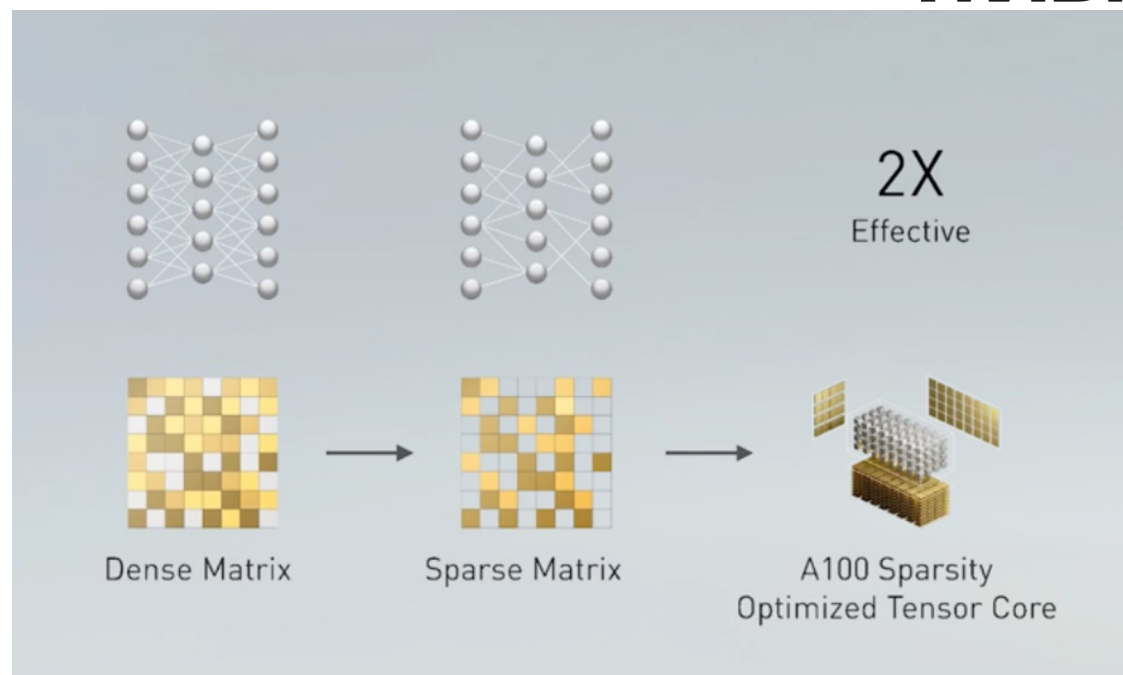


ESE [Han *et al.*, FPGA 2017]



SpArch [Zhang *et al.*, HPCA 2020]

SpAtten [Wang *et al.*, HPCA 2021]

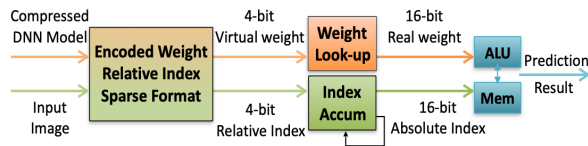


2:4 sparsity in A100 GPU

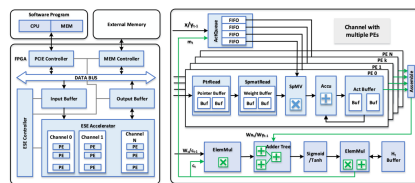
2X peak performance, 1.5X measured BERT speedup

<https://efficientml.ai>

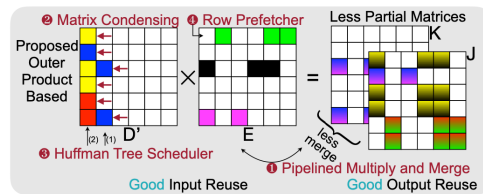
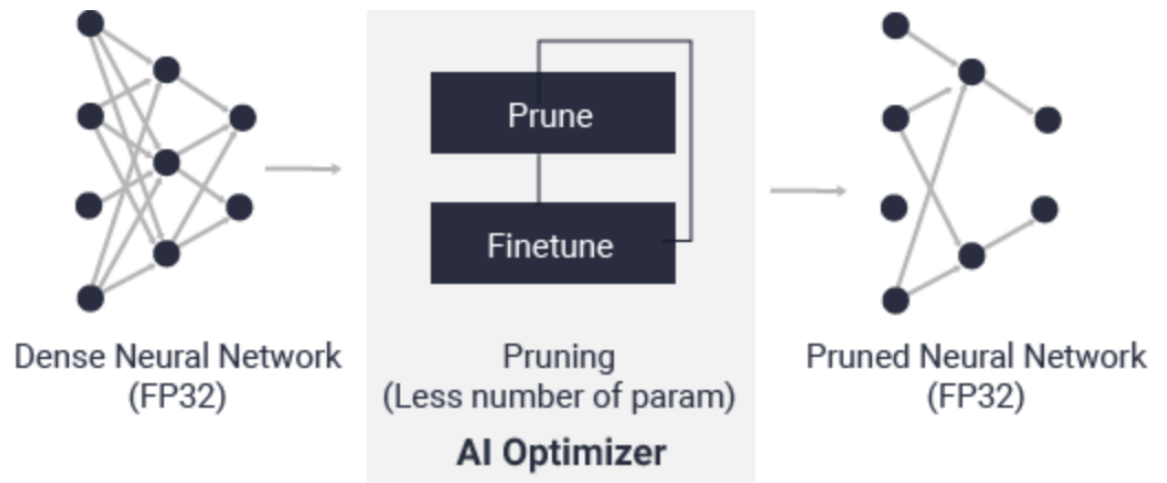
Hardware support for sparsity



EIE [Han *et al.*, ISCA 2016]



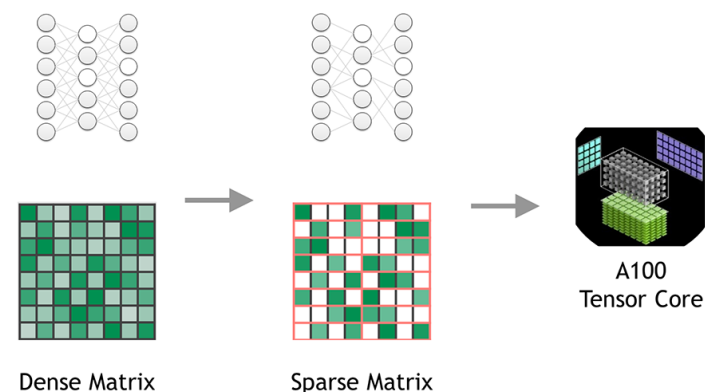
ESE [Han *et al.*, FPGA 2017]

SpArch [Zhang *et al.*, HPCA 2020]SpAtten [Wang *et al.*, HPCA 2021]

Reduce model complexity by 5x to 50x with minimal accuracy impact

NVIDIA GPU Support for Sparsity

- **Specialized Hardware:** NVIDIA GPUs with Tensor Cores (e.g., A100, H100, Blackwell B200) optimized for sparse matrix operations, enhancing sparse neural network performance.
- **Software and Library Support:** NVIDIA's cuSPARSE in CUDA, TensorFlow, and PyTorch provide software-level optimizations for sparse matrices.
- **Basic Usage:** No special flags are needed for general operations; automatic GPU acceleration in frameworks.
- **Advanced Optimizations:** Utilization of specific libraries/functions for custom sparse operations (e.g., [cuSPARSE](#)).
- **NVIDIA's Sparse Tensor Core API:** Specialized APIs for recent GPUs for more efficient sparse operations; requires specific API usage.
- **Recommendation:** Consult GPU and framework documentation for optimal use and leveraging sparsity support; evolving field with regular enhancements in hardware and software.



<https://developer.nvidia.com/blog/accelerating-inference-with-sparsity-using-ampere-and-tensorrt/>

Accelerators HW Support:

Evolving advancements in specialized AI hardware for sparsity support.

- **NVIDIA GPUs with Tensor Cores:**
 - Models: A100, H100, B200 (Ampere/Hopper/Blackwell)
 - Key Feature: Efficient handling of sparse matrices
- **Intel AI Accelerators (Habana Labs):**
 - Example: Intel Gaudi 2/Gaudi 3 (Habana Labs)
 - Focus: Dedicated AI accelerators with sparsity-aware compute
- **Google TPUs (Tensor Processing Units):**
 - Designed for TensorFlow operations
 - Application: High performance in sparse neural network computations
- **Graphcore IPU (Intelligence Processing Unit):**
 - Specialization: Efficient handling of sparse network computations
 - Benefit: High performance in both training and inference
- **AMD GPUs:**
 - Notable Architecture: CDNA3 (Instinct MI300X/MI325X)
 - Capability: Native structured sparsity in matrix cores, 2x throughput
- **FPGAs (Field-Programmable Gate Arrays):**
 - Customizability: Optimized for specific tasks, including sparsity
- **Apple Silicon (M3/M4 chips):**
 - Integration: Machine learning accelerators for neural network computations
 - Use: In Macs and iPads, potentially support sparse networks
- **Cerebras Systems and SambaNova:**
 - Cerebras: Efficient at large-scale neural network computations
 - SambaNova: Dataflow architecture for enterprise AI inference

Latest GPU Accelerator Support for Sparsity

Comparative Overview (as of 2026)

NVIDIA H100 (Hopper)

- Structured sparsity pattern: 2:4 (two zeros per four weights)
 - 2× theoretical throughput for compatible layers
- Up to ~1.5× real-world speedup on BERT, LLaMA FP8 workloads
 - Supported through cuSPARSELt, TensorRT, PyTorch Sparse

AMD Instinct MI300 Series (CDNA3)

- Native hardware sparsity support in matrix cores
- FP16 throughput: 1307 TFLOPS → 2615 TFLOPS (with sparsity)
 - Claimed structured sparsity support similar to NVIDIA 2:4
 - ROCm stack evolving for sparse kernel support

Key Takeaways

- Structured 2:4 sparsity remains the de facto pattern.
 - Gains depend on model alignment and library support.
 - Training cost, accuracy trade-offs, and metadata overhead persist.
 - NVIDIA Blackwell (B200/GB200) continues 2:4 support + adds FP4/MXFP4 precision.
- Emerging direction: N:M flexible sparsity + FP8/FP4 data types + Rubin (2026).
-

Neural Network Pruning

- **Introduction to Pruning**

- What is pruning?
- How should we formulate pruning?

- **Determine the Pruning Granularity**

- In what pattern should we prune the neural network?

- **Determine the Pruning Criterion**

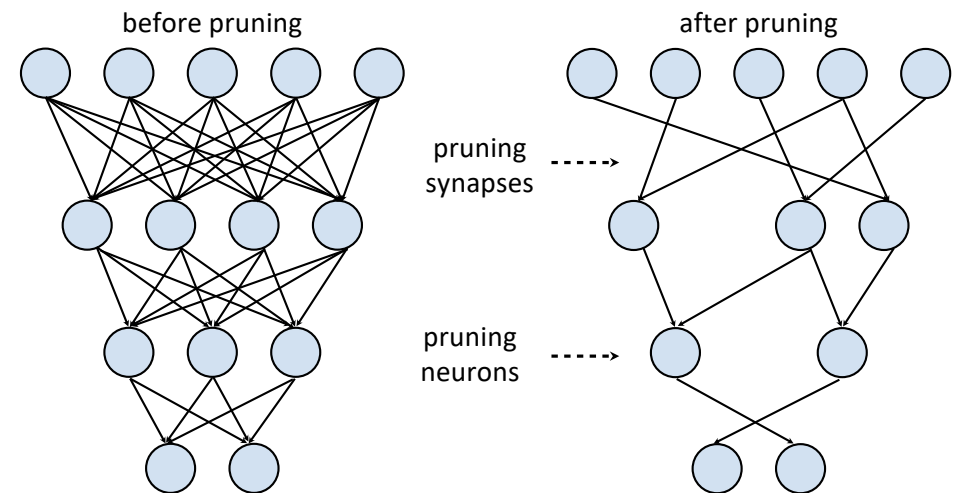
- What synapses/neurons should we prune?

- **Determine the Pruning Ratio**

- What should target sparsity be for each layer?

- **Fine-tune/Train Pruned Neural Network**

- How should we improve performance of pruned models?



- Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

Neural Network Pruning

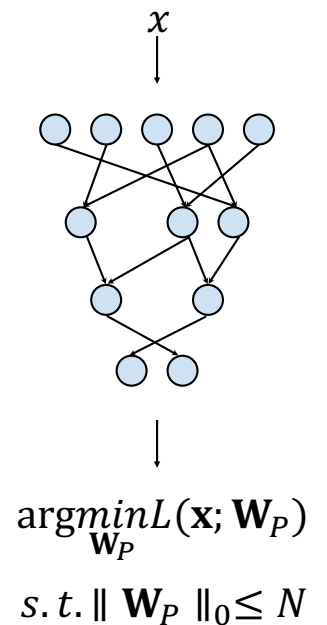
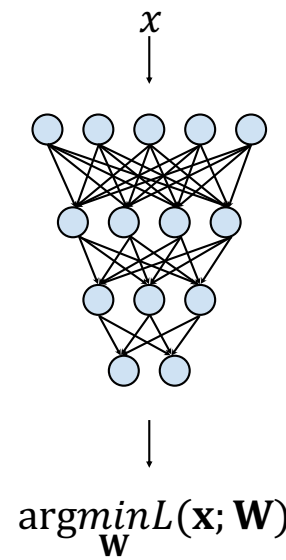
- In general, we could formulate the pruning as follows:

$$\operatorname{argmin}_{\mathbf{W}_P} L(\mathbf{x}; \mathbf{W}_P)$$

subject to

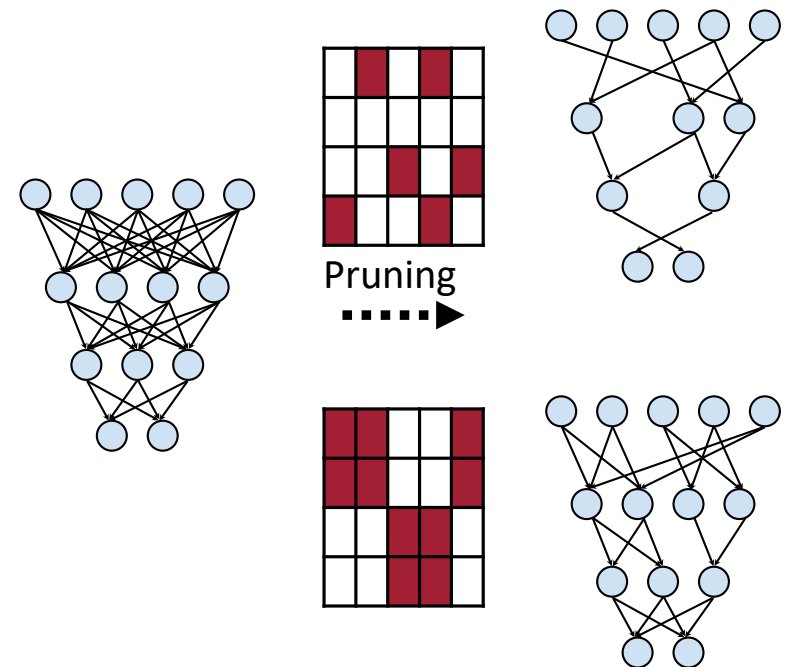
$$\|\mathbf{W}_P\|_0 \leq N$$

- L represents the objective function for neural network training;
- $\mathbf{x}$ is input, $\mathbf{W}$ is original weights, $\mathbf{W}_P$ is pruned weights;
- $\|\mathbf{W}_P\|_0$ calculates the #nonzeros in $\mathbf{W}_P$, and N is the target #nonzeros.



Neural Network Pruning

- **Introduction to Pruning**
 - What is pruning?
 - How should we formulate pruning?
- **Determine the Pruning Granularity**
 - In what pattern should we prune the neural network?
- **Determine the Pruning Criterion**
 - What synapses/neurons should we prune?
- **Determine the Pruning Ratio**
 - What should target sparsity be for each layer?
- **Fine-tune/Train Pruned Neural Network**
 - How should we improve performance of pruned models?



Neural Network Pruning

- **Introduction to Pruning**

- What is pruning?
- How should we formulate pruning?

- **Determine the Pruning Granularity**

- In what pattern should we prune the neural network?

- **Determine the Pruning Criterion**

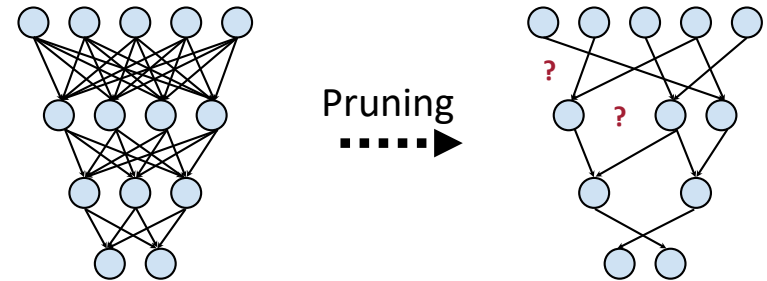
- What synapses/neurons should we prune?

- **Determine the Pruning Ratio**

- What should target sparsity be for each layer?

- **Fine-tune/Train Pruned Neural Network**

- How should we improve performance of pruned models?



which synapses?
which neurons?

- Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

Neural Network Pruning

- **Introduction to Pruning**

- What is pruning?
- How should we formulate pruning?

- **Determine the Pruning Granularity**

- In what pattern should we prune the neural network?

- **Determine the Pruning Criterion**

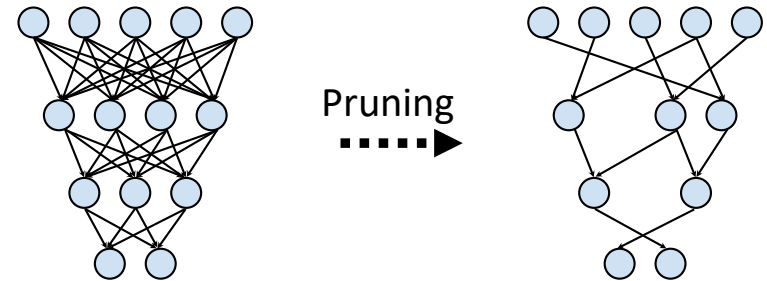
- What synapses/neurons should we prune?

- **Determine the Pruning Ratio**

- What should target sparsity be for each layer?

- **Fine-tune/Train Pruned Neural Network**

- How should we improve performance of pruned models?



prune 30%?
prune 50%?
prune 70%?

Neural Network Pruning

- **Introduction to Pruning**

- What is pruning?
- How should we formulate pruning?

- **Determine the Pruning Granularity**

- In what pattern should we prune the neural network?

- **Determine the Pruning Criterion**

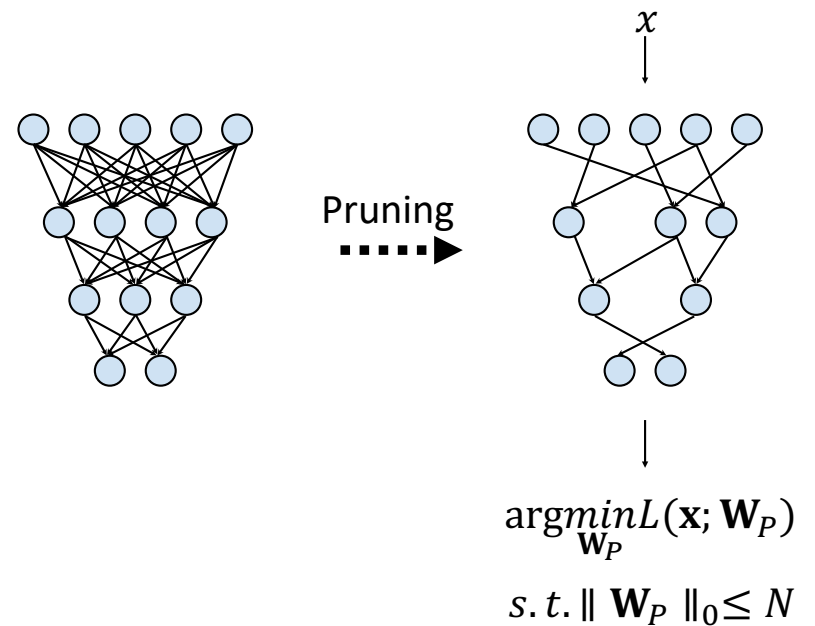
- What synapses/neurons should we prune?

- **Determine the Pruning Ratio**

- What should target sparsity be for each layer?

- **Fine-tune/Train Pruned Neural Network**

- How should we improve performance of pruned models?



Neural Network Pruning

- **Introduction to Pruning**

- What is pruning?
- How should we formulate pruning?

- **Determine the Pruning Granularity**

- In what pattern should we prune the neural network?

- **Determine the Pruning Criterion**

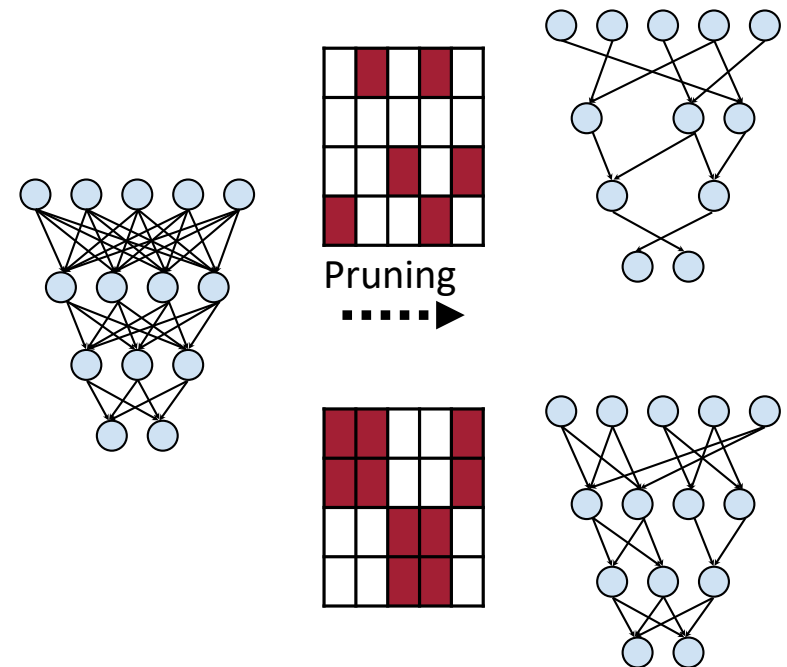
- What synapses/neurons should we prune?

- **Determine the Pruning Ratio**

- What should target sparsity be for each layer?

- **Fine-tune/Train Pruned Neural Network**

- How should we improve performance of pruned models?

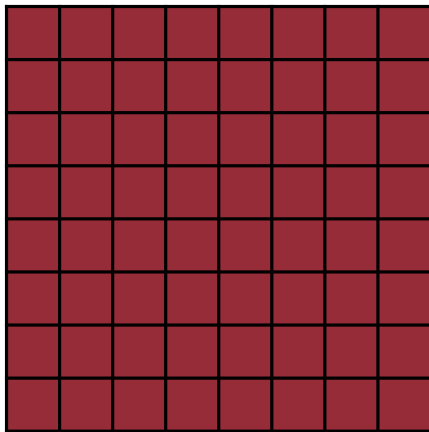


Section 2: Pruning Granularity

Pruning can be performed at different granularities, from structured to non-structured.

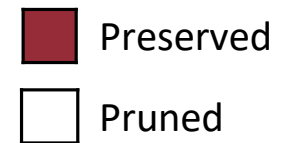
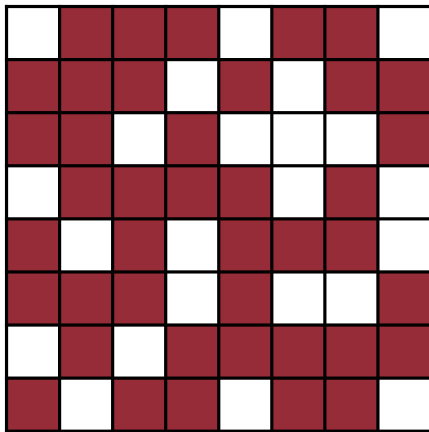
Pruning at Different Granularities

A simple example of 2D weight matrix



Pruning at Different Granularities

A simple example of 2D weight matrix



Fine-grained/Unstructured

- More flexible pruning index choice
- Hard to accelerate (irregular data expression)

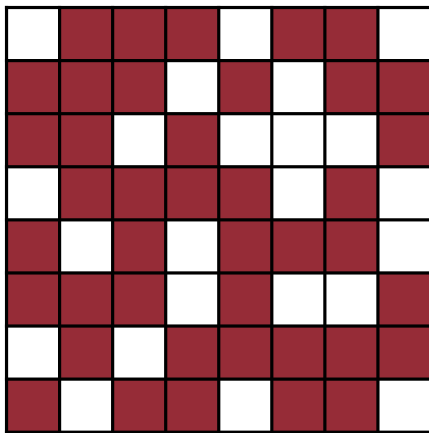
Bad for



<https://efficientml.ai>

Pruning at Different Granularities

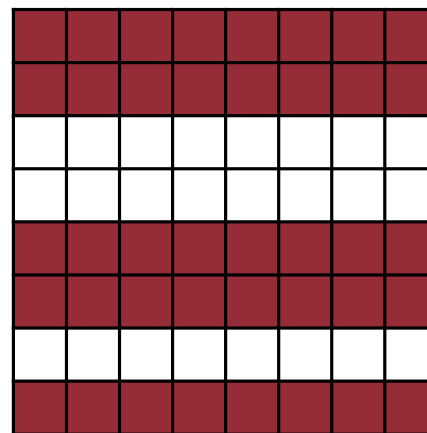
A simple example of 2D weight matrix



Fine-grained/Unstructured

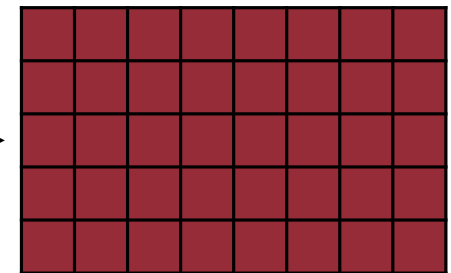
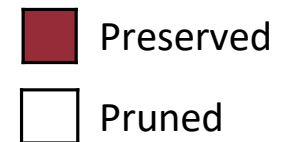
- More flexible pruning index choice
- Hard to accelerate (irregular data expression)

Bad for



Coarse-grained/Structured

- Less flexible pruning index choice (a subset of the fine-grained case)
- Easy to accelerate (just a smaller matrix!)



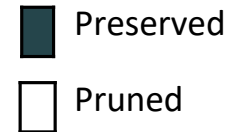
Pruning at Different Granularities

The case of convolutional layers

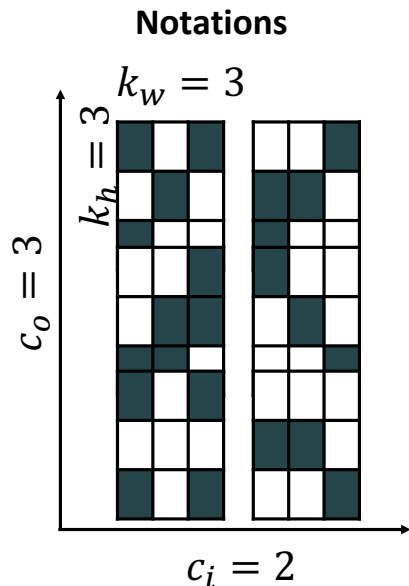
- The weights of convolutional layers have 4 dimensions $[c_o, c_i, k_h, k_w]$:
 - c_i : input channels (or channels)
 - c_o : output channels (or filters)
 - k_h : kernel size height
 - k_w : kernel size width
- The 4 dimensions give us more choices to select pruning granularities

Pruning at Different Granularities

The case of convolutional layers



- Some of the commonly used pruning granularities

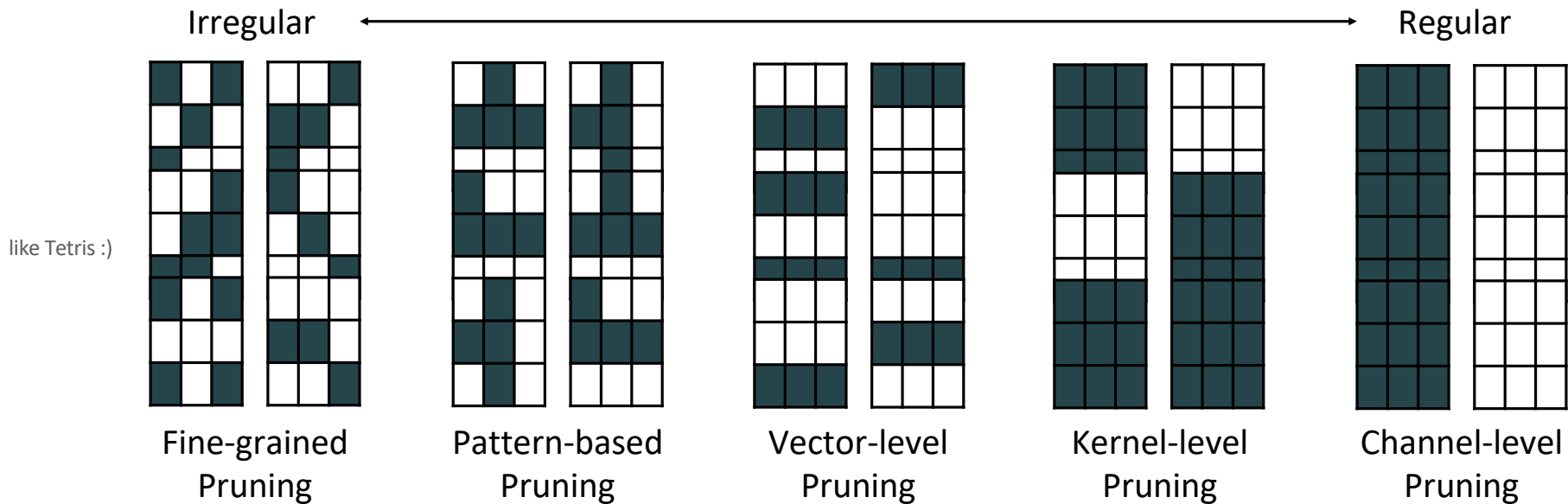
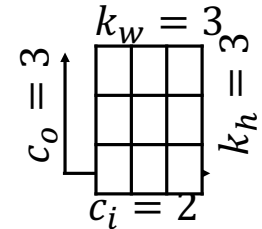
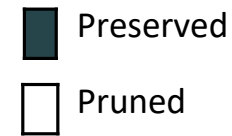


- The weights of convolutional layers have 4 dimensions $[c_o, c_i, k_h, k_w]$:
 - c_i : input channels (or channels)
 - c_o : output channels (or filters)
 - k_h : kernel size height
 - k_w : kernel size width
- The 4 dimensions give us more choices to select pruning granularities

Pruning at Different Granularities

The case of convolutional layers

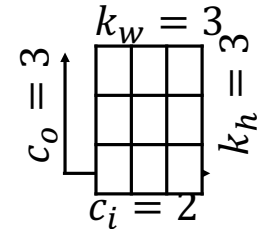
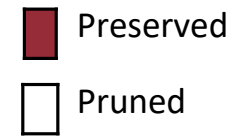
- Some of the commonly used pruning granularities



Pruning at Different Granularities

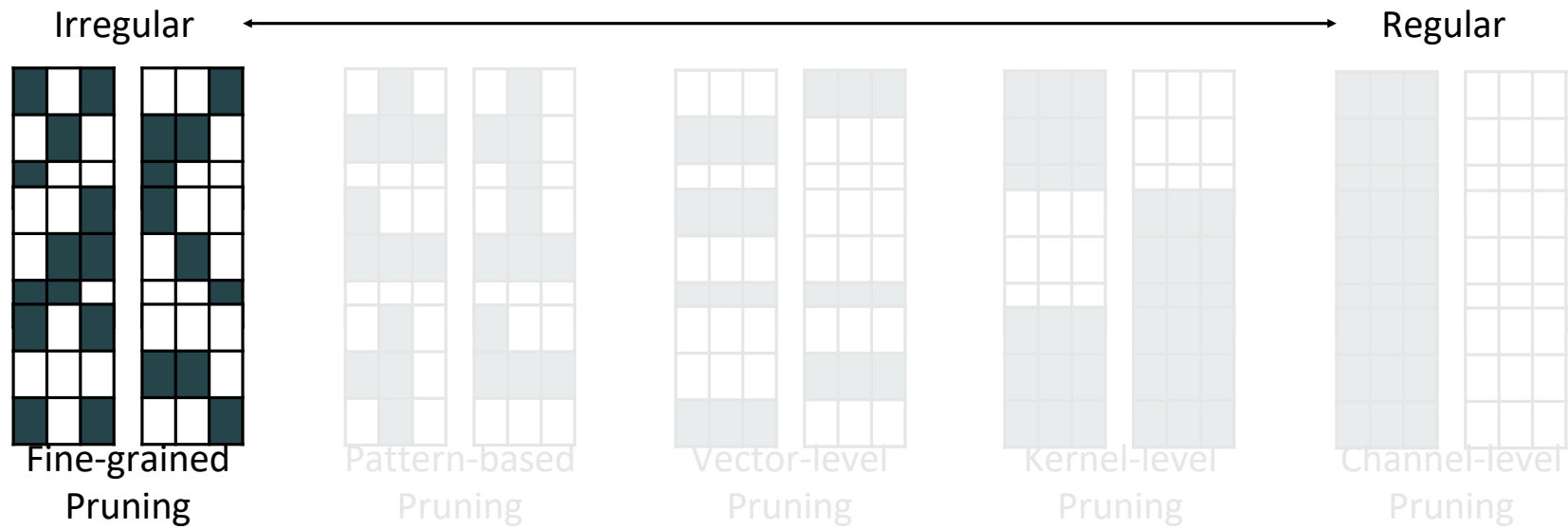
The case of convolutional layers

- Some of the commonly used pruning granularities



Pros?

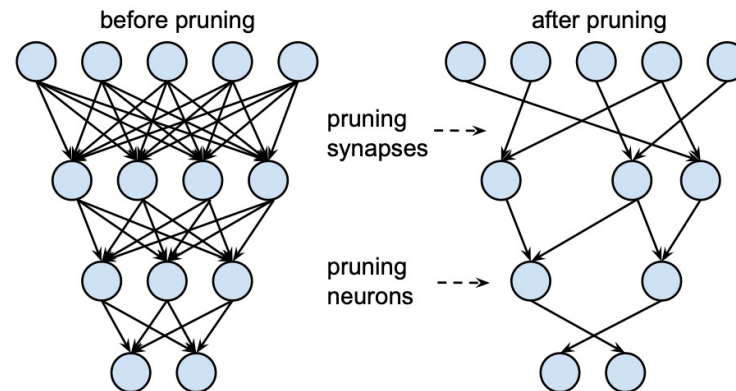
Cons?



Pruning at Different Granularities

Let's look into some cases

- **Fine-grained Pruning** (the case we show before)
 - Flexible pruning indices



Pruning at Different Granularities

Let's look into some cases

- **Fine-grained Pruning** (the case we show before)
 - Flexible pruning indices
 - Usually larger compression ratio since we can flexibly find “redundant” weights (we will later discuss how we find them)

Neural Network	#Parameters		
	Before Pruning	After Pruning	Reduction
AlexNet	61 M	6.7 M	9 ×
VGG-16	138 M	10.3 M	12 ×
GoogleNet	7 M	2.0 M	3.5 ×
ResNet50	26 M	7.47 M	3.4 ×

Pruning at Different Granularities

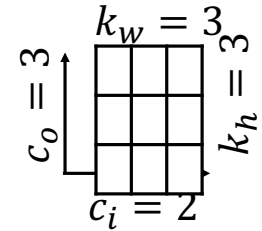
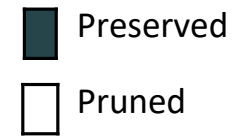
Let's look into some cases

- **Fine-grained Pruning** (the case we show before)
 - Flexible pruning indices
 - Usually larger compression ratio since we can flexibly find “redundant” weights (we will later discuss how we find them)
 - Can deliver speed up on some custom hardware (e.g., EIE) but not GPU (easily)

Pruning at Different Granularities

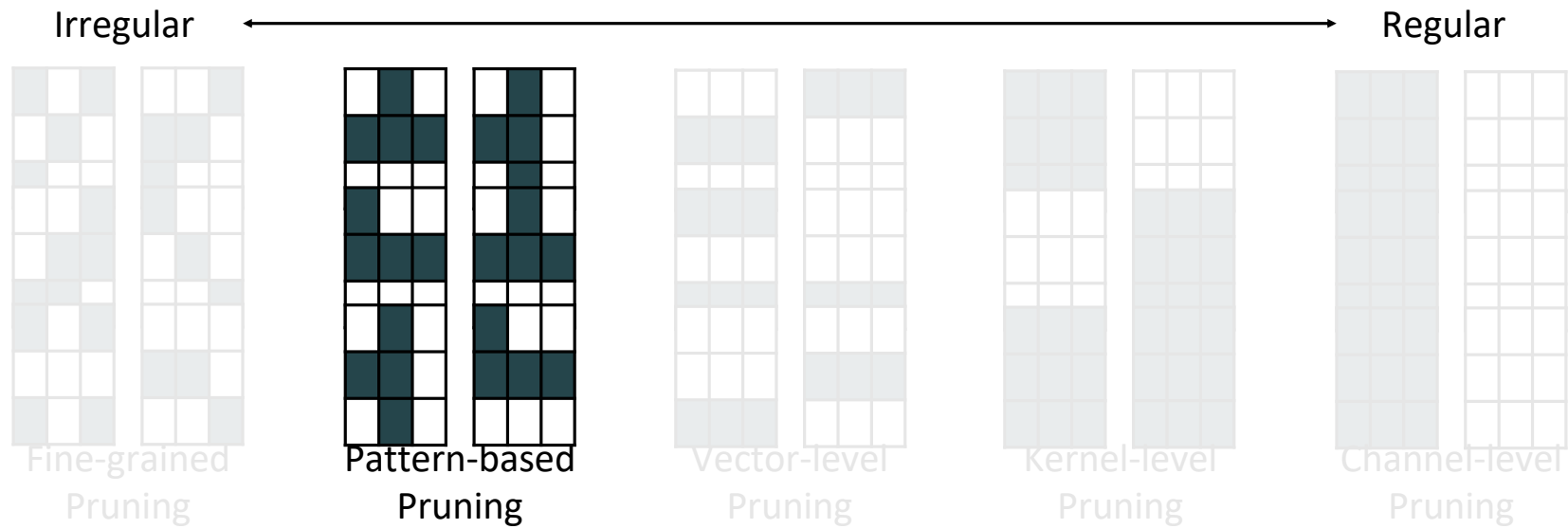
The case of convolutional layers

- Some of the commonly used pruning granularities



Pros?

Cons?

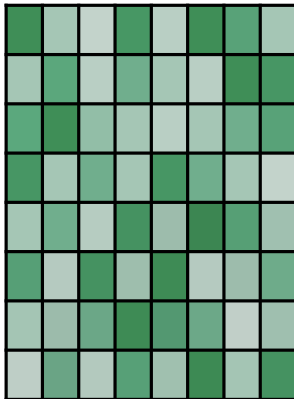


Pruning at Different Granularities

Let's look into some cases

- **Pattern-based Pruning: N:M sparsity**
 - N:M sparsity means that in each contiguous M elements, N of them is pruned

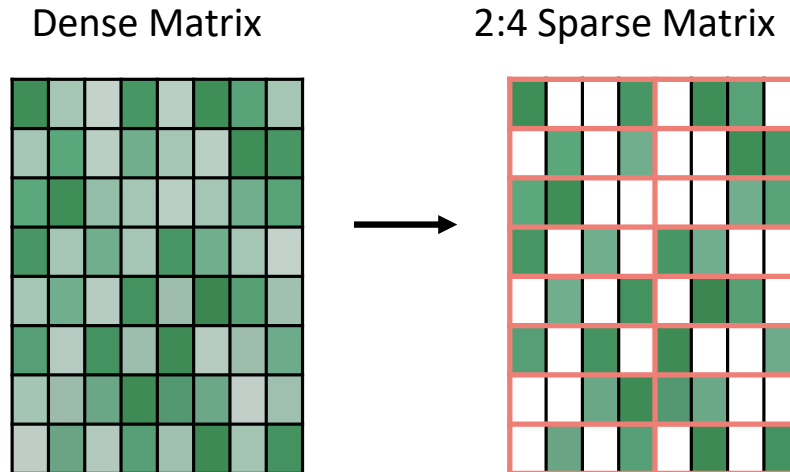
Dense Matrix



Pruning at Different Granularities

Let's look into some cases

- **Pattern-based Pruning: N:M sparsity**
 - N:M sparsity means that in each contiguous M elements, N of them is pruned
 - A classic case is 2:4 sparsity (50% sparsity) (available in A100 GPUs)



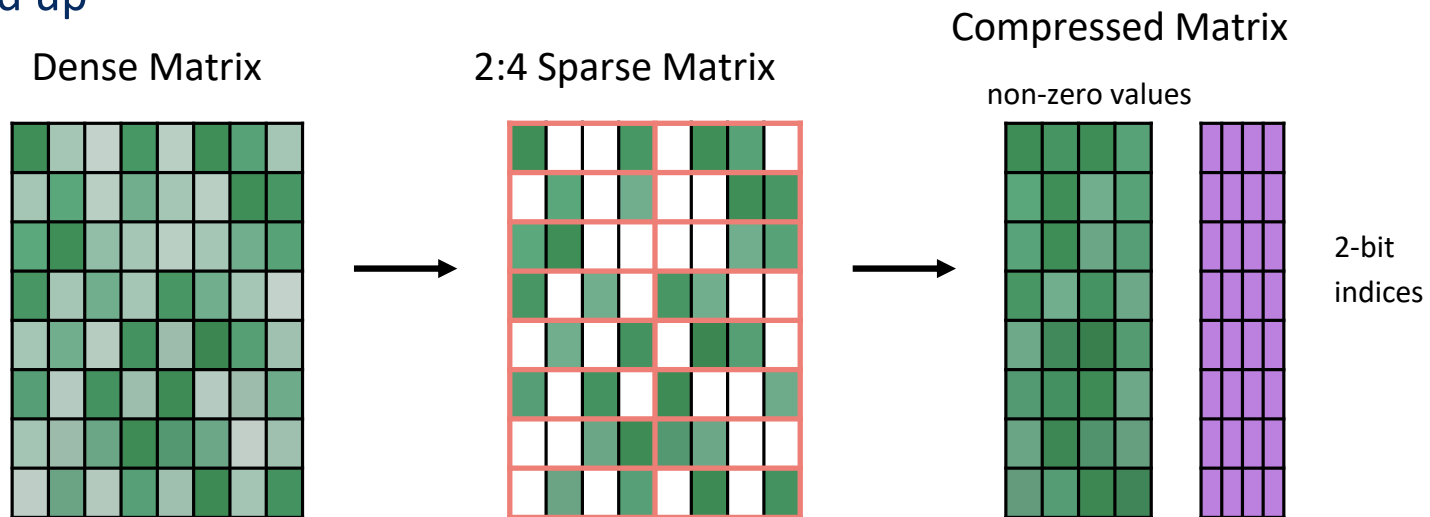
[Accelerating Inference with Sparsity Using the NVIDIA Ampere Architecture and NVIDIA TensorRT](#)

Pruning at Different Granularities

Let's look into some cases

- **Pattern-based Pruning: N:M sparsity**

- N:M sparsity means that in each contiguous M elements, N of them is pruned
- A classic case is 2:4 sparsity (50% sparsity)
- It is supported by NVIDIA's Ampere GPU Architecture, which delivers up to 2x speed up



[Accelerating Inference with Sparsity Using the NVIDIA Ampere Architecture and NVIDIA TensorRT](#)

Pruning at Different Granularities

Let's look into some cases

- **Pattern-based Pruning: N:M sparsity**

- N:M sparsity means that in each contiguous M elements, N of them is pruned
- A classic case is 2:4 sparsity (50% sparsity)
- It is supported by NVIDIA's Ampere GPU Architecture, which delivers ~2x speed up
- Usually maintains accuracy (tested on varieties of tasks)

Network	Data Set	Metric	Dense FP16	Sparse FP16
ResNet-50	ImageNet	Top-1	76.1	76.2
ResNeXt-101_32x8d	ImageNet	Top-1	79.3	79.3
Xception	ImageNet	Top-1	79.2	79.2
SSD-RN50	COC02017	bbAP	24.8	24.8
MaskRCNN-RN50	COC02017	bbAP	37.9	37.9
FairSeq Transformer	EN-DE WMT'14	BLEU	28.2	28.5
BERT-Large	SQuAD v1.1	F1	91.9	91.9

[Accelerating Inference with Sparsity Using the NVIDIA Ampere Architecture and NVIDIA TensorRT](#)

Pruning at Different Granularities

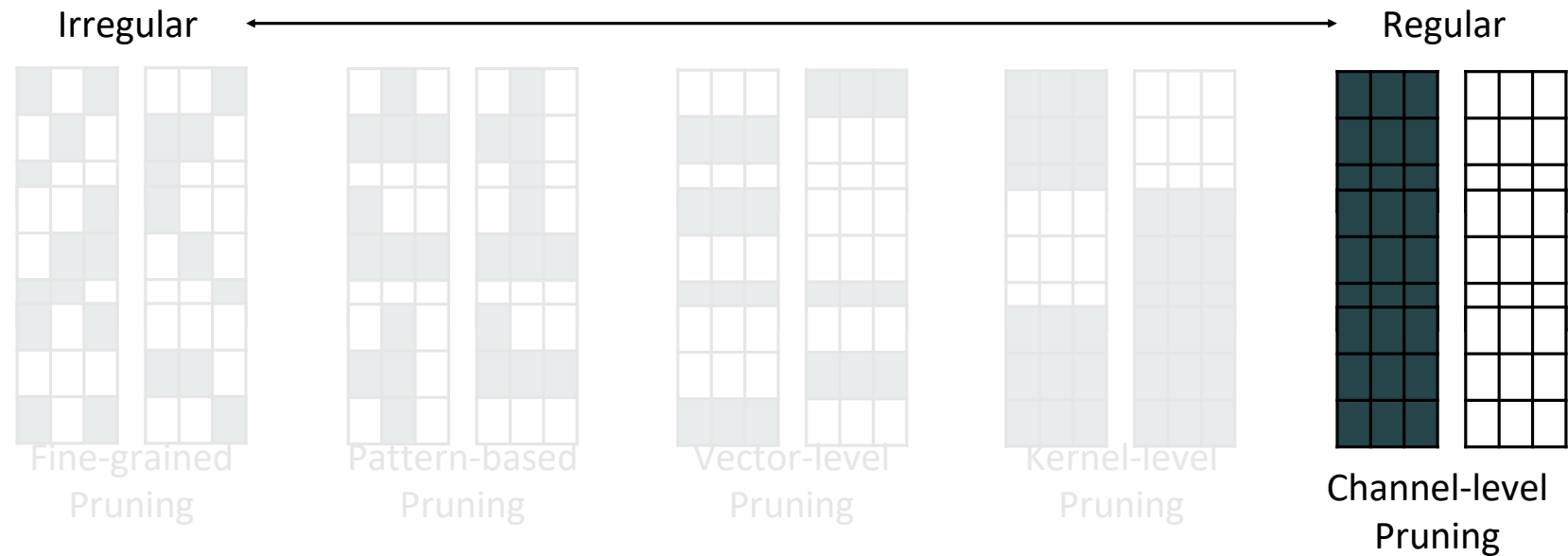
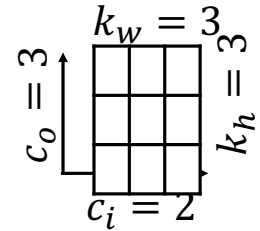
The case of convolutional layers

- Some of the commonly used pruning granularities

Pros?

Cons?

■ Preserved
□ Pruned

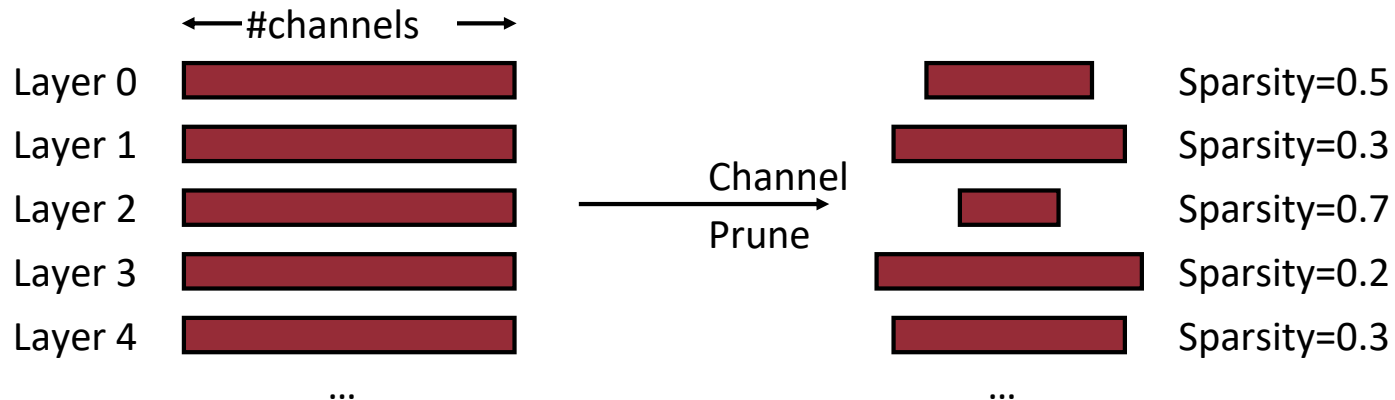


Pruning at Different Granularities

Let's look into some cases

- **Channel Pruning**

- **Pro:** Direct speed up due to reduced channel numbers (leading to an NN with smaller #channels)
- **Con:** smaller compression ratio
- Most popular on CPUs and off the shelf hardware



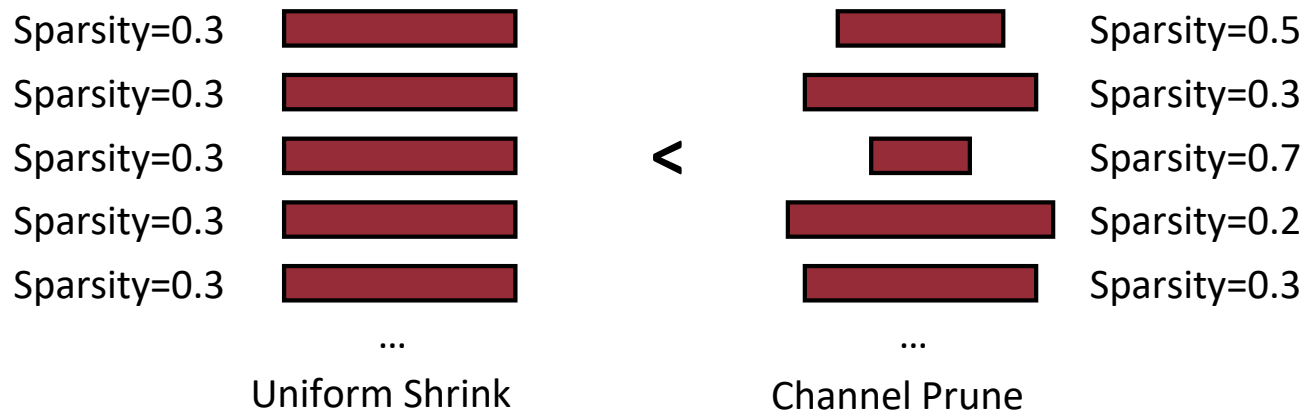
Pruning at Different Granularities

Let's look into some cases

- **Channel Pruning**

- **Pro:** Direct speed up due to reduced channel numbers (leading to an NN with smaller #channels)
- **Con:** smaller compression ratio
- Most popular on CPUs and off the shelf hardware

We will later discuss how to find sparsity ratios

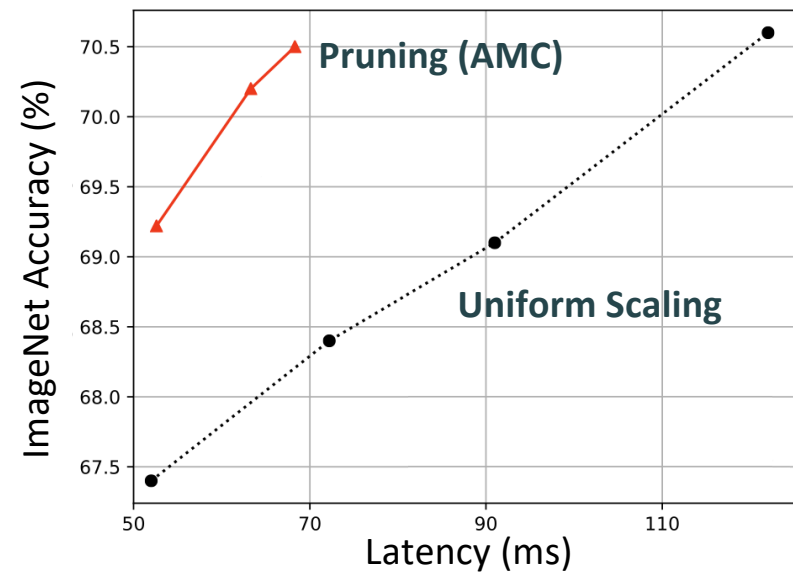
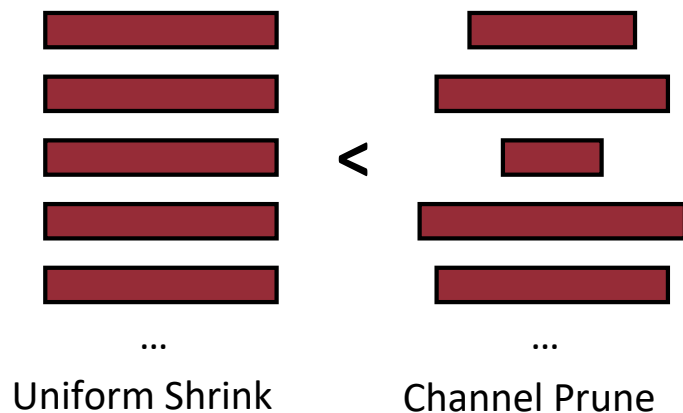


Pruning at Different Granularities

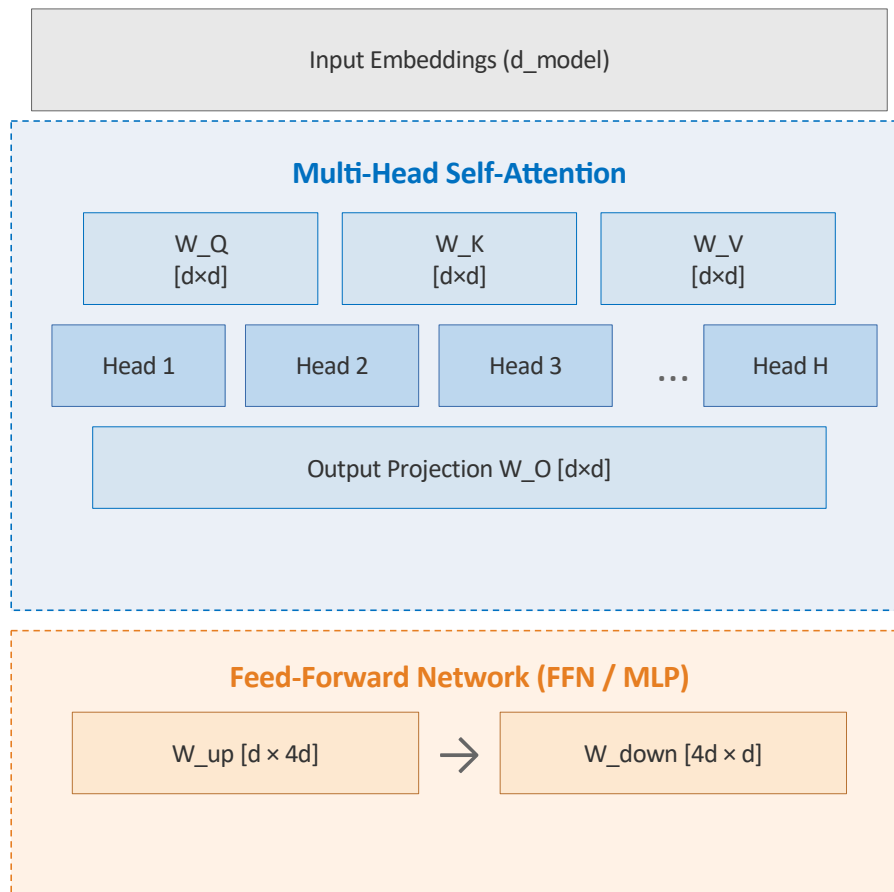
Let's look into some cases

- **Channel Pruning**

- Pro: Direct speed up due to reduced channel numbers (leading to an NN with smaller #channels)
- Con: smaller compression ratio



Anatomy of a Transformer Block: Where Can We Prune?



Prunable Components

1. Individual Weights

Fine-grained: any element in W_Q , W_K , W_V , W_{up} , W_{down}

2. Attention Heads

Remove entire heads (structured). Each head = d/H dimensions in Q, K, V .

3. FFN Intermediate Dims

Width pruning: reduce $4d$ intermediate dimension in MLP block.

4. Entire Transformer Layers

Depth pruning: remove full blocks (Llama 2: $80 \rightarrow 32$ layers).

5. N:M Structured (2:4)

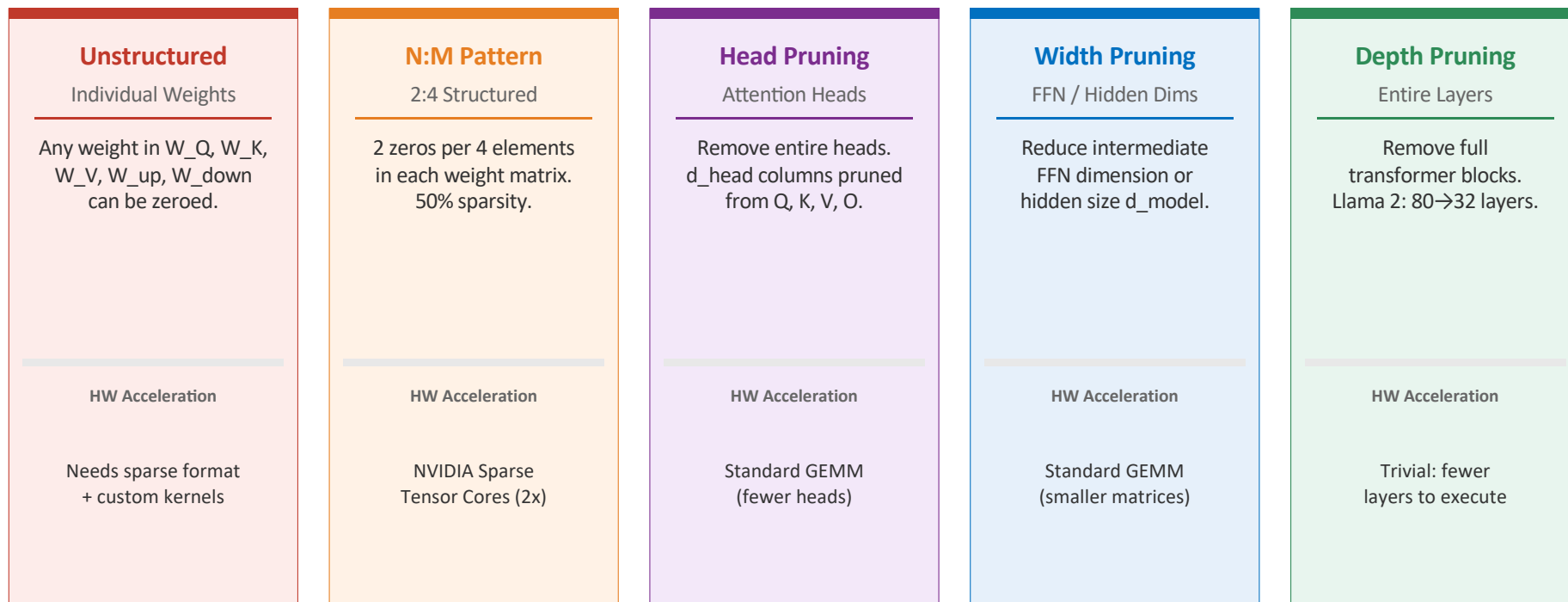
Pattern-based on $W_Q/K/V/\text{up/down}$ matrices. HW-accelerated.

Pruning at Different Granularities: Transformer Edition

The same irregular $\leftrightarrow$ regular trade-off applies, but with transformer-specific components

◀ Irregular

Regular ▶



Flexibility:

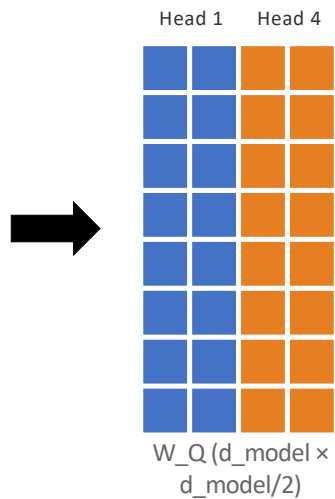
Highest ————— Lowest

Head Pruning: Removing Entire Attention Heads

Before Pruning



After Pruning (Head 2 & 3 removed)



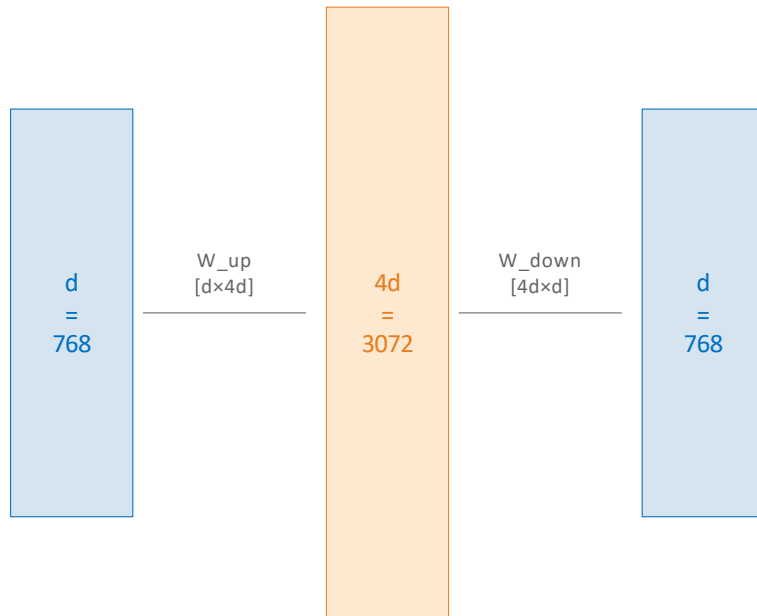
Key Properties

- Structured: entire d_{head} columns removed from W_Q , W_K , W_V , and W_O simultaneously
- Reduces computation proportionally to heads removed (e.g., 50% heads = ~50% attention cost)
- Many heads are redundant: Voita et al. (2019) showed only a few heads are critical in NMT
- Michel et al. (2019): BERT can lose 20-40% of heads with <1% accuracy drop on GLUE
- Direct GPU speedup: smaller matrices = standard GEMM, no special sparse format needed

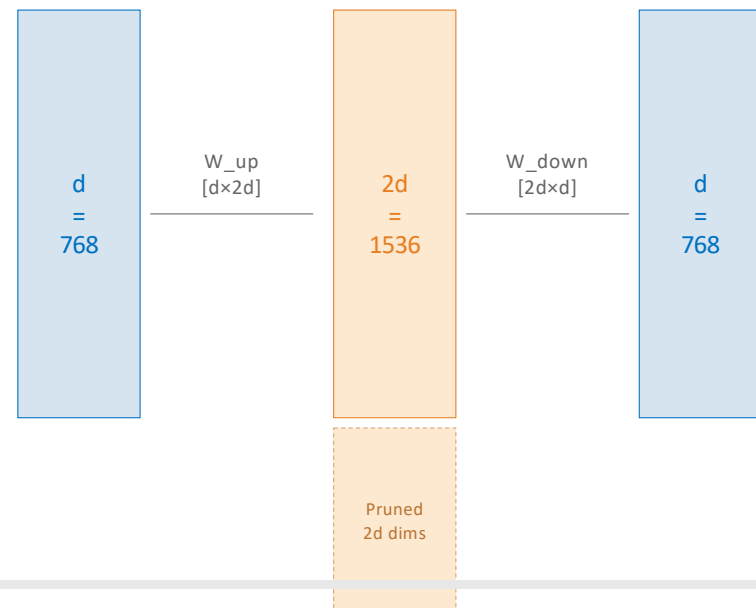
Are Sixteen Heads Really Better than One? [Michel et al., NeurIPS 2019]

Width Pruning: Reducing FFN Intermediate Dimensions

Before: Standard FFN Block



After: Pruned FFN (Width = $2d$)



Why it works: FFN layers consume $\sim 2/3$ of transformer FLOPs. Reducing intermediate dim from $4d$ to $2d$ cuts FFN cost by 50%.

Llama 2 example: NVIDIA reduced intermediate dims from 28,762 to 14,336 (width pruning) alongside depth pruning for 2.5x total speedup.

HW friendly: Result is a standard dense matrix, just smaller. Works on any GPU/CPU with no special sparse support needed.

Depth Pruning: Removing Entire Transformer Layers



Depth Pruning in Practice

- Coarsest granularity: remove entire transformer blocks
- NVIDIA Llama 2 70B: 80 → 32 layers (60% reduction)
- Combined with width pruning: 2.5x speedup, 99% accuracy
- Key insight: middle layers are often most redundant
- First and last layers tend to be critical (like conv Layer 0)
- Layer importance can be measured by: output similarity, gradient magnitude, or perplexity impact
- Sheared-LLaMA (Xia et al., 2023): targeted structured pruning of depth + width + heads jointly

Sheared-LLaMA: Accelerating Language Model Pre-training via Structured Pruning [Xia et al., ICLR 2024]

N:M Structured Sparsity in Transformer Weight Matrices

Applying 2:4 sparsity to Q, K, V, and FFN weight matrices for hardware-accelerated LLM inference

W_Q with 2:4 Sparsity Pattern

0	0	w	w	w	0	0	w
w	0	0	w	w	w	0	0
w	w	0	0	0	w	w	0
0	w	w	0	0	w	0	w
0	w	0	w	w	0	w	0
w	0	w	0	0	0	w	w
0	0	w	w	w	0	0	w
w	0	0	w	w	w	0	0

Group 1: 2 of 4 = 0

Group 2: 2 of 4 = 0



Non-zero (kept)



Pruned (zero)

Where 2:4 Applies in a Transformer

W_Q, W_K, W_V

Query/Key/Value projections

~25% of params each

W_O

Output projection

~25% of attention params

W_{up} / W_{gate}

FFN up-projection

~33% of FFN params

W_{down}

FFN down-projection

~33% of FFN params

Results on LLMs

- BERT (2:4): <1% accuracy drop on SQuAD, 1.5x speedup on A100
- LLaMA-7B (2:4): ~1% perplexity increase, 1.3x end-to-end speedup
 - Combined with INT8/FP8: up to 2.37x throughput (torchao)
 - MLPerf 2024: TensorRT-LLM uses 2:4 sparsity on H100/H200

CNN vs. Transformer Pruning: Granularity Comparison

Granularity	CNN Equivalent	Transformer Equivalent	HW Acceleration
Fine-grained (Unstructured)	Individual conv kernel weights	Individual weights in W_Q/K/V/FFN	Custom HW only (EIE, etc.)
Pattern-based (N:M / 2:4)	N:M pattern on conv weight tensors	2:4 pattern on all transformer matrices	Sparse Tensor Cores (A100/H100/B200)
Kernel / Head Pruning	Remove entire conv kernels	Remove entire attention heads	Standard GEMM (smaller matrices)
Channel / Width Pruning	Remove output channels (filters)	Reduce FFN dims or d_model	Standard GEMM (smaller matrices)
Layer / Depth Pruning	Remove entire conv layers (rare)	Remove entire transformer blocks	Trivial: fewer layers to run

Key Insight for LLM Practitioners

Modern LLM optimization stacks multiple granularities: **2:4 structured sparsity** on weight matrices + **depth pruning** (remove layers) + **width pruning** (reduce FFN/hidden dims) + **quantization** (FP8/INT4). These techniques are complementary and often combined in production (e.g., NVIDIA MLPerf, Sheared-LLaMA).

Neural Network Pruning

- **Introduction to Pruning**

- What is pruning?
- How should we formulate pruning?

- **Determine the Pruning Granularity**

- In what pattern should we prune the neural network?

- **Determine the Pruning Criterion**

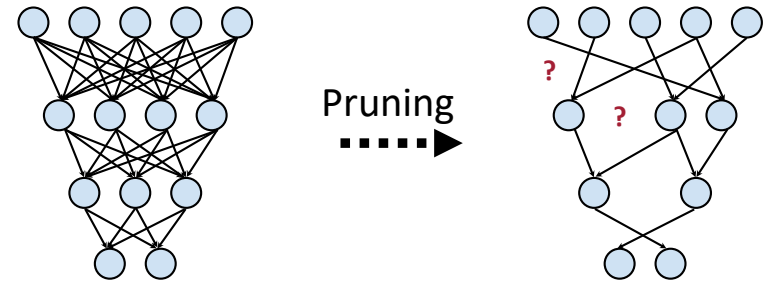
- What synapses/neurons should we prune?

- **Determine the Pruning Ratio**

- What should target sparsity be for each layer?

- **Fine-tune/Train Pruned Neural Network**

- How should we improve performance of pruned models?



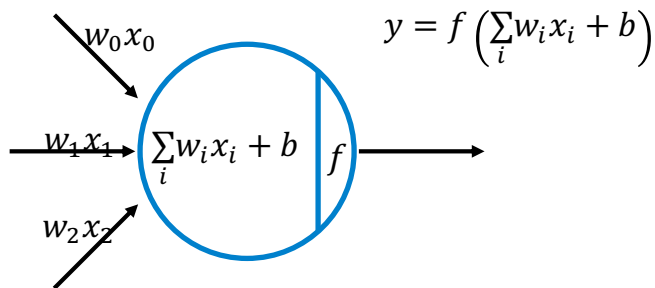
- Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

Section 3: Pruning Criterion

What synapses and neurons should we prune?

Selection of Synapses to Prune

- When removing parameters from a neural network model,
 - **the less important** the parameters being removed are,
 - the better the performance of pruned neural network is.



Example

$$f(\cdot) = \text{ReLU}(\cdot), W = [10, -8, 0.1]$$
$$y = \text{ReLU}(10x_0 - 8x_1 + 0.1x_2)$$

- If only one weight will be removed, which one? Why?
- If only one weight will be kept, which one? Why?

Magnitude-based Pruning

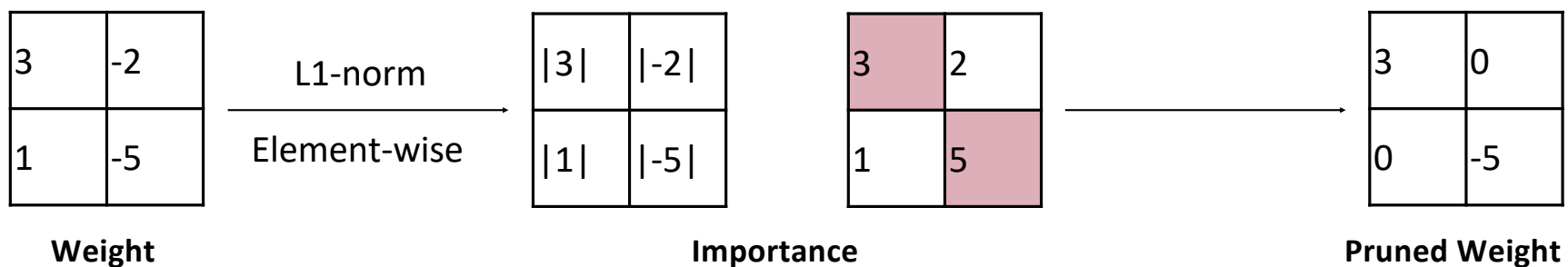
A heuristic pruning criterion

- Magnitude-based pruning considers weights with ***larger absolute values*** are more important than other weights.

- For element-wise pruning,

$$Importance = |W|$$

- **Example**



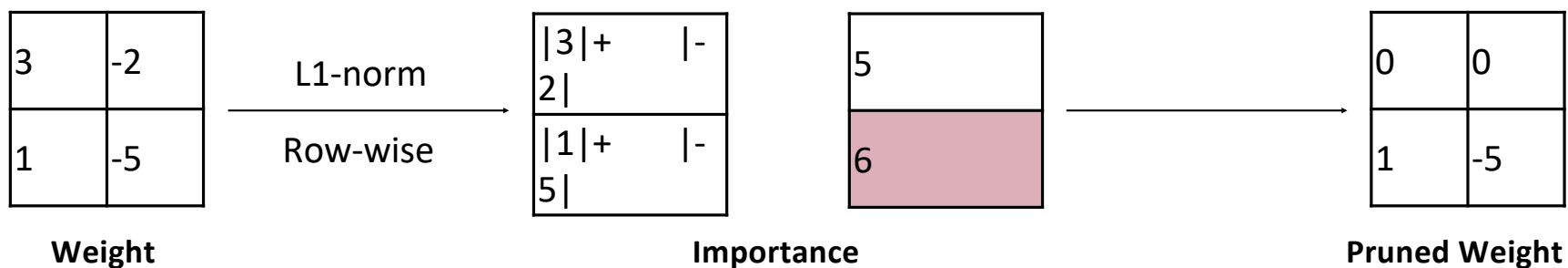
Magnitude-based Pruning

A heuristic pruning criterion

- Magnitude-based pruning considers weights with ***larger absolute values*** are more important than other weights.
 - For row-wise pruning, the L1-norm magnitude can be defined as,

$$\text{Importance} = \sum_{i \in S} |w_i|, \text{ where } \mathbf{W}^{(S)} \text{ is the structural set } S \text{ of parameters } \mathbf{W}$$

- **Example**



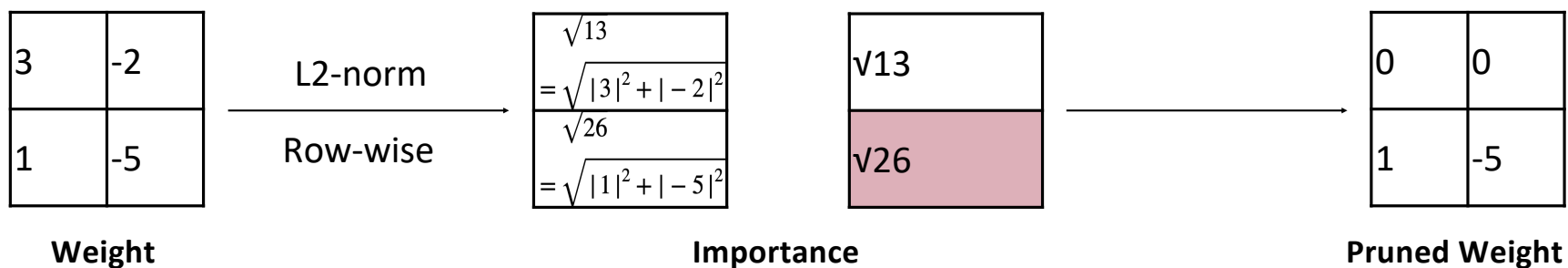
Magnitude-based Pruning

A heuristic pruning criterion

- Magnitude-based pruning considers weights with ***larger absolute values*** are more important than other weights.
 - For row-wise pruning, the L2-norm magnitude can be defined as,

$$\text{Importance} = \sqrt{\sum_{i \in S} |w_i|^2}, \text{ where } \mathbf{W}^{(S)} \text{ is the structural set } S \text{ of parameters } \mathbf{W}$$

- **Example**



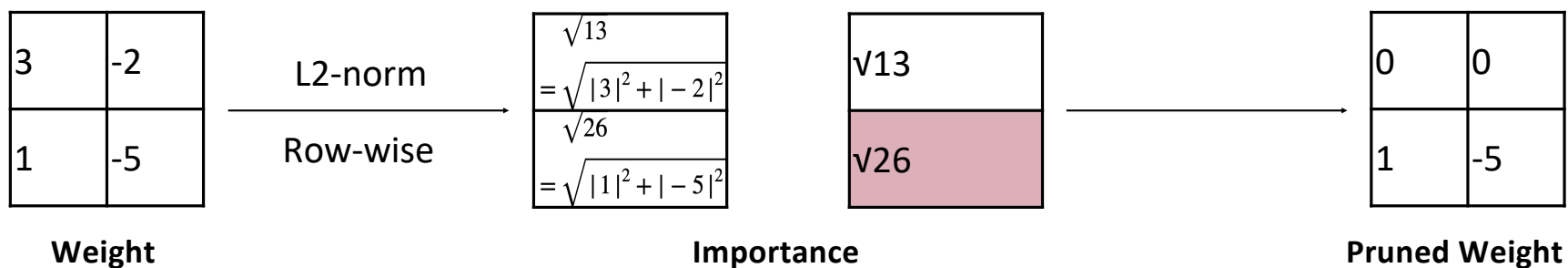
Magnitude-based Pruning

A heuristic pruning criterion

- Magnitude-based pruning considers weights with ***larger absolute values*** are more important than other weights.
- Magnitude is also known as L_p -norm defined as,

$$\| \mathbf{W}^{(S)} \|_p = \left(\sum_{i \in S} |w_i|^p \right)^{\frac{1}{p}}, \text{ where } \mathbf{W}^{(S)} \text{ is a structural set of parameters}$$

- **Example**



L1 vs L2 for Magnitude-Based Pruning

The answer depends on what you are scoring

Case 1 — Scoring individual weights

Unstructured pruning

L1 score: $|w|$

L2 score: w^2

Example — rank these weights by size:

weight w	+0.5	-0.8	+0.1	-0.3
$ w $	0.5	0.8	0.1	0.3
w^2	0.25	0.64	0.01	0.09
rank	3rd	4th	1st	2nd

Same ranking → same pruning decision. The L1 vs L2 choice does not matter here.

Case 2 — Scoring a group of weights

Structured pruning (head, filter, channel)

L1 score: $\sum |w_i|$

L2 score: $\sqrt{\sum w_i^2}$

Example — two groups, L1 and L2 disagree on which to prune:

group	weights	L1	L2
A (spiky)	0.9, 0.1, 0.1, 0.1	1.20	0.92
B (even)	0.4, 0.4, 0.4, 0.4	1.60	0.80
prunes	← L1 and L2 flip	A	B

Decision flips. L2 protects groups with a dominant weight; L1 protects even spread.

In practice

Unstructured

Just use $|w|$. L1 and L2 give identical rankings.

Structured

L2 is the common default for heads and filters. The gap over L1 is usually small.

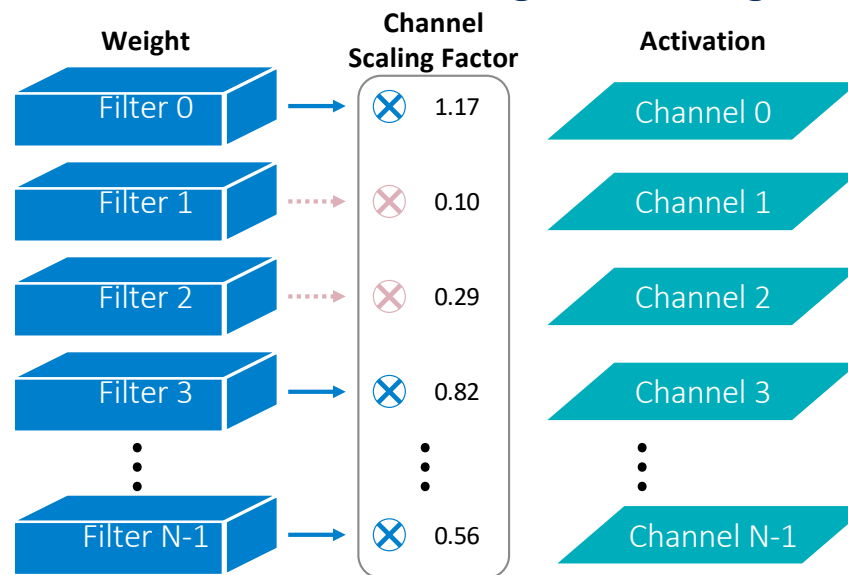
Modern LLMs

Wanda ($|w| \cdot \|x\|$) and SparseGPT beat plain magnitude — activations and curvature matter more than L1 vs L2.

Scaling-based Pruning

Pruning criterion for filter pruning

- A scaling factor is associated with each filter (*i.e.*, output channel) in convolutional layers
 - The scaling factor is multiplied to the output of that channel
 - The scaling factors are trainable parameters
- The filters/output channels with small scaling factor magnitude will be pruned

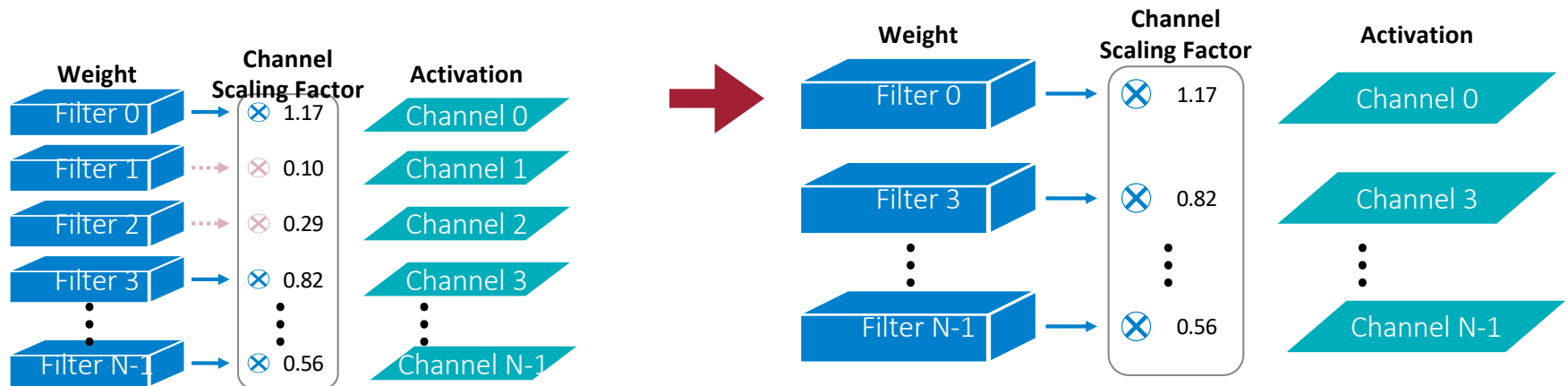


Learning Efficient Convolutional Networks through Network Slimming [Liu *et al.*, ICCV 2017]

Scaling-based Pruning

Pruning criterion for filter pruning

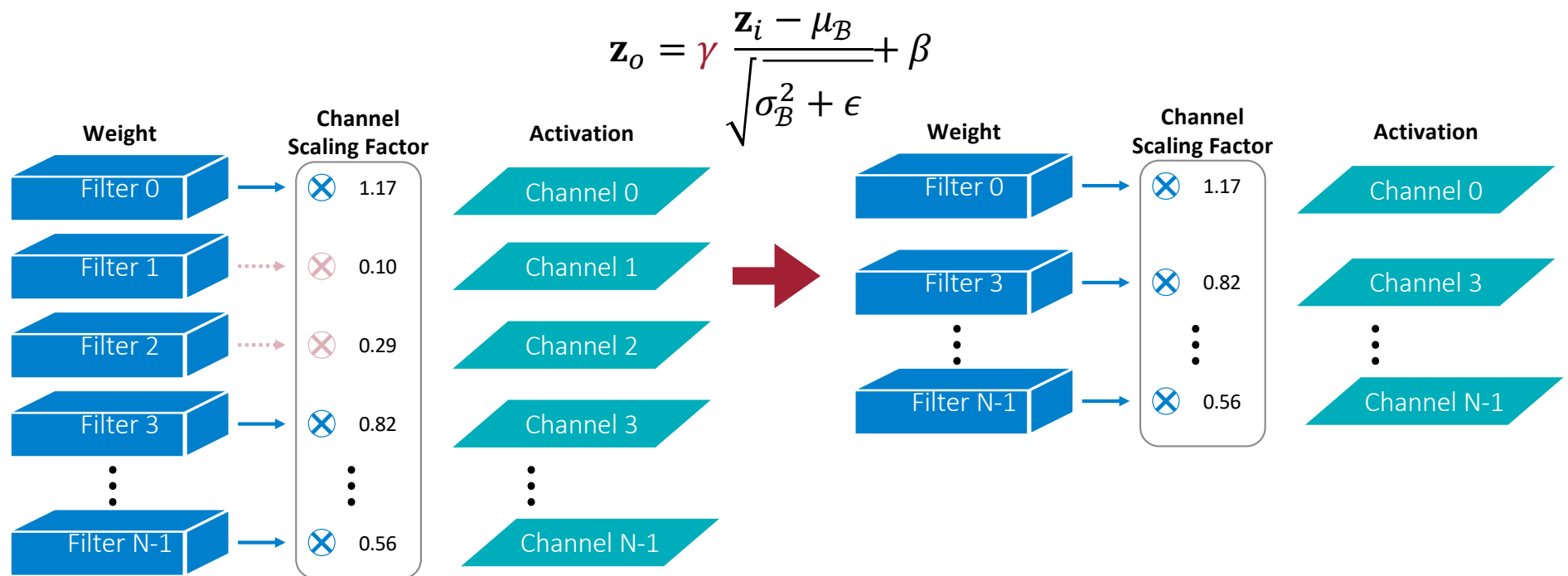
- A scaling factor is associated with each filter (*i.e.*, output channel) in convolutional layers
 - The scaling factor is multiplied to the output of that channel
 - The scaling factors are trainable parameters
- The filters/output channels with small scaling factor magnitude will be pruned



Scaling-based Pruning

Pruning criterion for filter pruning

- A scaling factor is associated with each filter (*i.e.*, output channel) in convolutional layers
- The scaling factors can be reused from batch normalization layer



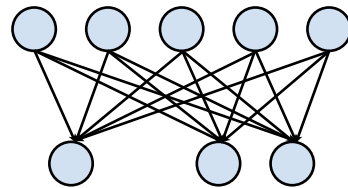
Learning Efficient Convolutional Networks through Network Slimming [Liu *et al.*, ICCV 2017]

Selection of Neurons to Prune

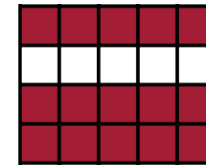
- When removing neurons from a neural network model,
 - ***the less useful*** the neurons being removed are,
 - the better the performance of pruned neural network is.

Recall: Neuron pruning is coarse-grained pruning indeed.

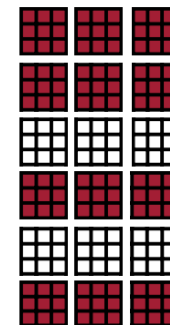
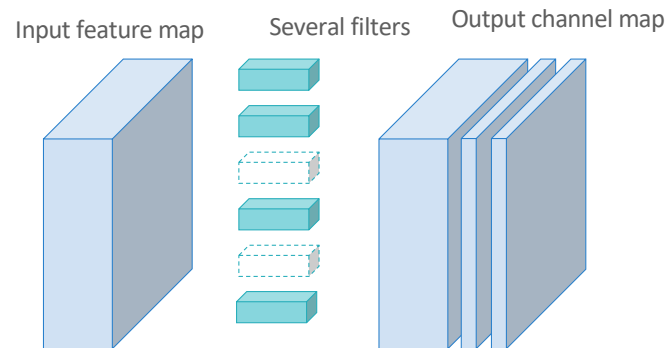
Neuron Pruning
in Linear Layer



Weight Matrix



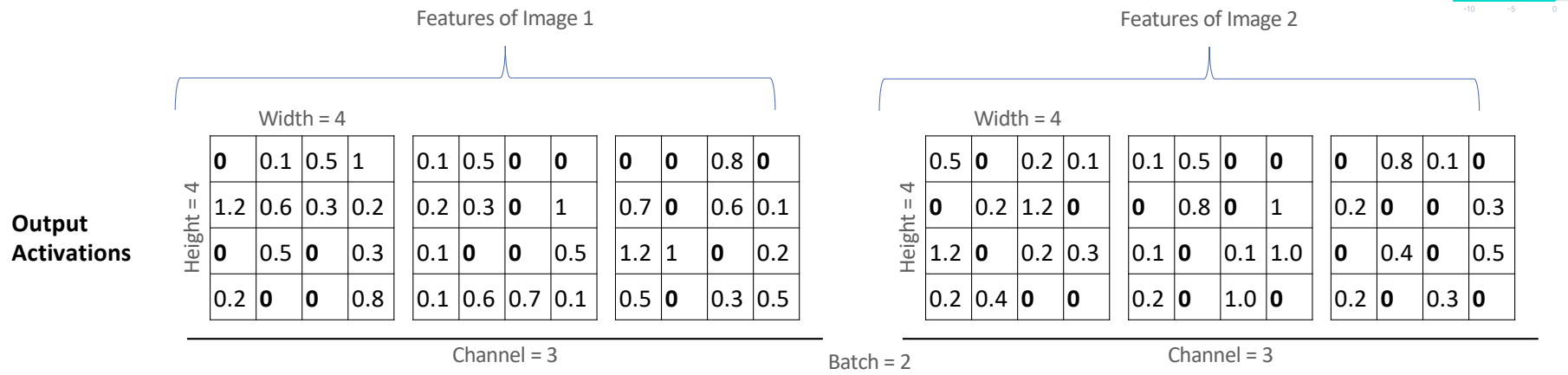
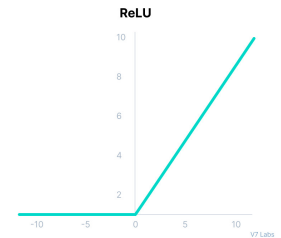
Channel Pruning
in Convolution Layer



<https://efficientml.ai>

Percentage-of-Zero-Based Pruning (POZ)

- ReLU activation will generate zeros in the output activation.



Percentage-of-Zero-Based Pruning

- ReLU activation will generate zeros in the output activation.
- Similar to magnitude of weights, the Average Percentage of Zero activations (APoZ) can be exploited to measure the importance of the neurons.

Output Activations		Width = 4								Width = 4							
		Height = 4	0	0.1	0.5	1	0.1	0.5	0	0	0	0	0.8	0	0.5	0.8	0.1
			1.2	0.6	0.3	0.2	0.2	0.3	0	1	0.7	0	0.6	0.1	1.2	1	0
			0	0.5	0	0.3	0.1	0	0	0.5	1.2	1	0	0.2	0.5	0	0.3
			0.2	0	0	0.8	0.1	0.6	0.7	0.1	0.5	0	0.3	0.5	0.2	0.4	0
		Channel = 3				Batch = 2				Channel = 3							

Average Percentage of Zeros (APoZ)

$= \frac{5 + 6}{2 \cdot 4 \cdot 4} = \frac{11}{32}$ Channel 0	$= \frac{5 + 7}{2 \cdot 4 \cdot 4} = \frac{12}{32}$ Channel 1	$= \frac{6 + 8}{2 \cdot 4 \cdot 4} = \frac{14}{32}$ Channel 2
--	--	--

Percentage-of-Zero-Based Pruning

- ReLU activation will generate zeros in the output activation.
- Similar to magnitude of weights, the Average Percentage of Zero activations (APoZ) can be exploited to measure the importance of the neurons.
- The smaller APoZ is, the more importance the neuron has.

Output Activations

Height = 4

Width = 4

0	0.1	0.5	1
1.2	0.6	0.3	0.2
0	0.5	0	0.3
0.2	0	0	0.8

Channel = 3

0.1	0.5	0	0
0.2	0.3	0	1
0.1	0	0	0.5
0.1	0.6	0.7	0.1

Batch = 2

0	0	0.8	0
0.7	0	0.6	0.1
1.2	1	0	0.2
0.5	0	0.3	0.5

Height = 4

Width = 4

0.5	0	0.2	0.1
0	0.2	1.2	0
1.2	0	0.2	0.3
0.2	0.4	0	0

Channel = 3

0.1	0.5	0	0
0	0.8	0	1
0.1	0	0.1	1.0
0.2	0	1.0	0

0	0.8	0.1	0
0.2	0	0	0.3
0	0.4	0	0.5
0.2	0	0.3	0

Average Percentage of Zeros (APoZ)

$= \frac{5 + 6}{2 \cdot 4 \cdot 4} = \frac{11}{32}$ Channel 0	$= \frac{5 + 7}{2 \cdot 4 \cdot 4} = \frac{12}{32}$ Channel 1	 $= \frac{6 + 8}{2 \cdot 4 \cdot 4} = \frac{14}{32}$ Channel 2
--	--	--

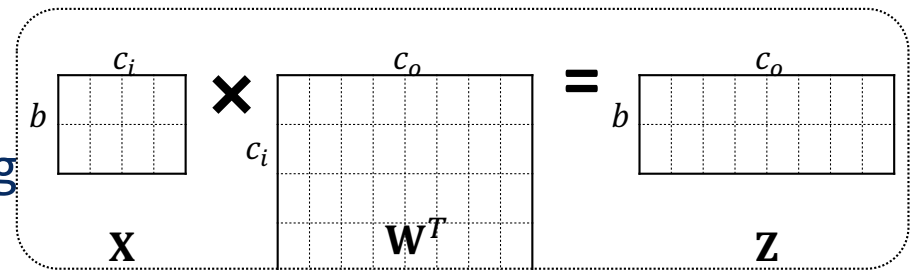
Network Trimming: A Data-Driven Neuron Pruning Approach towards Efficient Deep Architectures [Hu *et al.*, ArXiv 2017]

<https://efficientml.ai>

Regression-based Pruning

Minimize reconstruction error of the corresponding layer's outputs

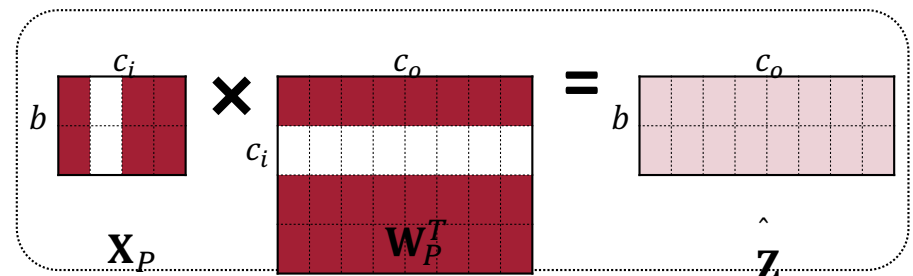
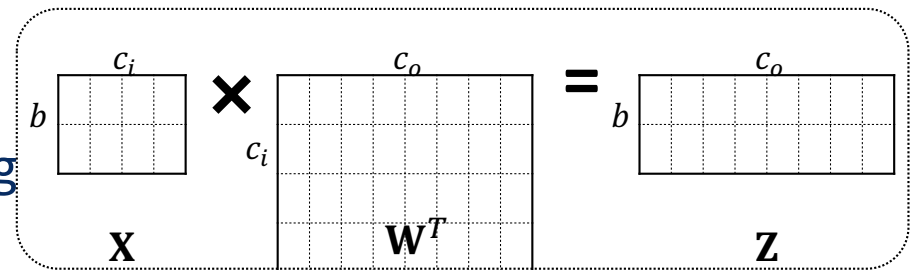
- Instead of considering the pruning error of the objective function $L(\mathbf{x}; \mathbf{W})$, regression-based pruning minimizes the reconstruction error of the corresponding layer's outputs.



Regression-based Pruning

Minimize reconstruction error of the corresponding layer's outputs

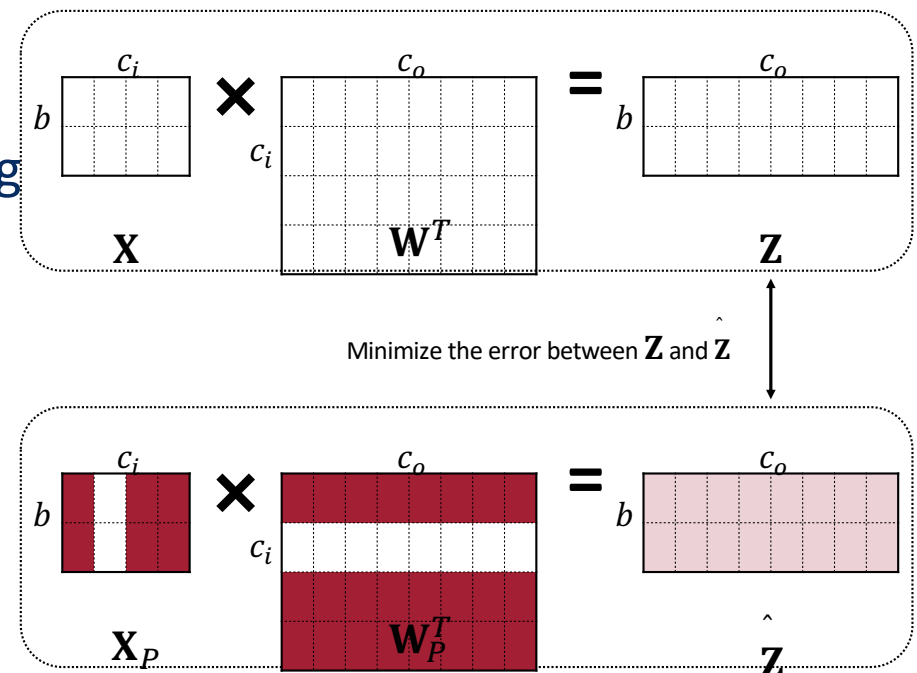
- Instead of considering the pruning error of the objective function $L(\mathbf{x}; \mathbf{W})$, regression-based pruning minimizes the reconstruction error of the corresponding layer's outputs.



Regression-based Pruning

Minimize reconstruction error of the corresponding layer's outputs

- Instead of considering the pruning error of the objective function $L(\mathbf{x}; \mathbf{W})$, regression-based pruning minimizes the reconstruction error of the corresponding layer's outputs.

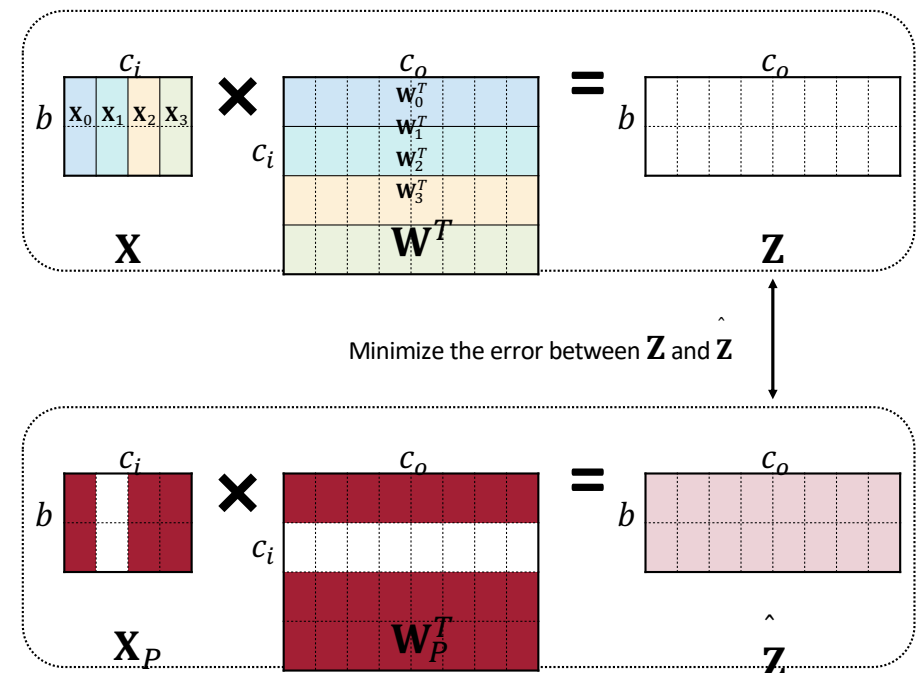


Regression-based Pruning

Minimize reconstruction error of the corresponding layer's outputs

- Let

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^T = \sum_{c=0}^{c_i-1} \mathbf{X}_c \mathbf{W}_c^T$$



Regression-based Pruning

Minimize reconstruction error of the corresponding layer's outputs

- Let

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^T = \sum_{c=0}^{c_i-1} \mathbf{X}_c \mathbf{W}_c^T$$

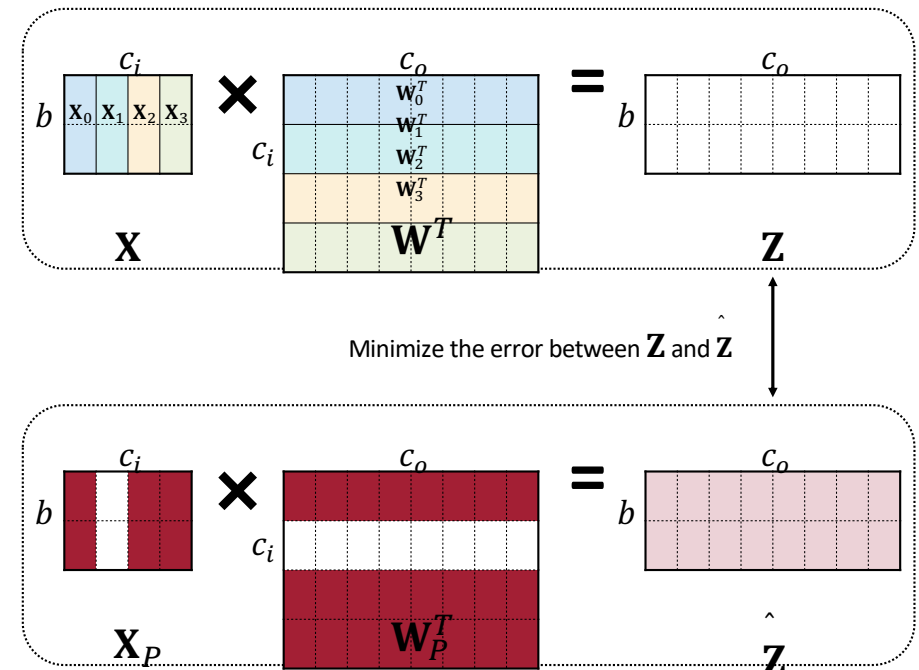
- The problem can be formulate as

$$\operatorname{argmin}_{\mathbf{W}, \beta} \|\mathbf{Z} - \hat{\mathbf{Z}}\|_F^2 = \|\mathbf{Z} - \sum_{c=0}^{c_i-1} \beta_c \mathbf{X}_c \mathbf{W}_c^T\|_F^2$$

subject to

$$\|\beta\|_0 \leq N_c$$

- β is coefficient vector of length c_i for channel selection. $\beta_c = 0$ means channel c is pruned.
- N_c is the number of nonzero channels.



Regression-based Pruning

Minimize reconstruction error of the corresponding layer's outputs

- Let

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^T = \sum_{c=0}^{c_i-1} \mathbf{X}_c \mathbf{W}_c^T$$

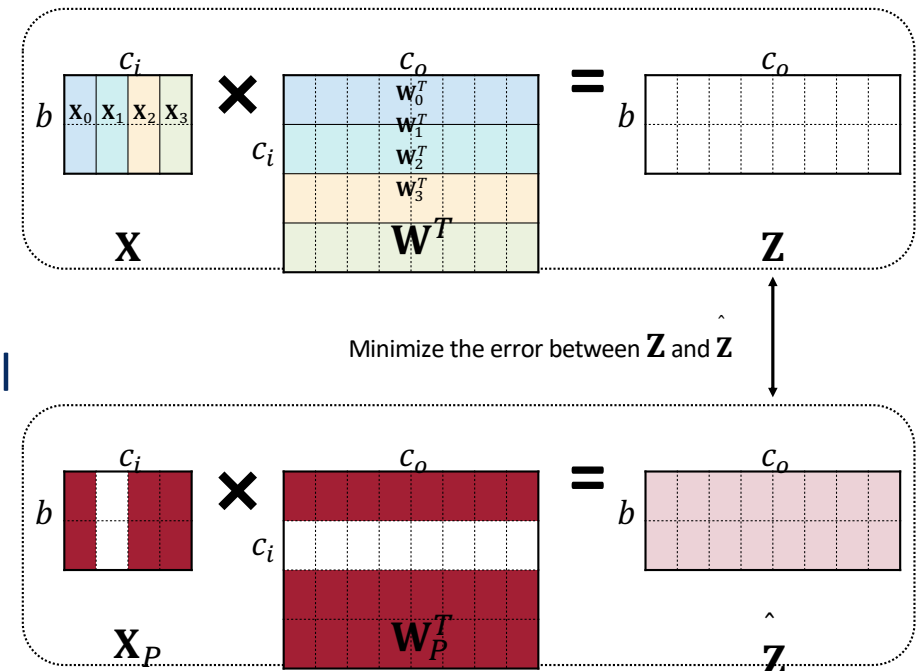
- The problem can be formulate as

$$\operatorname{argmin}_{\mathbf{W}, \beta} \|\mathbf{Z} - \hat{\mathbf{Z}}\|_F^2 = \|\mathbf{Z} - \sum_{c=0}^{c_i-1} \beta_c \mathbf{X}_c \mathbf{W}_c^T\|_F^2$$

subject to

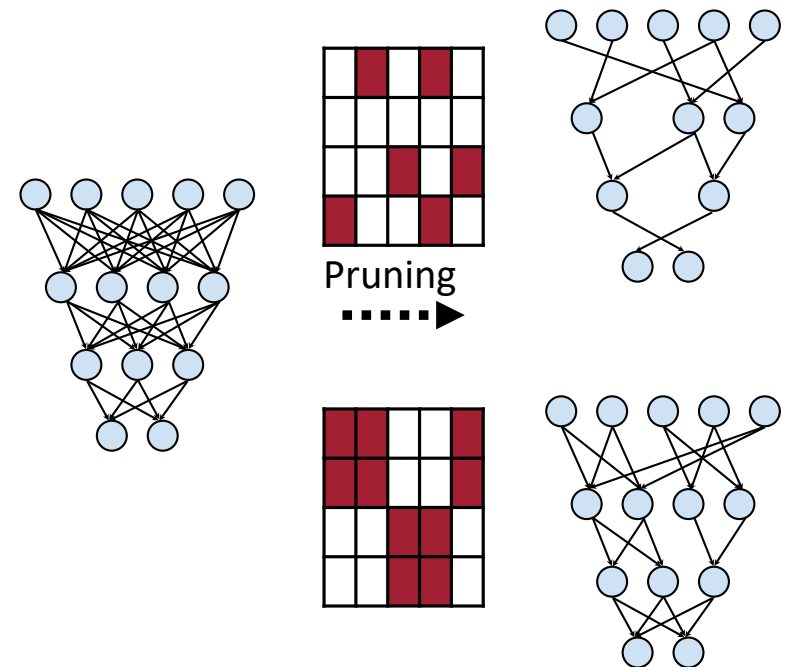
$$\|\beta\|_0 \leq N_c$$

- β is coefficient vector of length c_i for channel selection. $\beta_c = 0$ means channel c is pruned.
- N_c is the number of nonzero channels.
- Solve the problem by:
 - Fix $\mathbf{W}$, solve β for channel selection
 - Fix β , solve $\mathbf{W}$ to minimize reconstruction error



Neural Network Pruning

- **Introduction to Pruning**
 - What is pruning?
 - How should we formulate pruning?
- **Determine the Pruning Granularity**
 - In what pattern should we prune the neural network?
- **Determine the Pruning Criterion**
 - What synapses/neurons should we prune?
- **Determine the Pruning Ratio**
 - What should target sparsity be for each layer?
- **Fine-tune/Train Pruned Neural Network**
 - How should we improve performance of pruned models?

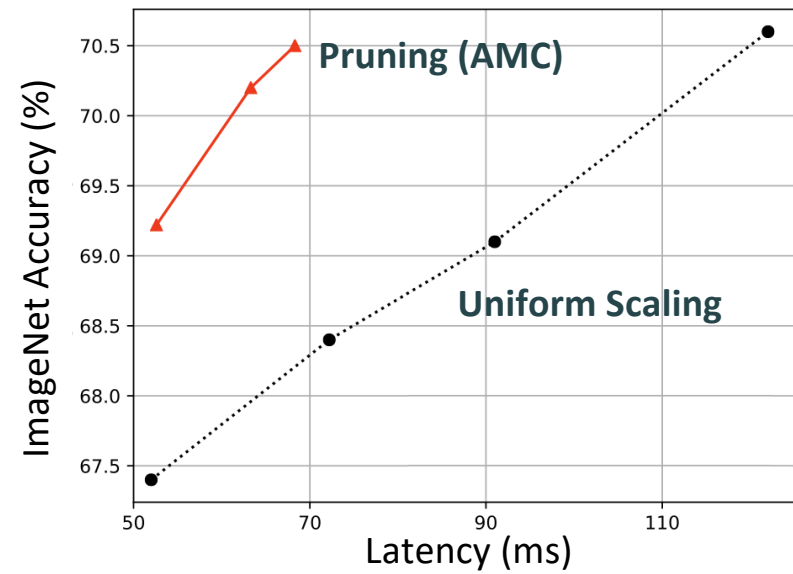
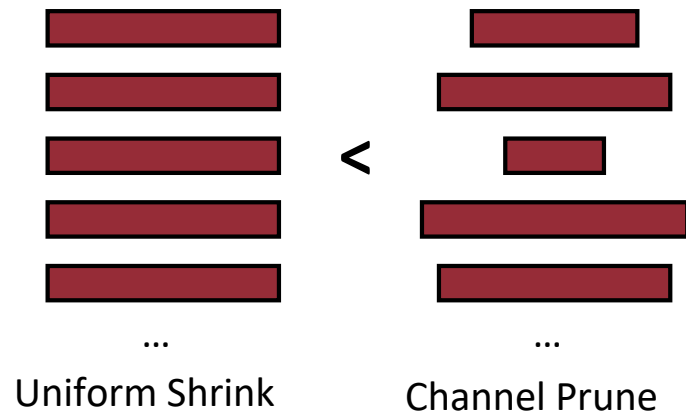


Pruning Ratio

How should we find per-layer pruning ratios?

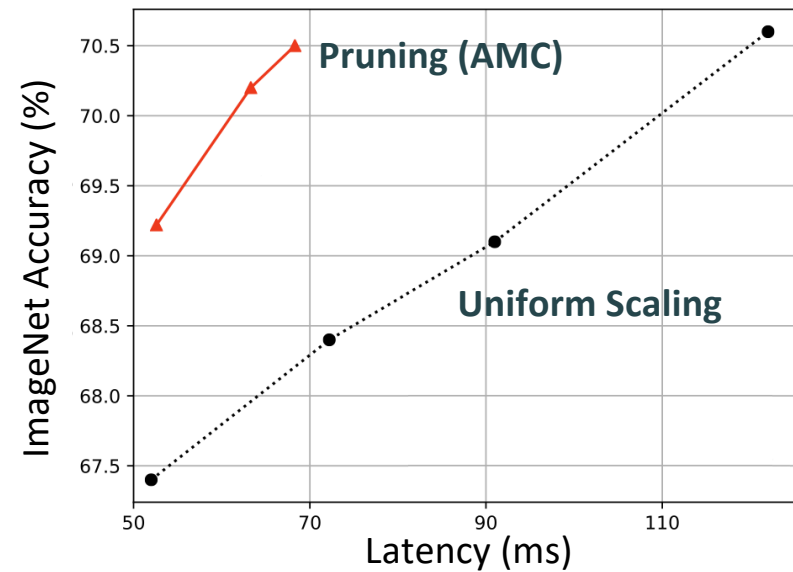
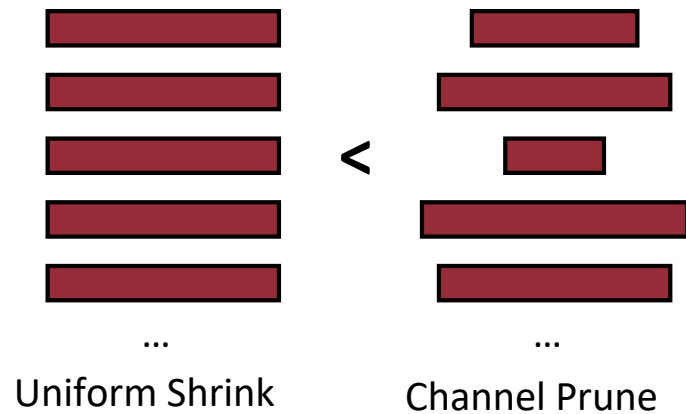
Recap

Non-uniform pruning is better than uniform shrinking



Recap

Non-uniform pruning is better than uniform shrinking



Question: how should we find ratios for each layer?

Finding Pruning Ratios

Analyze the sensitivity of each layer

- We need different pruning ratios for each layer since different layers have different **sensitivity**
 - Some layers are more sensitive (e.g., first layer)
 - Some layers are more redundant

Finding Pruning Ratios

Analyze the sensitivity of each layer

- We need different pruning ratios for each layer since different layers have different **sensitivity**
 - Some layers are more sensitive (e.g., first layer)
 - Some layers are more redundant
- We can perform **sensitivity analysis** to determine the per-layer pruning ratio

Finding Pruning Ratios

Analyze the sensitivity of each layer

(* VGG-11 on CIFAR-10 dataset)

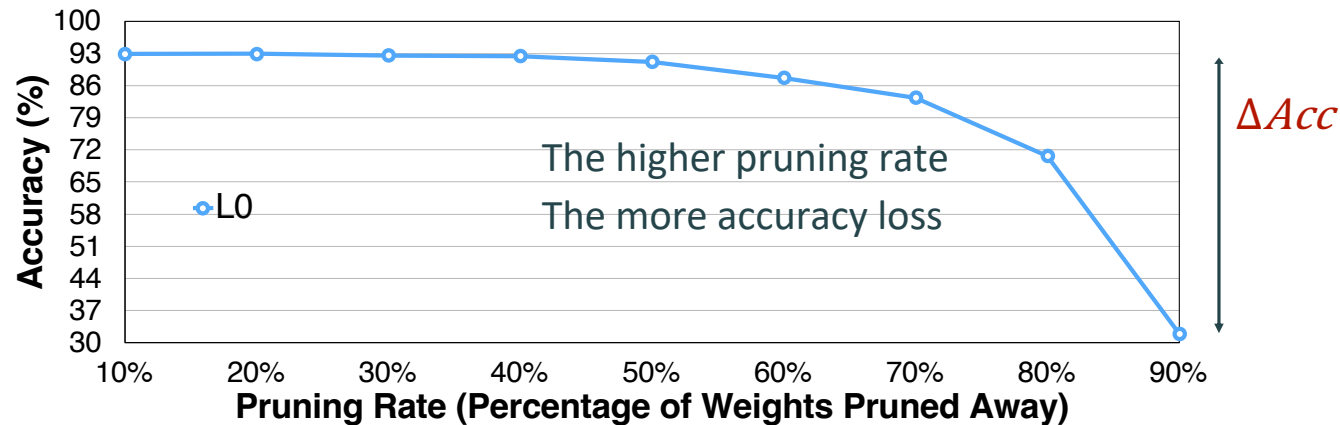
- The process of Sensitivity Analysis
 - Pick a layer L_i in the model

Finding Pruning Ratios

Analyze the sensitivity of each layer

(* VGG-11 on CIFAR-10 dataset)

- The process of Sensitivity Analysis
 - Pick a layer L_i in the model
 - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, \dots, 0.9\}$ (or other strides)
 - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio

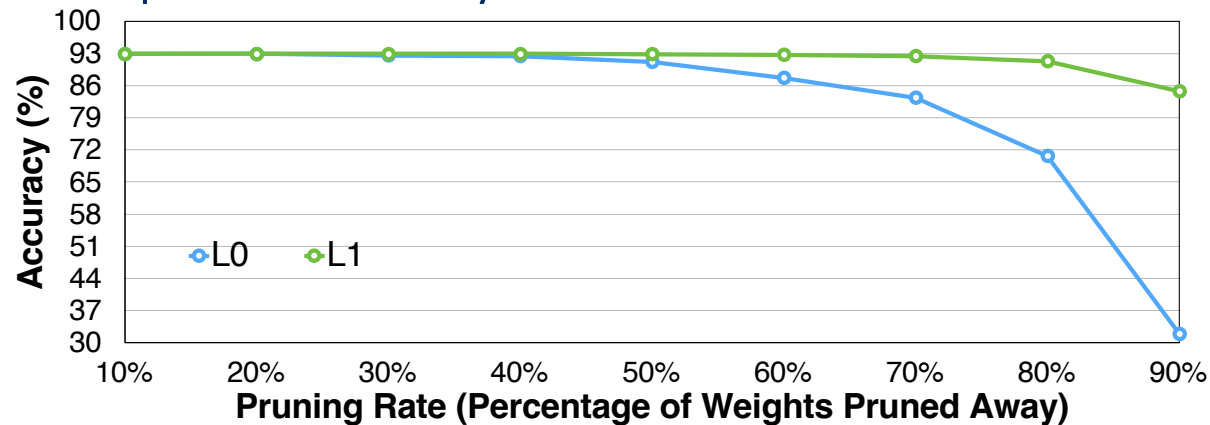


Finding Pruning Ratios

Analyze the sensitivity of each layer

(* VGG-11 on CIFAR-10 dataset)

- The process of Sensitivity Analysis
 - Pick a layer L_i in the model
 - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, \dots, 0.9\}$ (or other strides)
 - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
 - Repeat the process for all layers

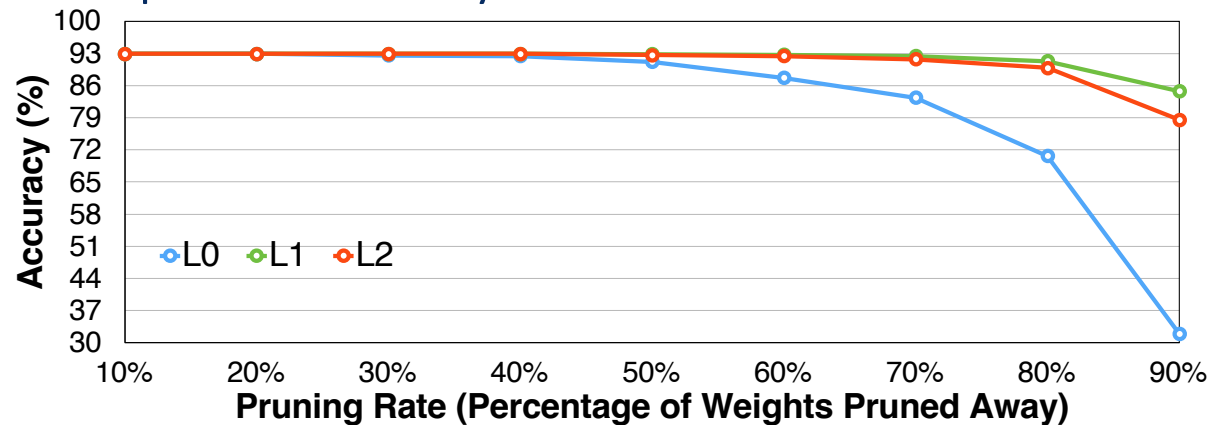


Finding Pruning Ratios

Analyze the sensitivity of each layer

(* VGG-11 on CIFAR-10 dataset)

- The process of Sensitivity Analysis
 - Pick a layer L_i in the model
 - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, \dots, 0.9\}$ (or other strides)
 - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
 - Repeat the process for all layers

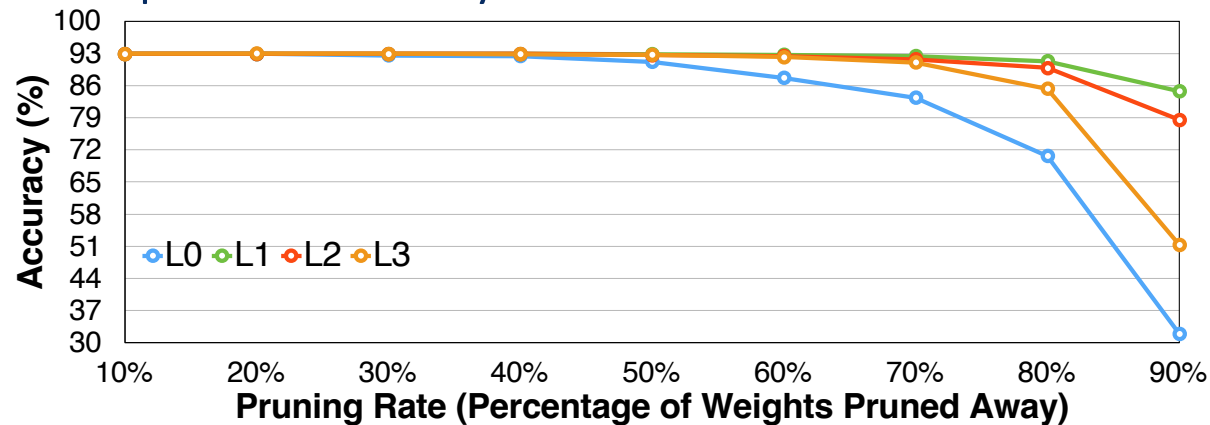


Finding Pruning Ratios

Analyze the sensitivity of each layer

(* VGG-11 on CIFAR-10 dataset)

- The process of Sensitivity Analysis
 - Pick a layer L_i in the model
 - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, \dots, 0.9\}$ (or other strides)
 - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
 - Repeat the process for all layers

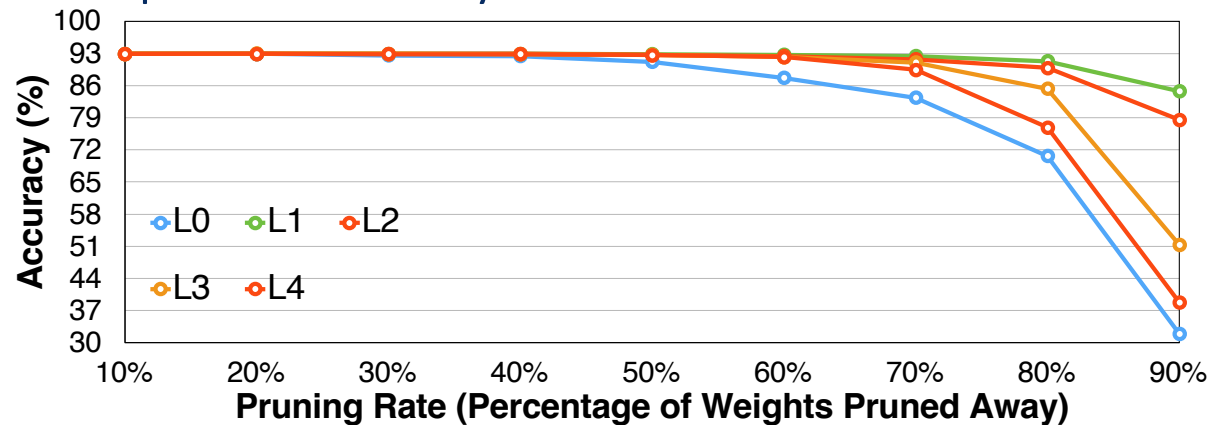


Finding Pruning Ratios

Analyze the sensitivity of each layer

(* VGG-11 on CIFAR-10 dataset)

- The process of Sensitivity Analysis
 - Pick a layer L_i in the model
 - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, \dots, 0.9\}$ (or other strides)
 - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
 - Repeat the process for all layers

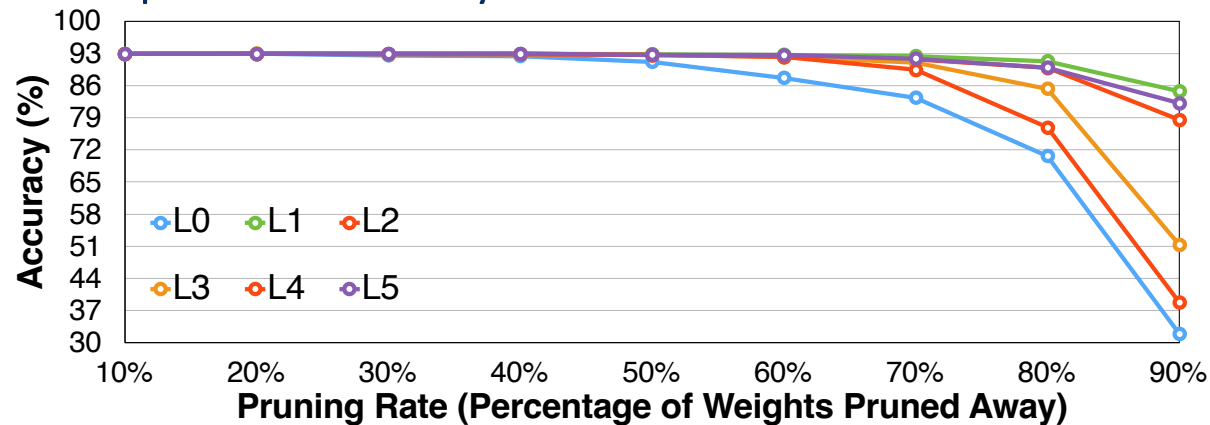


Finding Pruning Ratios

Analyze the sensitivity of each layer

(* VGG-11 on CIFAR-10 dataset)

- The process of Sensitivity Analysis
 - Pick a layer L_i in the model
 - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, \dots, 0.9\}$ (or other strides)
 - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
 - Repeat the process for all layers



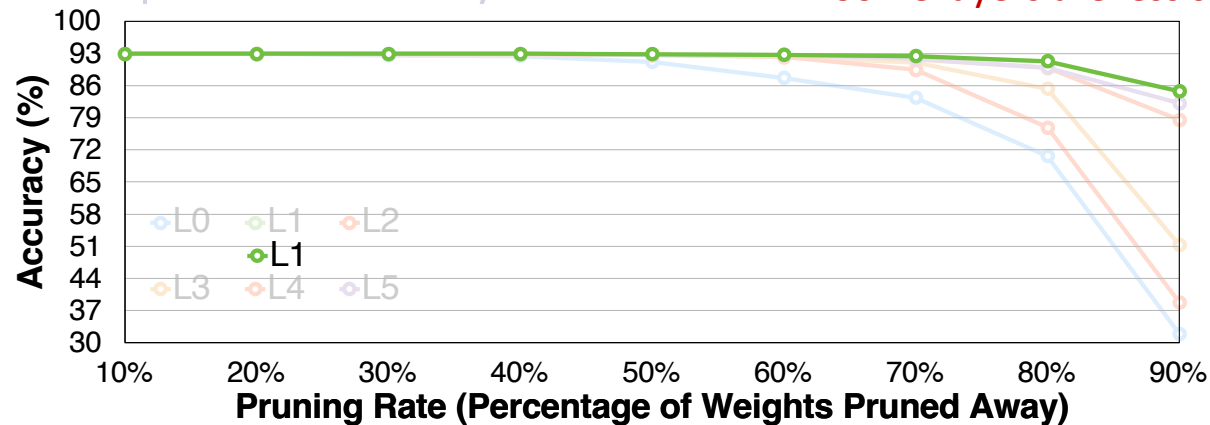
Finding Pruning Ratios

Analyze the sensitivity of each layer

(* VGG-11 on CIFAR-10 dataset)

- The process of Sensitivity Analysis
 - Pick a layer L_i in the model
 - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, \dots, 0.9\}$ (or other strides)
 - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
 - Repeat the process for all layers

Some layers are less sensitive to pruning

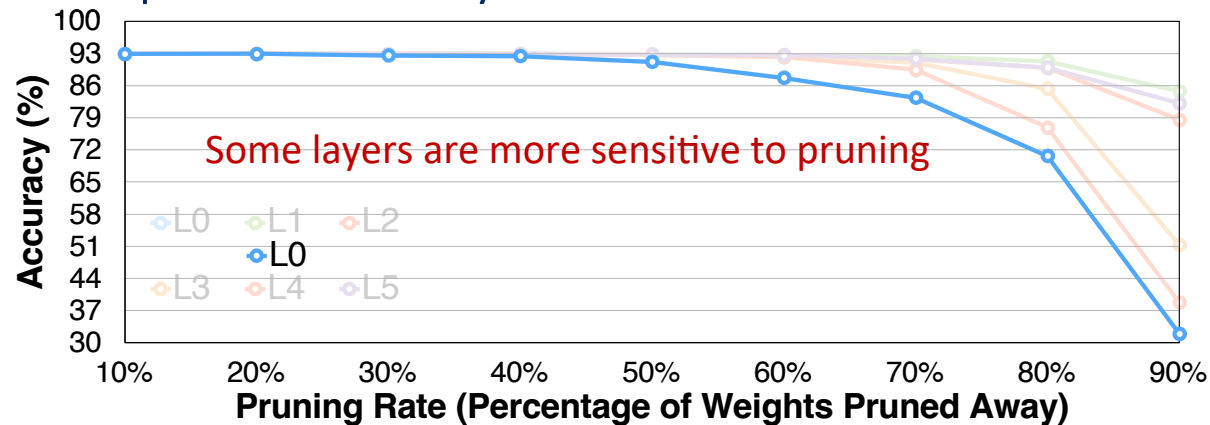


Finding Pruning Ratios

Analyze the sensitivity of each layer

(* VGG-11 on CIFAR-10 dataset)

- The process of Sensitivity Analysis
 - Pick a layer L_i in the model
 - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, \dots, 0.9\}$ (or other strides)
 - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
 - Repeat the process for all layers



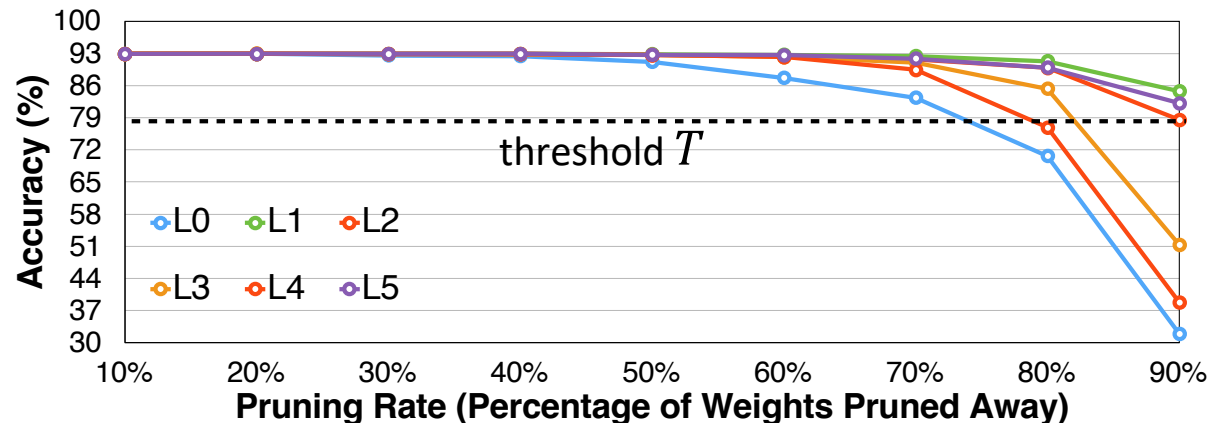
Finding Pruning Ratios

Analyze the sensitivity of each layer

- The process of Sensitivity Analysis

(* VGG-11 on CIFAR-10 dataset)

- Pick a layer L_i in the model
 - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, \dots, 0.9\}$ (or other strides)
 - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
- Repeat the process for all layers
- Pick a degradation threshold T such that the overall pruning rate is desired

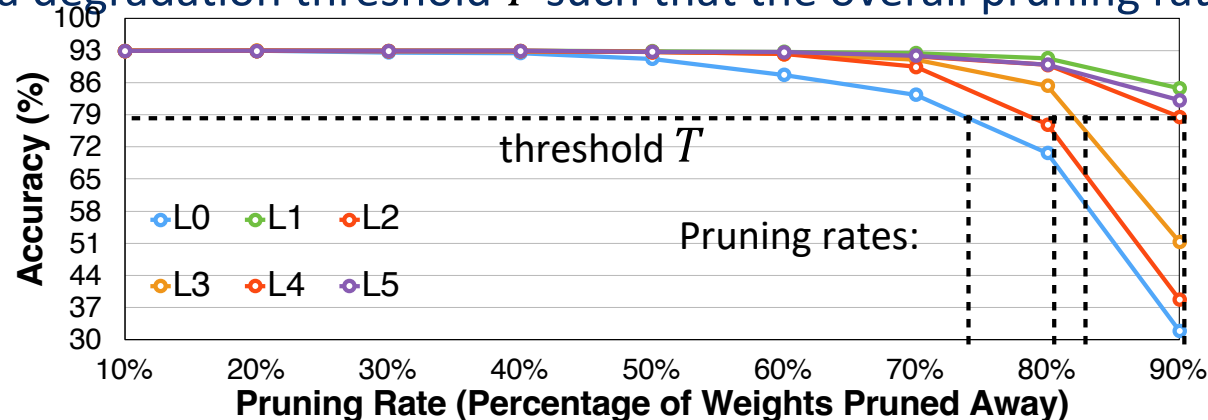


Finding Pruning Ratios

Analyze the sensitivity of each layer

(* VGG-11 on CIFAR-10 dataset)

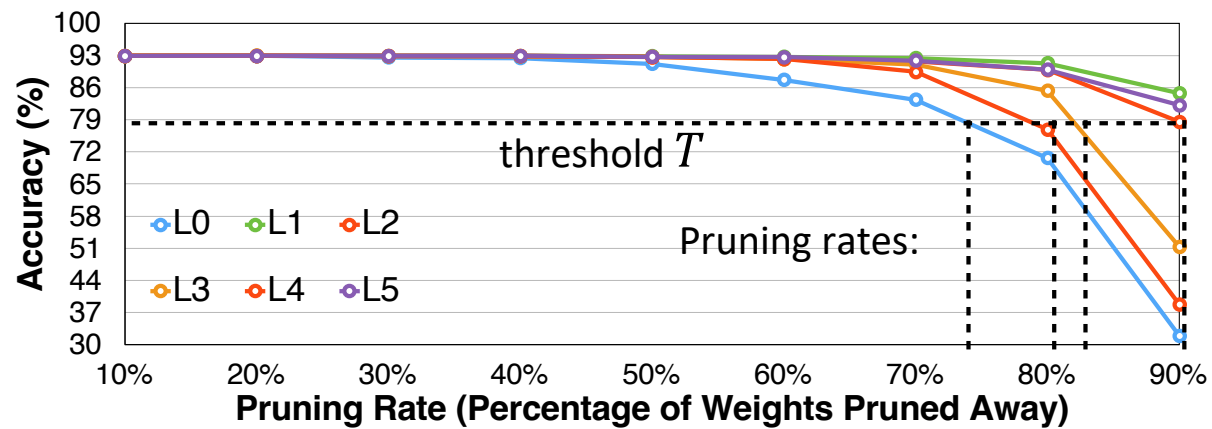
- The process of Sensitivity Analysis
 - Pick a layer L_i in the model
 - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, \dots, 0.9\}$ (or other strides)
 - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
 - Repeat the process for all layers
 - Pick a degradation threshold T such that the overall pruning rate is desired



Finding Pruning Ratios

Analyze the sensitivity of each layer

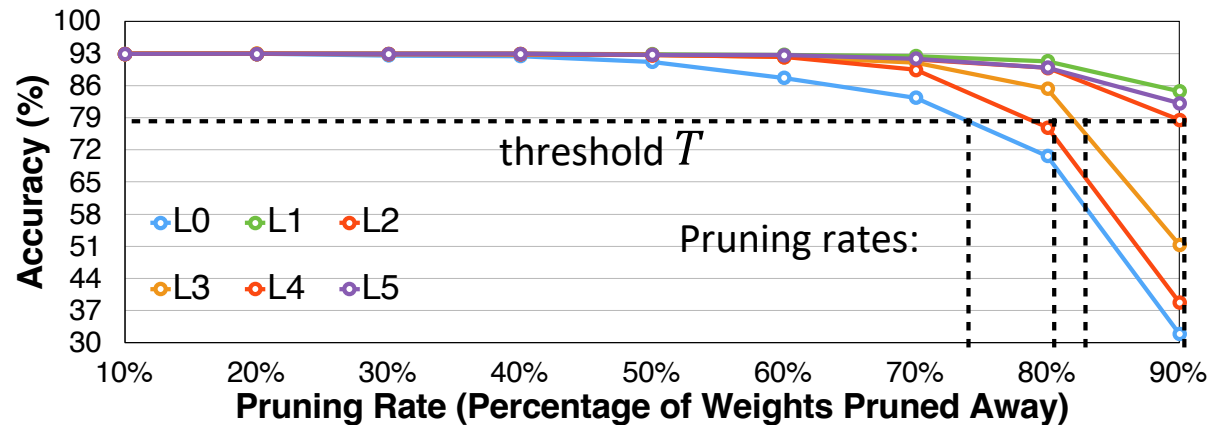
- Is this optimal?



Finding Pruning Ratios

Analyze the sensitivity of each layer

- Is this optimal?
 - Maybe not. We do not consider the interaction between layers.
- Can we go beyond the heuristics?
 - Yes!



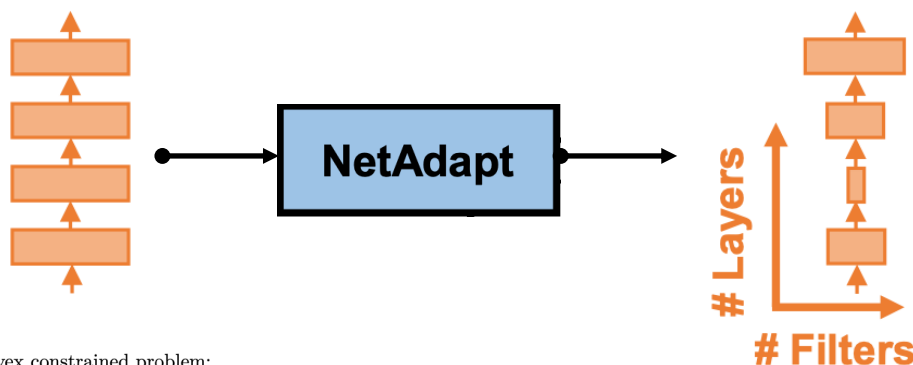
Automatic Pruning

- Given an **overall** compression ratio, how do we choose **per-layer** pruning ratios?
 - Sensitivity analysis ignores the interaction between layers -> sub-optimal

NetAdapt

A rule-based iterative/progressive method

- The goal of NetAdapt is to find a per-layer pruning ratio to meet a global resource constraint (e.g., latency, energy, ...)
- The process is done iteratively
- We take **latency** constraint as an example



NetAdapt aims to solve the following non-convex constrained problem:

$$\begin{aligned} & \underset{Net}{\text{maximize}} && Acc(Net) \\ & \text{subject to} && Res_j(Net) \leq Bud_j, \quad j = 1, \dots, m, \end{aligned}$$

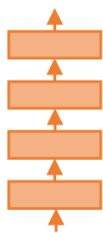
The resource can be latency, energy, memory footprint, etc., or a combination of these metrics.

NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications [Yang *et al.*, ECCV 2018]

<https://efficientml.ai>

NetAdapt

- For each iteration, we aim to reduce the latency by a certain amount ΔR (manually defined)



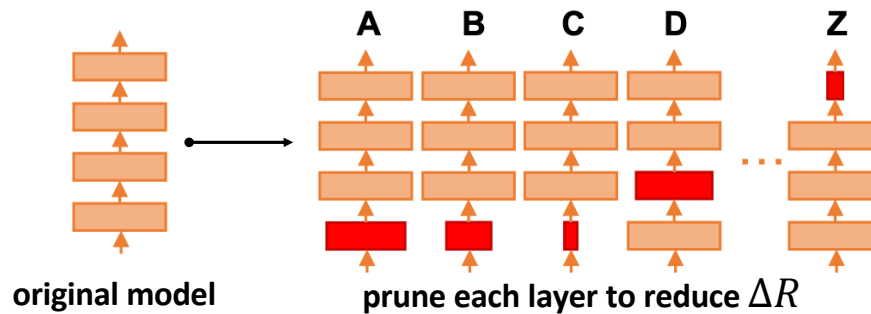
original model

NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications [Yang *et al.*, ECCV 2018]

<https://efficientml.ai>

NetAdapt

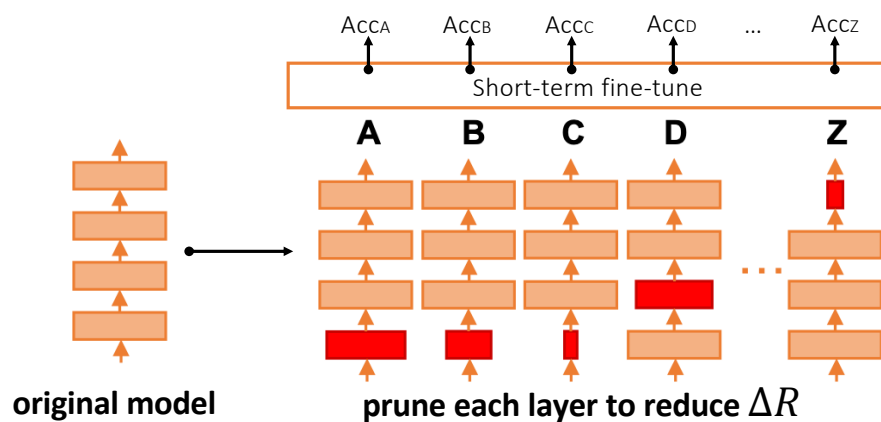
- For each iteration, we aim to reduce the latency by a certain amount ΔR (manually defined)
 - For each layer L_k (k in A-Z in the figure)
 - Prune the layer s.t. the latency reduction meets ΔR (based on a pre-built lookup table)



NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications [Yang *et al.*, ECCV 2018]
<https://efficientml.ai>

NetAdapt

- For each iteration, we aim to reduce the latency by a certain amount ΔR (manually defined)
 - For each layer L_k (k in A-Z in the figure)
 - Prune the layer s.t. the latency reduction meets ΔR (based on a pre-built lookup table)
 - Short-term fine-tune model (10k iterations); measure accuracy after fine-tuning

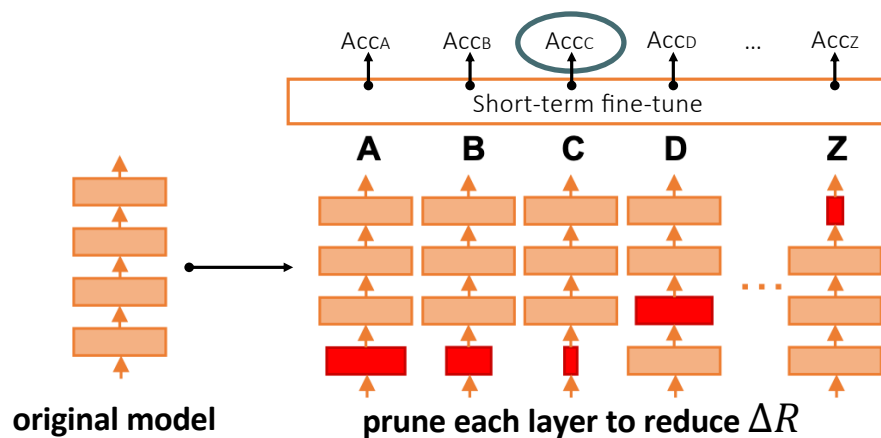


NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications [Yang *et al.*, ECCV 2018]

<https://efficientml.ai>

NetAdapt

- For each iteration, we aim to reduce the latency by a certain amount ΔR (manually defined)
 - For each layer L_k (k in A-Z in the figure)
 - Prune the layer s.t. the latency reduction meets ΔR (based on a pre-built lookup table)
 - Short-term fine-tune model (10k iterations); measure accuracy after fine-tuning
 - Choose and prune the layer with the highest accuracy

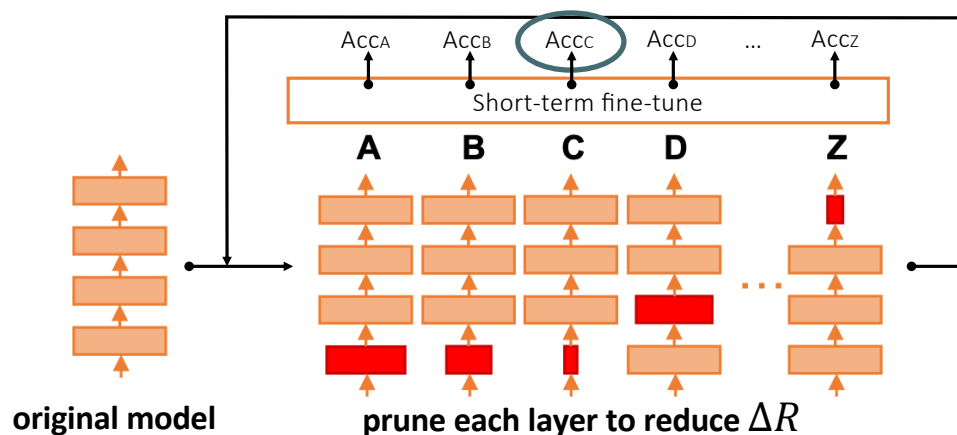


NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications [Yang *et al.*, ECCV 2018]

<https://efficientml.ai>

NetAdapt

- For each iteration, we aim to reduce the latency by a certain amount ΔR (manually defined)
 - For each layer L_k (k in A-Z in the figure)
 - Prune the layer s.t. the latency reduction meets ΔR (based on a pre-built lookup table)
 - Short-term fine-tune model (10k iterations); measure accuracy after fine-tuning
 - Choose and prune the layer with the highest accuracy
- Repeat until the total latency reduction satisfies the constraint

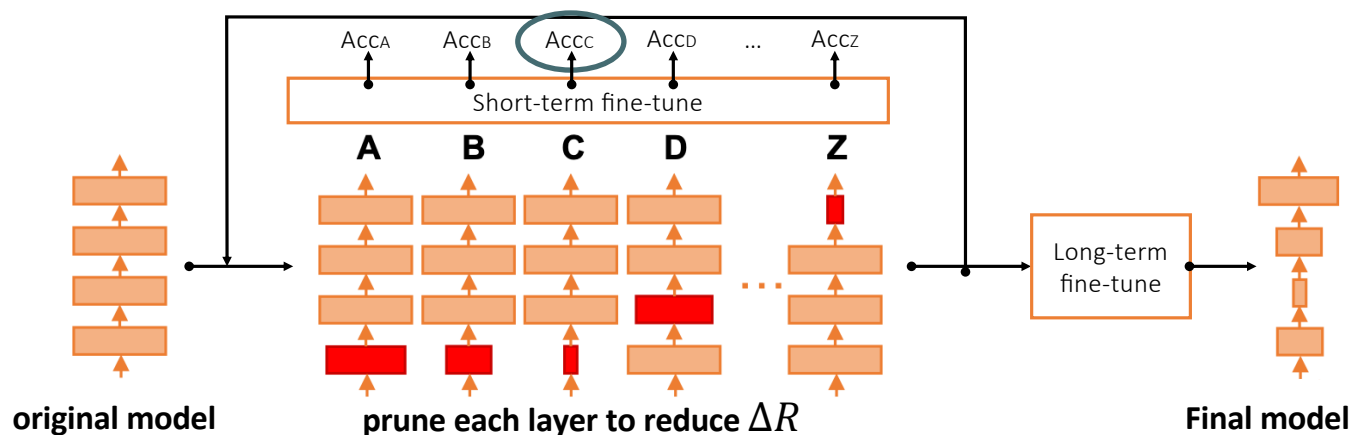


NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications [Yang *et al.*, ECCV 2018]

<https://efficientml.ai>

NetAdapt

- For each iteration, we aim to reduce the latency by a certain amount ΔR (manually defined)
 - For each layer L_k (k in A-Z in the figure)
 - Prune the layer s.t. the latency reduction meets ΔR (based on a pre-built lookup table)
 - Short-term fine-tune model (10k iterations); measure accuracy after fine-tuning
 - Choose and prune the layer with the highest accuracy
- Repeat until the total latency reduction satisfies the constraint
- Long-term fine-tune to recover accuracy

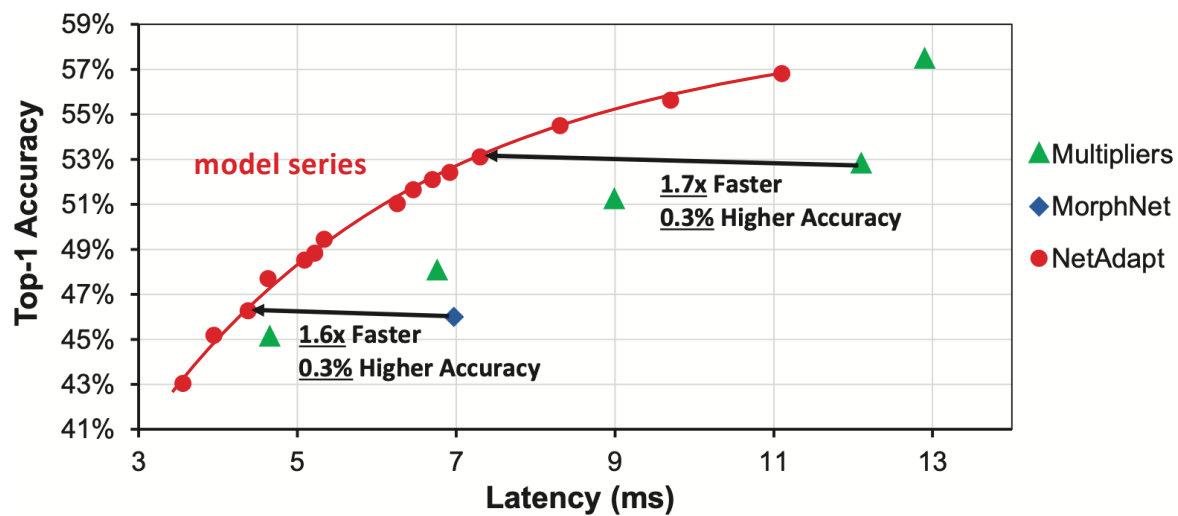


NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications [Yang *et al.*, ECCV 2018]

<https://efficientml.ai>

NetAdapt

- The iterative nature allows us to obtain a **serial of** models with different costs
 - #models = #iterations

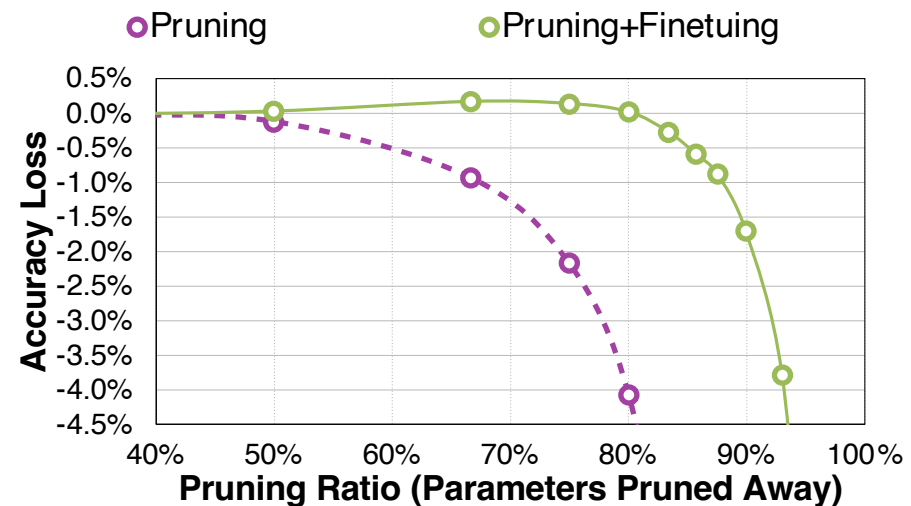
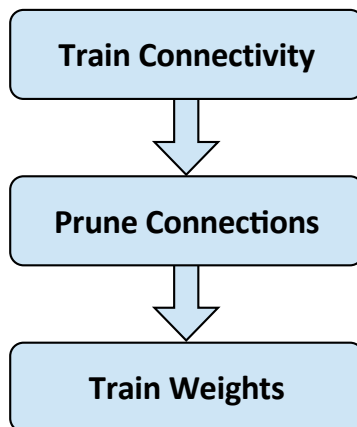


Fine-tuning / Training

How should we improve performance of sparse models?

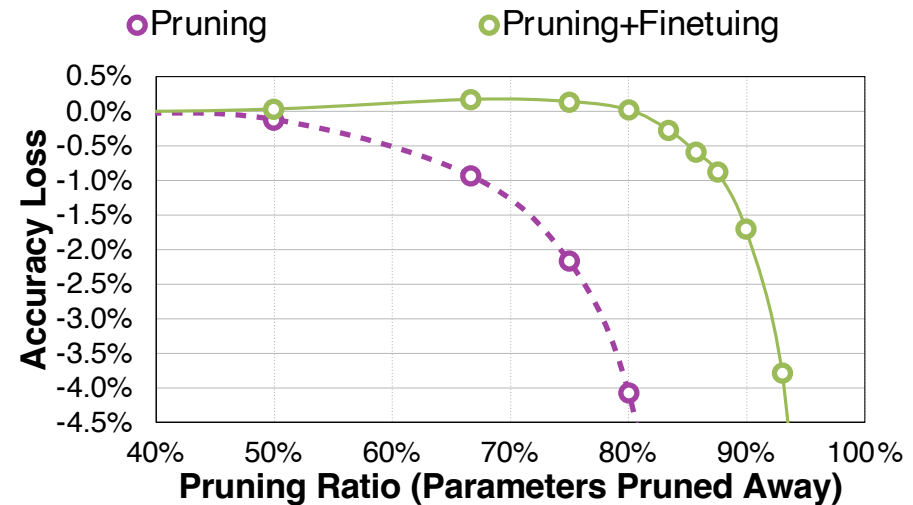
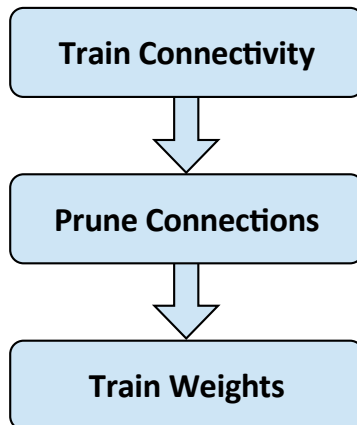
Finetuning Pruned Neural Networks

- After pruning, the model may decrease, especially for larger pruning ratio.
- Fine-tuning the pruned neural networks will help recover the accuracy and push the pruning ratio higher.
 - **Learning rate for fine-tuning is usually 1/100 or 1/10 of the original learning rate.**



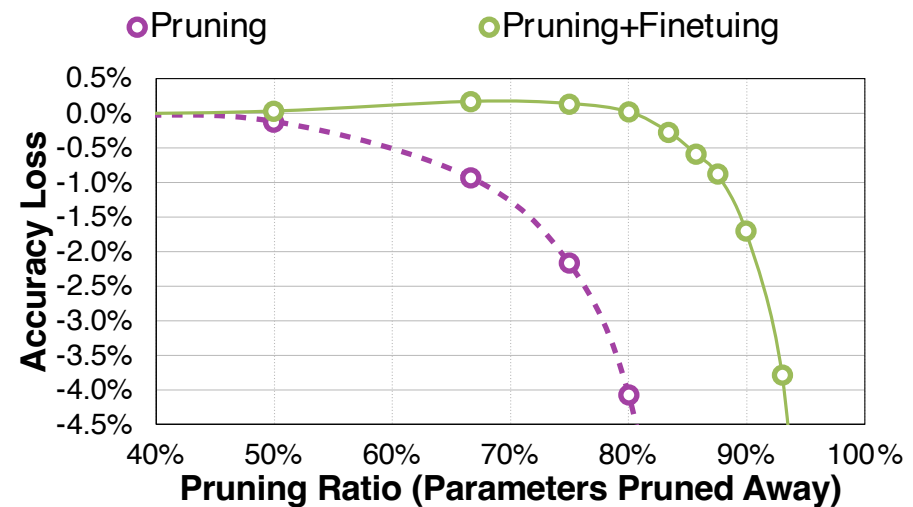
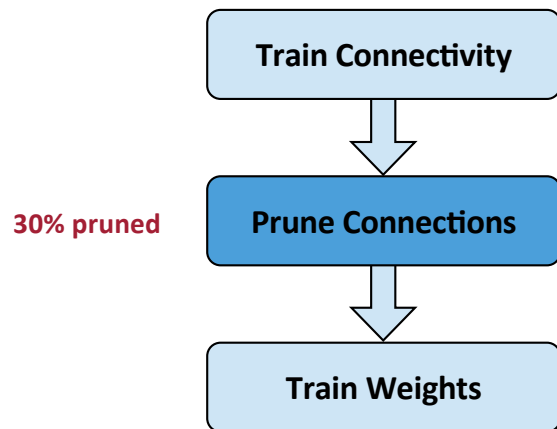
Iterative Pruning

- Consider pruning followed by a fine-tuning is one iteration.
- Iterative pruning gradually increases the target sparsity in each iteration.



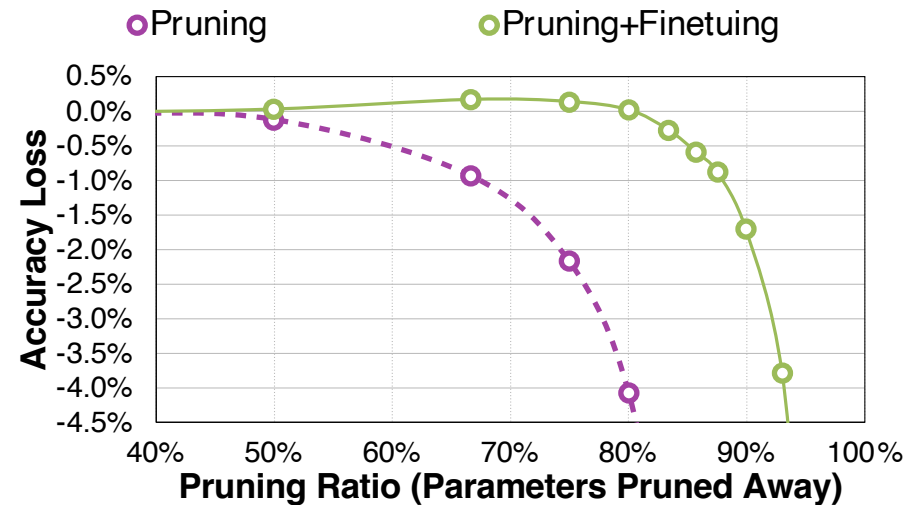
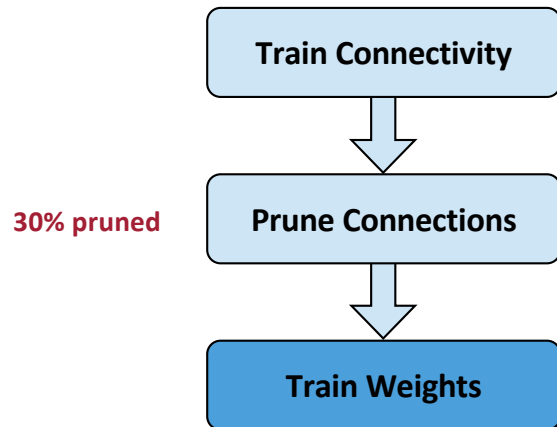
Iterative Pruning

- Consider pruning followed by a fine-tuning is one iteration.
- Iterative pruning gradually increases the target sparsity in each iteration.



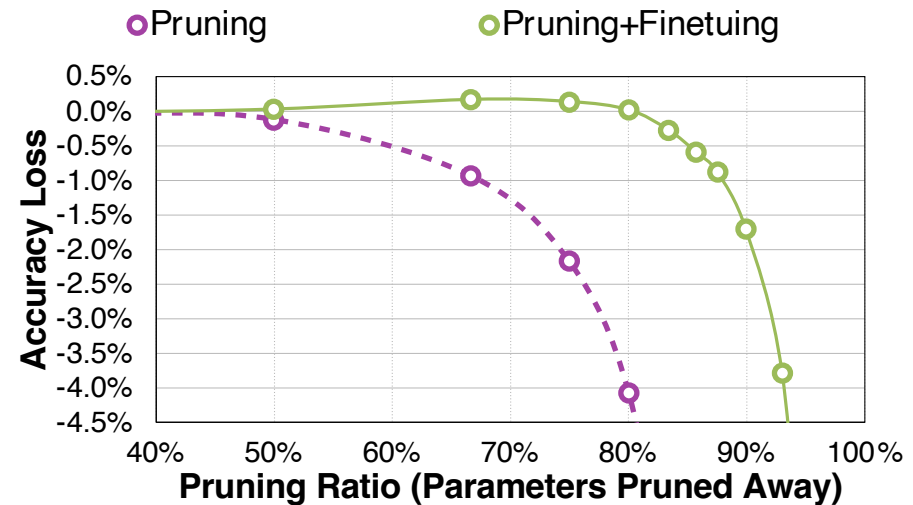
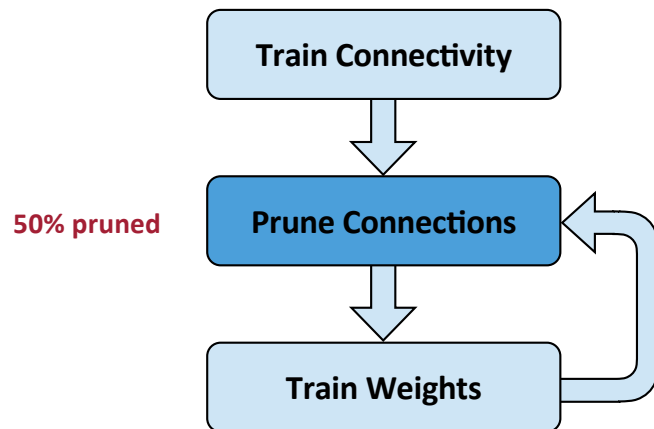
Iterative Pruning

- Consider pruning followed by a fine-tuning is one iteration.
- Iterative pruning gradually increases the target sparsity in each iteration.



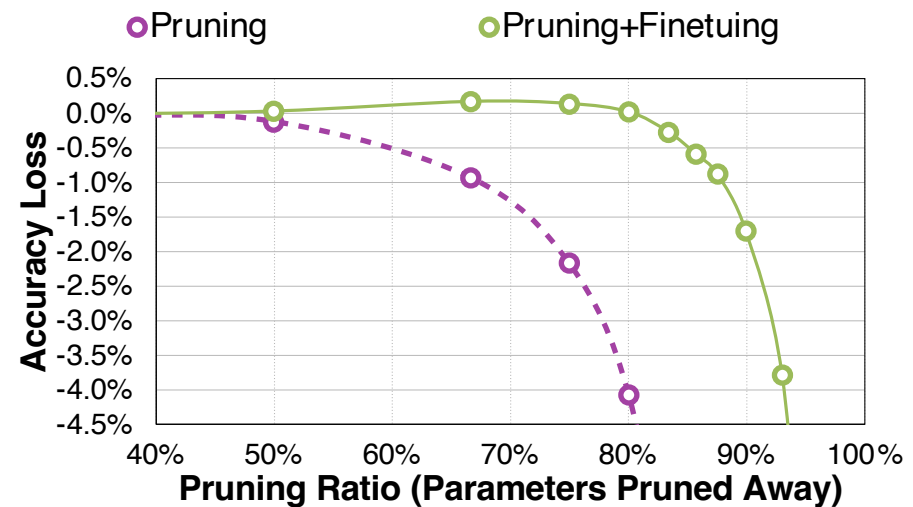
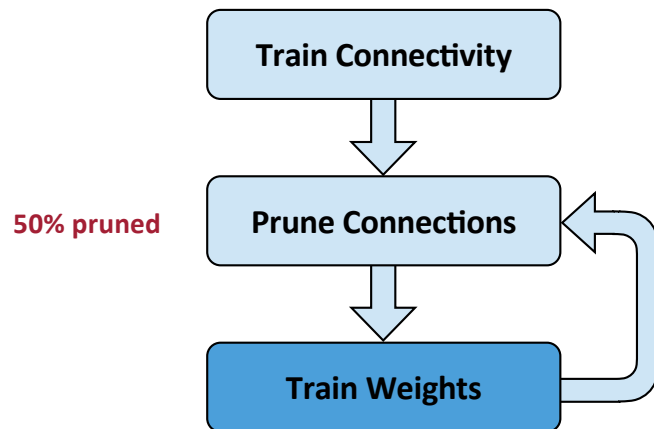
Iterative Pruning

- Consider pruning followed by a fine-tuning is one iteration.
- Iterative pruning gradually increases the target sparsity in each iteration.



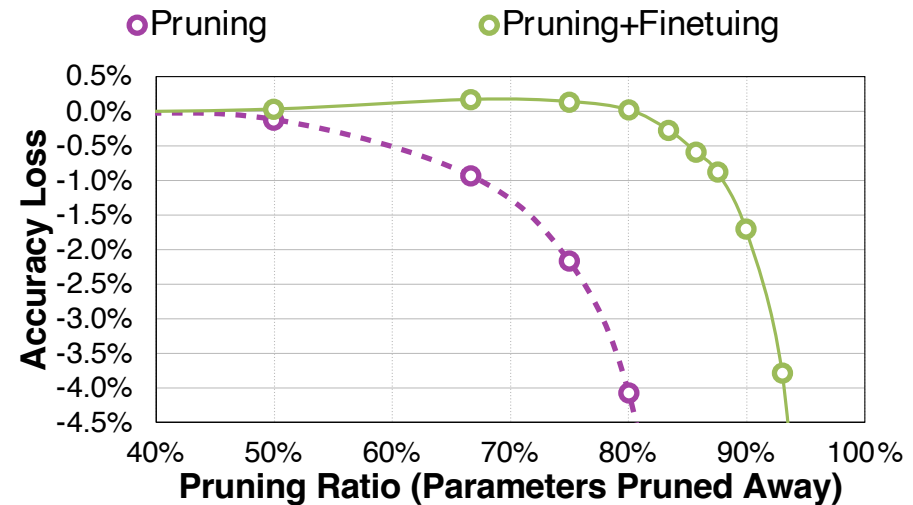
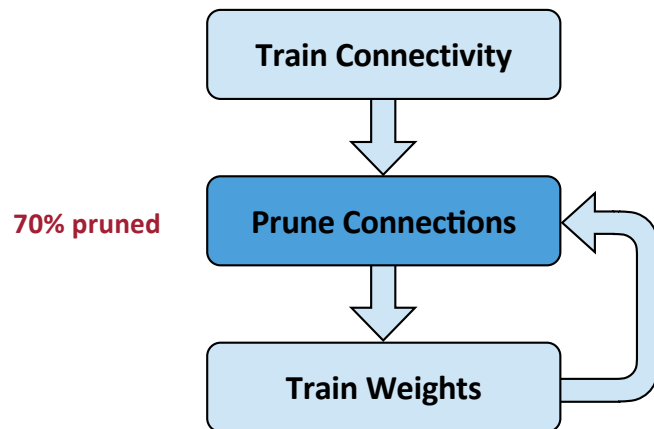
Iterative Pruning

- Consider pruning followed by a fine-tuning is one iteration.
- Iterative pruning gradually increases the target sparsity in each iteration.



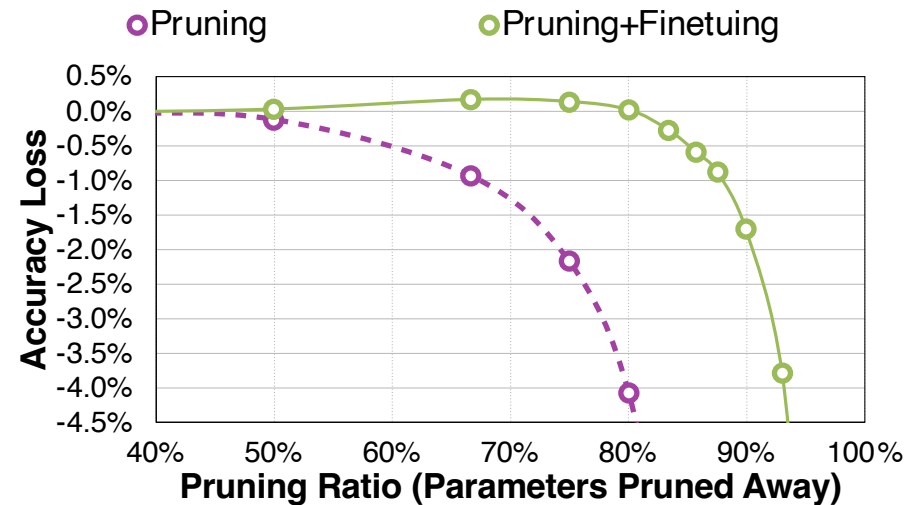
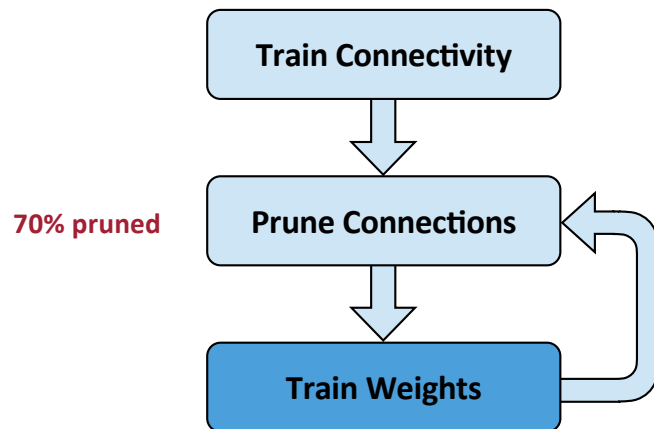
Iterative Pruning

- Consider pruning followed by a fine-tuning is one iteration.
- Iterative pruning gradually increases the target sparsity in each iteration.



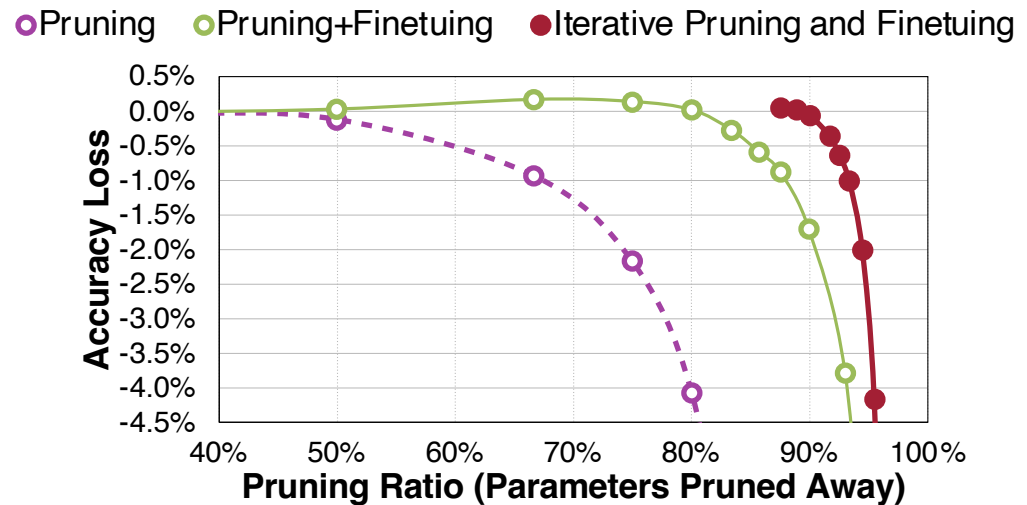
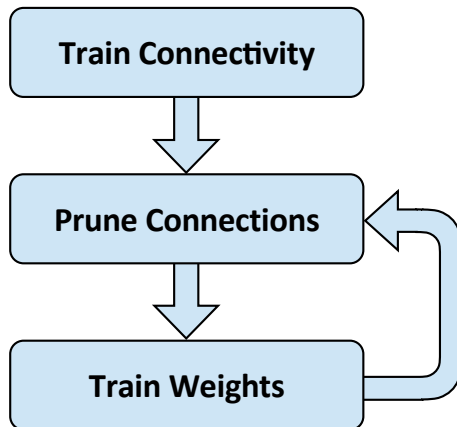
Iterative Pruning

- Consider pruning followed by a fine-tuning is one iteration.
- Iterative pruning gradually increases the target sparsity in each iteration.



Iterative Pruning

- Consider pruning followed by a fine-tuning is one iteration.
- Iterative pruning gradually increases the target sparsity in each iteration.
 - boost pruning ratio from 5X to 9X on AlexNet compared to single-step aggressive pruning.



Regularization

- When training neural networks or fine-tuning quantized neural networks, regularization is added to the loss term to
 - *penalize non-zero parameters*
 - *encourage smaller parameters*
- The most common regularization for improving performance of pruning is L1/L2 regularization.

- L1-Regularization

$$L' = L(\mathbf{x}; \mathbf{W}) + \lambda |\mathbf{W}|$$

- L2-Regularization

$$L' = L(\mathbf{x}; \mathbf{W}) + \lambda \|\mathbf{W}\|^2$$

- **Examples:**

- Magnitude-based Fine-grained Pruning applies L2 regularization on weights
- Network Slimming applies smooth-L1 regularization on channel scaling factors.

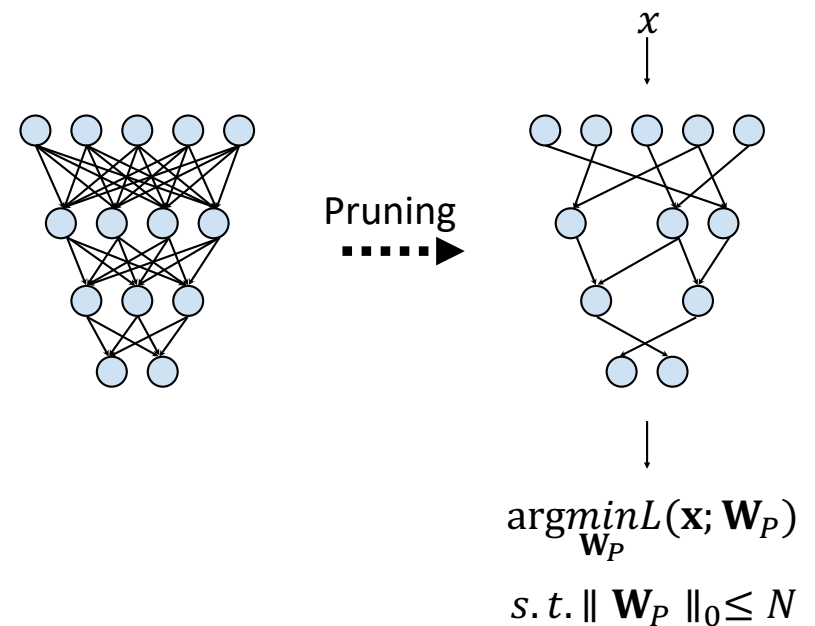
Learning Efficient Convolutional Networks through Network Slimming [Liu et al., ICCV 2017]

Learning Both Weights and Connections for Efficient Neural Network [Han et al., NeurIPS 2015]

<https://efficientml.ai>

Neural Network Pruning

- **Introduction to Pruning**
 - What is pruning?
 - How should we formulate pruning?
- **Determine the Pruning Granularity**
 - In what pattern should we prune the neural network?
- **Determine the Pruning Criterion**
 - What synapses/neurons should we prune?
- **Determine the Pruning Ratio**
 - What should target sparsity be for each layer?
- **Fine-tune/Train Pruned Neural Network**
 - How should we improve performance of pruned models?



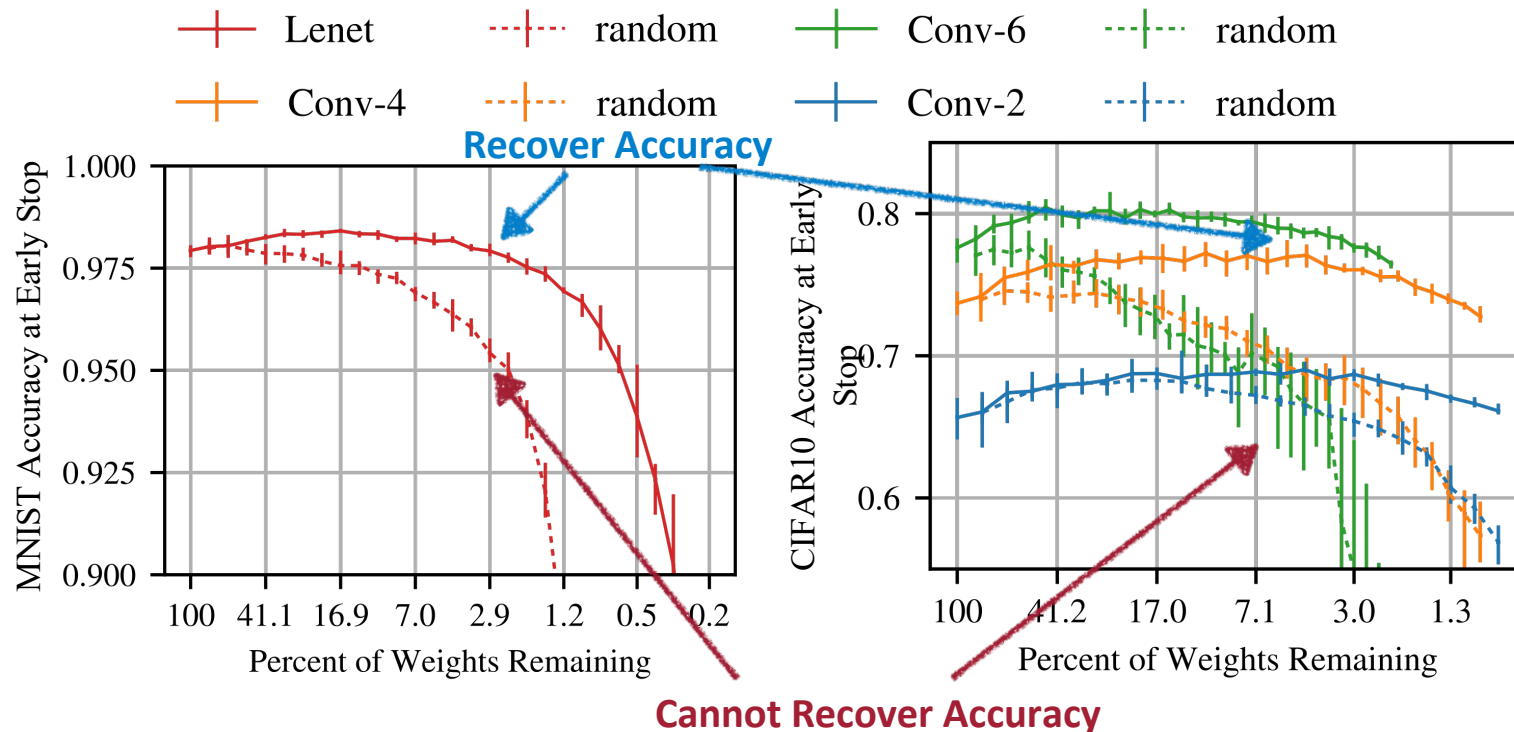
Lottery Ticket Hypothesis

Can we train a sparse neural network from scratch?

Train Sparse Neural Network From Scratch

- Neural Network Pruning shows that
 - **a neural network can be reduced in size.**
- Question: **Can we directly train this sparse neural network from scratch?**
- Contemporary experience tells us that
 - **the architectures uncovered by pruning are harder to train from the start,**
 - **reaching lower accuracy than the original networks**

The Lottery Ticket Hypothesis

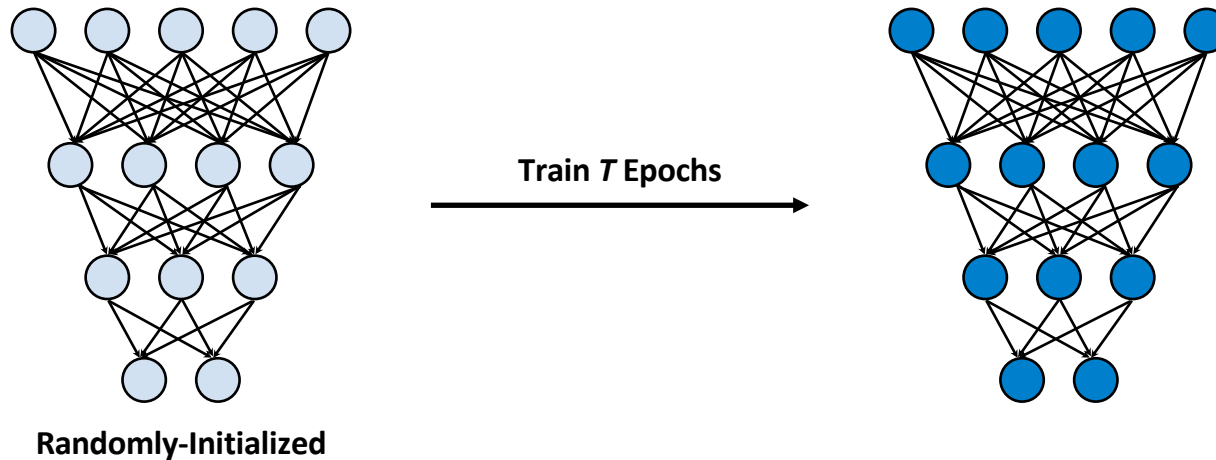


- Dashed Lines: randomly sampled sparse networks
- Solid Lines: correct sparsity mask (found by training) & same initialization.

The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks [Frankle *et al.*, ICLR 2019]
<https://efficientml.ai>

The Lottery Ticket Hypothesis

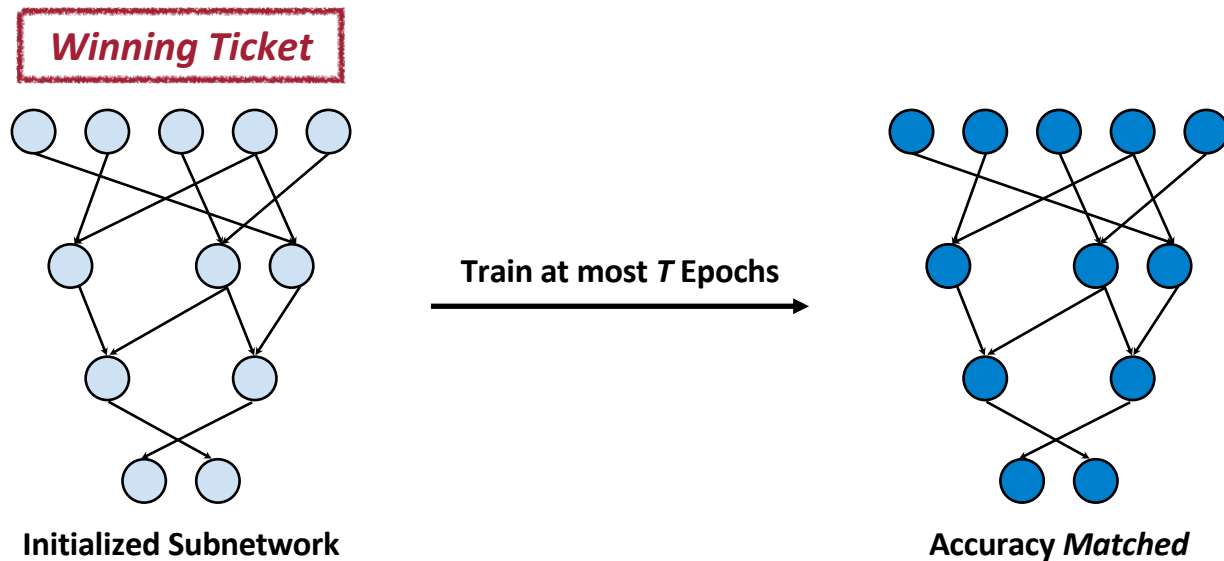
A randomly-initialized, dense neural network contains a subnetwork that is initialized such that—when trained in isolation—it can match the test accuracy of the original network after training for at most the same number of iterations.
—The Lottery Ticket Hypothesis



The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks [Frankle *et al.*, ICLR 2019]
<https://efficientml.ai>

The Lottery Ticket Hypothesis

*A randomly-initialized, dense neural network contains **a subnetwork** that is initialized such that—when **trained in isolation**—it can **match the test accuracy** of the original network after training for **at most the same number of iterations**.*
—The Lottery Ticket Hypothesis



The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks [Frankle *et al.*, ICLR 2019]
<https://efficientml.ai>

Intuition

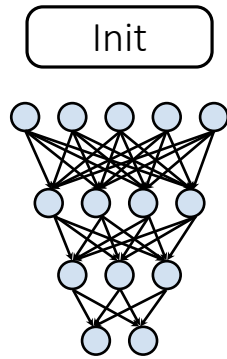
- You start with a big random network.
- Train it → prune small weights → get a sparse subnetwork.
- Reset that subnetwork to its **original initialization**.
- Train it again → surprisingly, it learns just as well.
- That subnetwork = “**winning ticket**.”

Why does initialization matter?

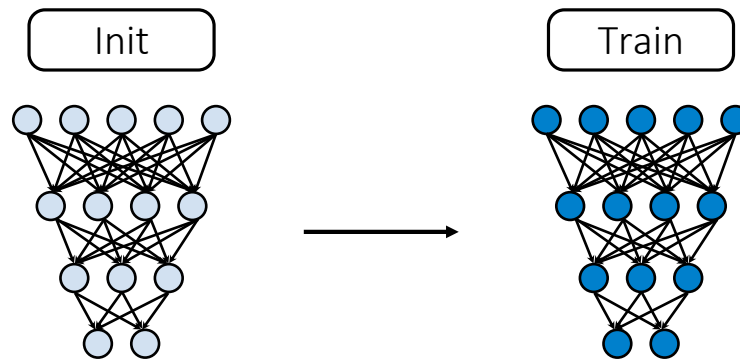
- **Optimization trajectory is encoded in initialization**
 - The initial random weights are not “neutral”:
 - They define **which gradient directions are reachable early**
 - They influence **which features emerge**

👉 The “winning ticket” is not just *which weights*, but also *how they started*

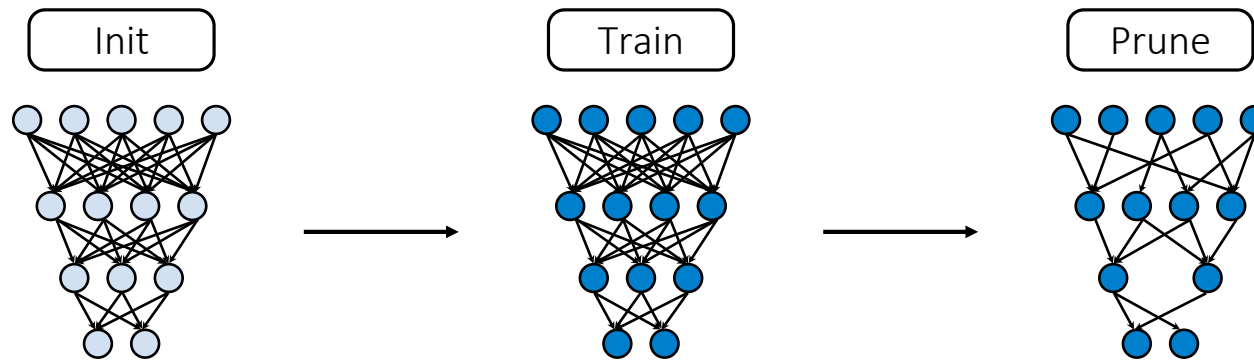
Iterative Magnitude Pruning



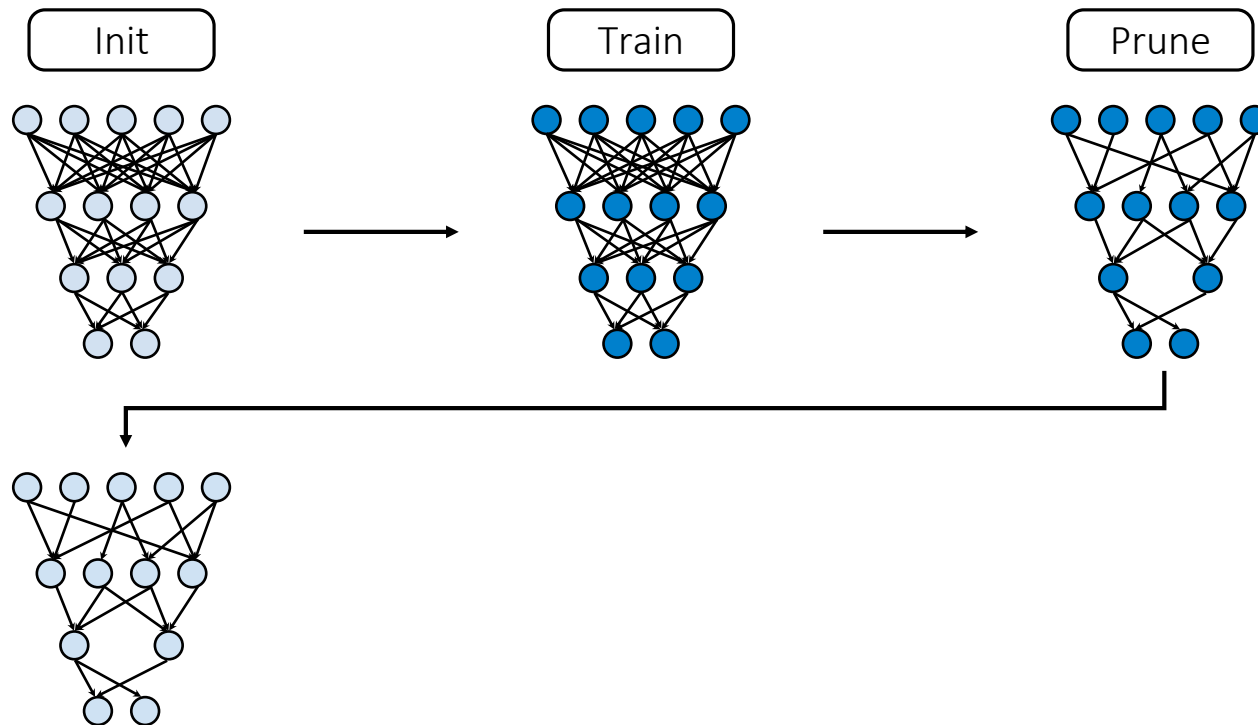
Iterative Magnitude Pruning



Iterative Magnitude Pruning

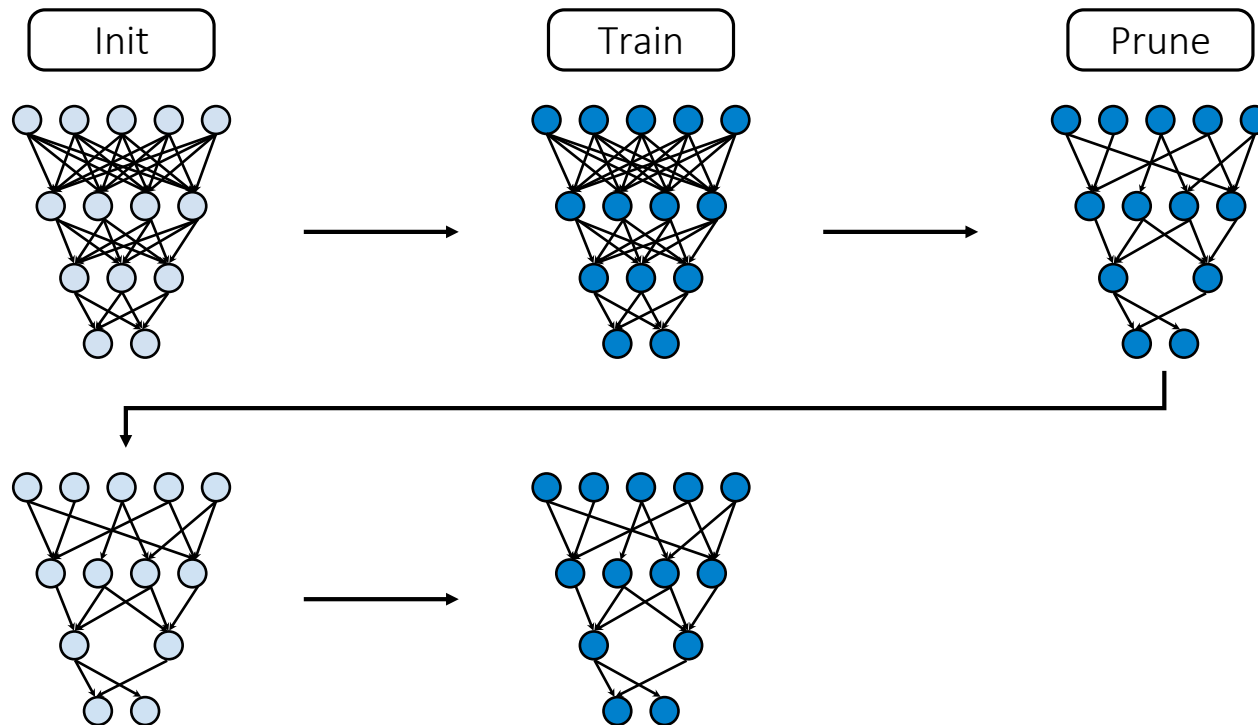


Iterative Magnitude Pruning



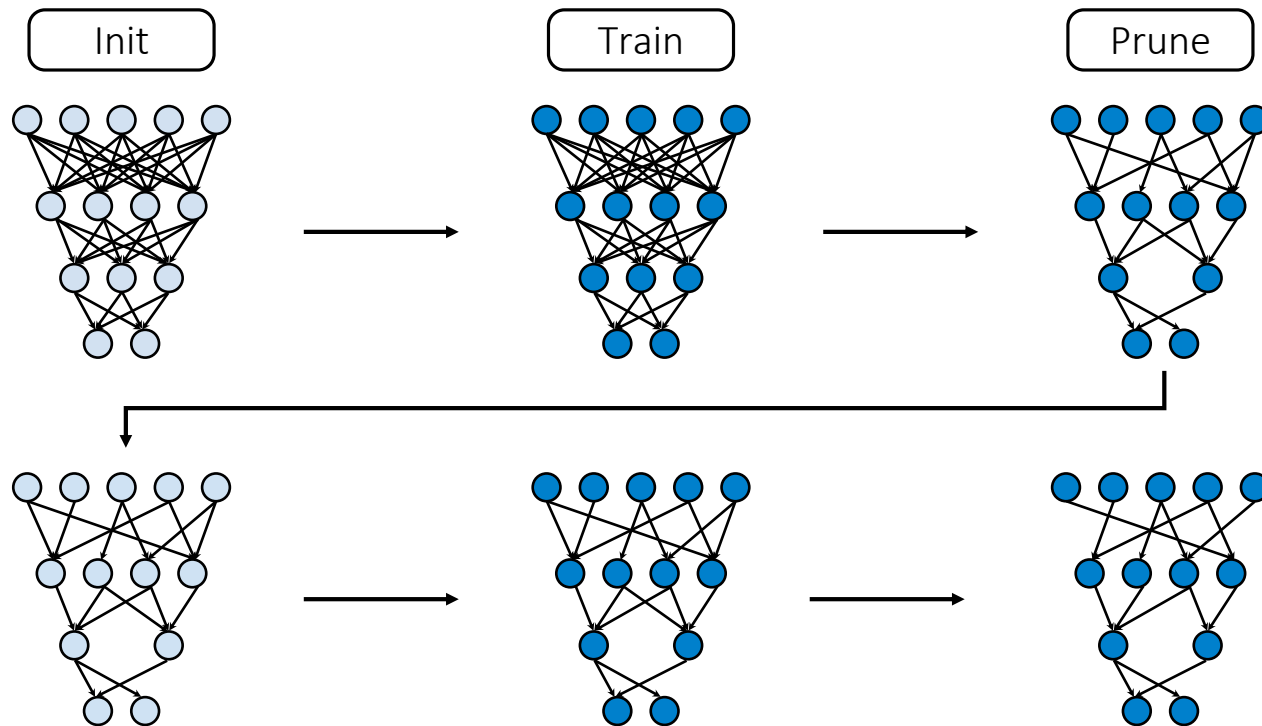
The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks [Frankle *et al.*, ICLR 2019]
<https://efficientml.ai>

Iterative Magnitude Pruning



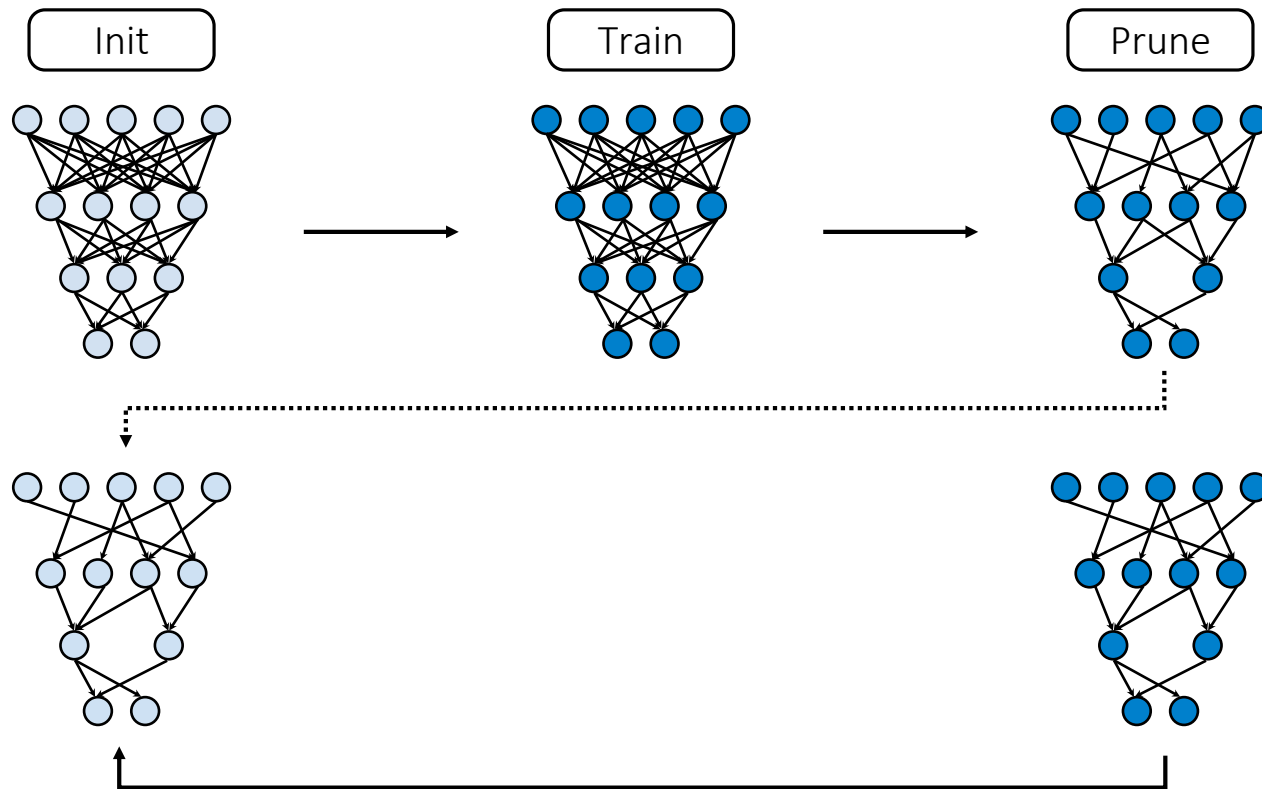
The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks [Frankle *et al.*, ICLR 2019]
<https://efficientml.ai>

Iterative Magnitude Pruning



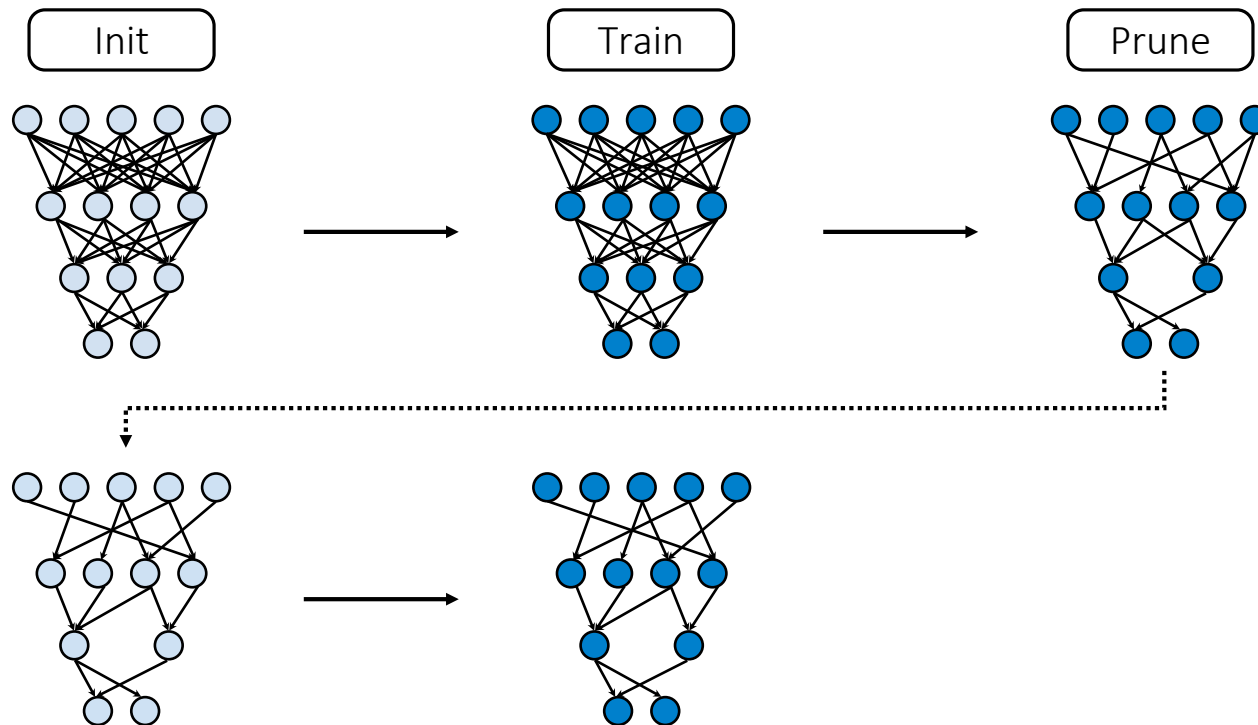
The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks [Frankle *et al.*, ICLR 2019]
<https://efficientml.ai>

Iterative Magnitude Pruning



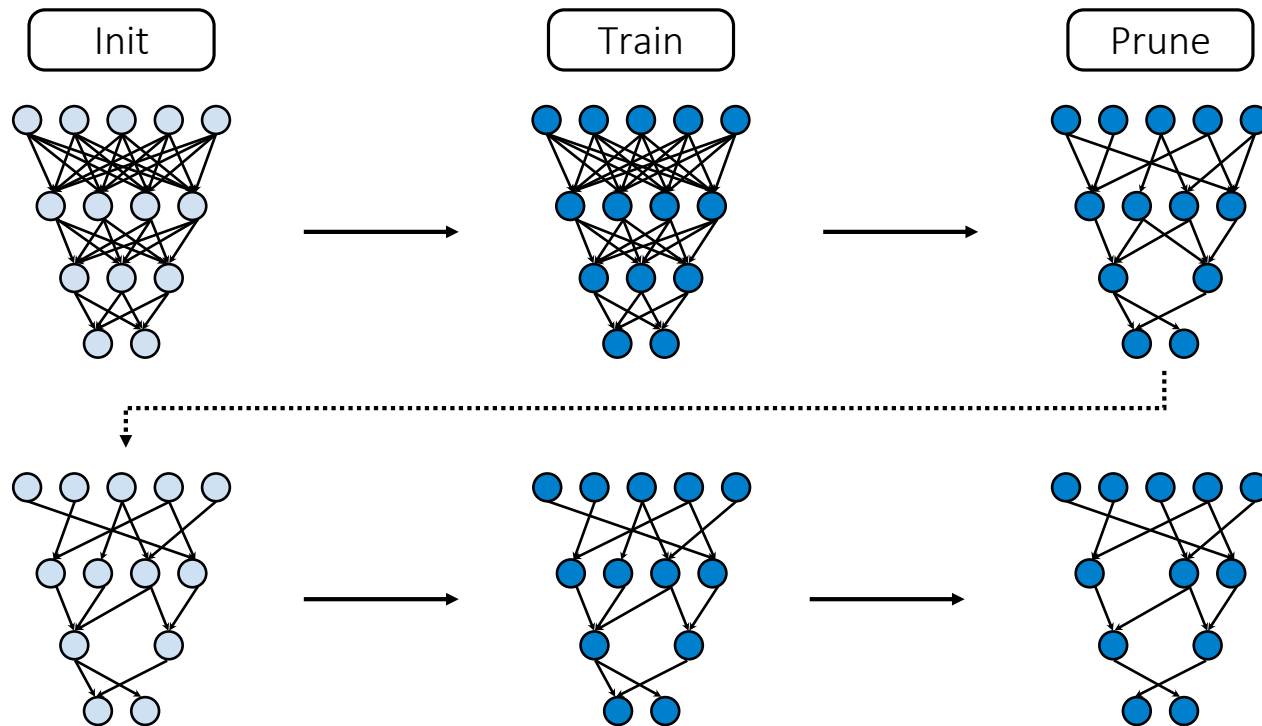
The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks [Frankle *et al.*, ICLR 2019]
<https://efficientml.ai>

Iterative Magnitude Pruning



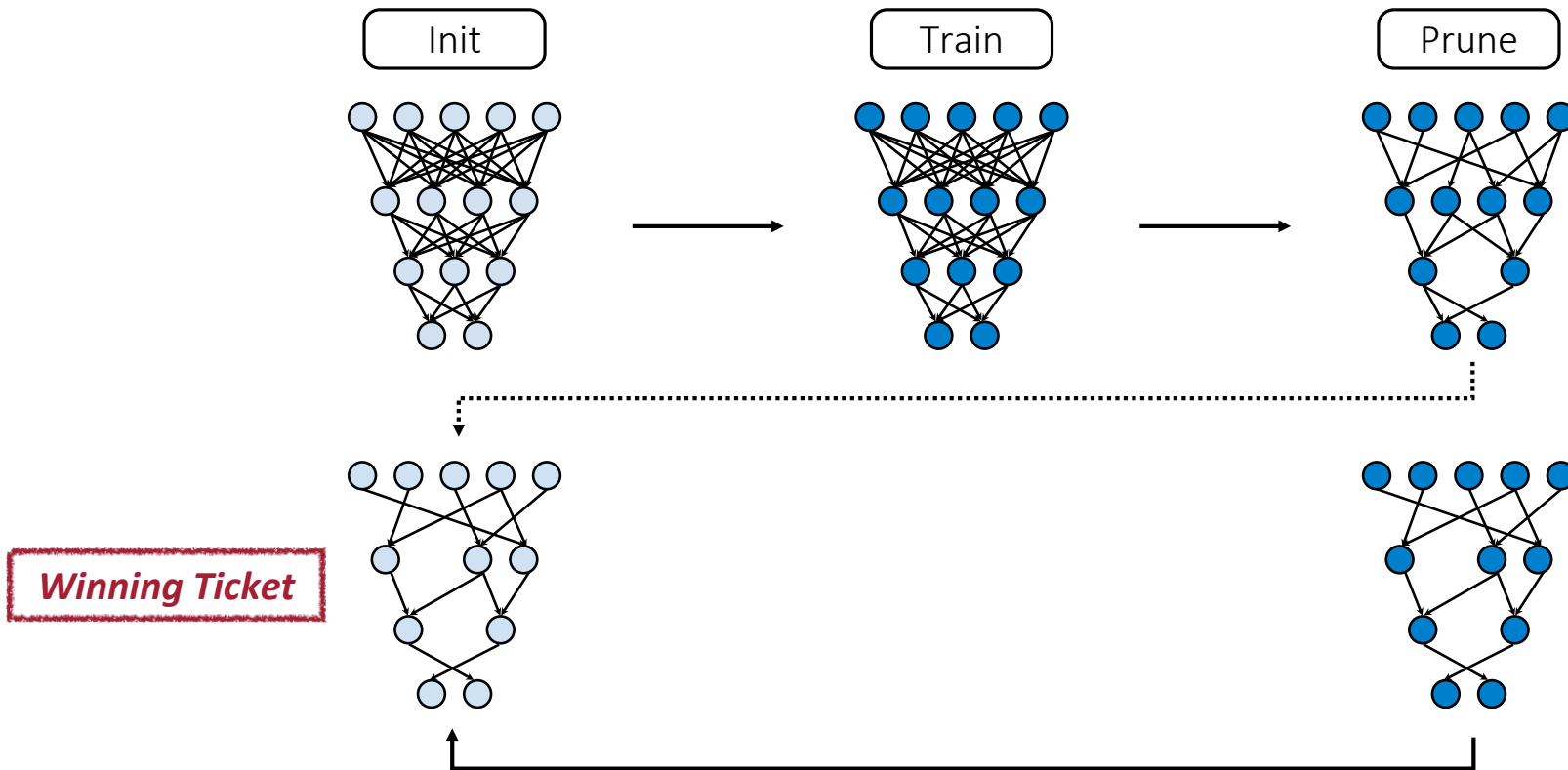
The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks [Frankle *et al.*, ICLR 2019]
<https://efficientml.ai>

Iterative Magnitude Pruning



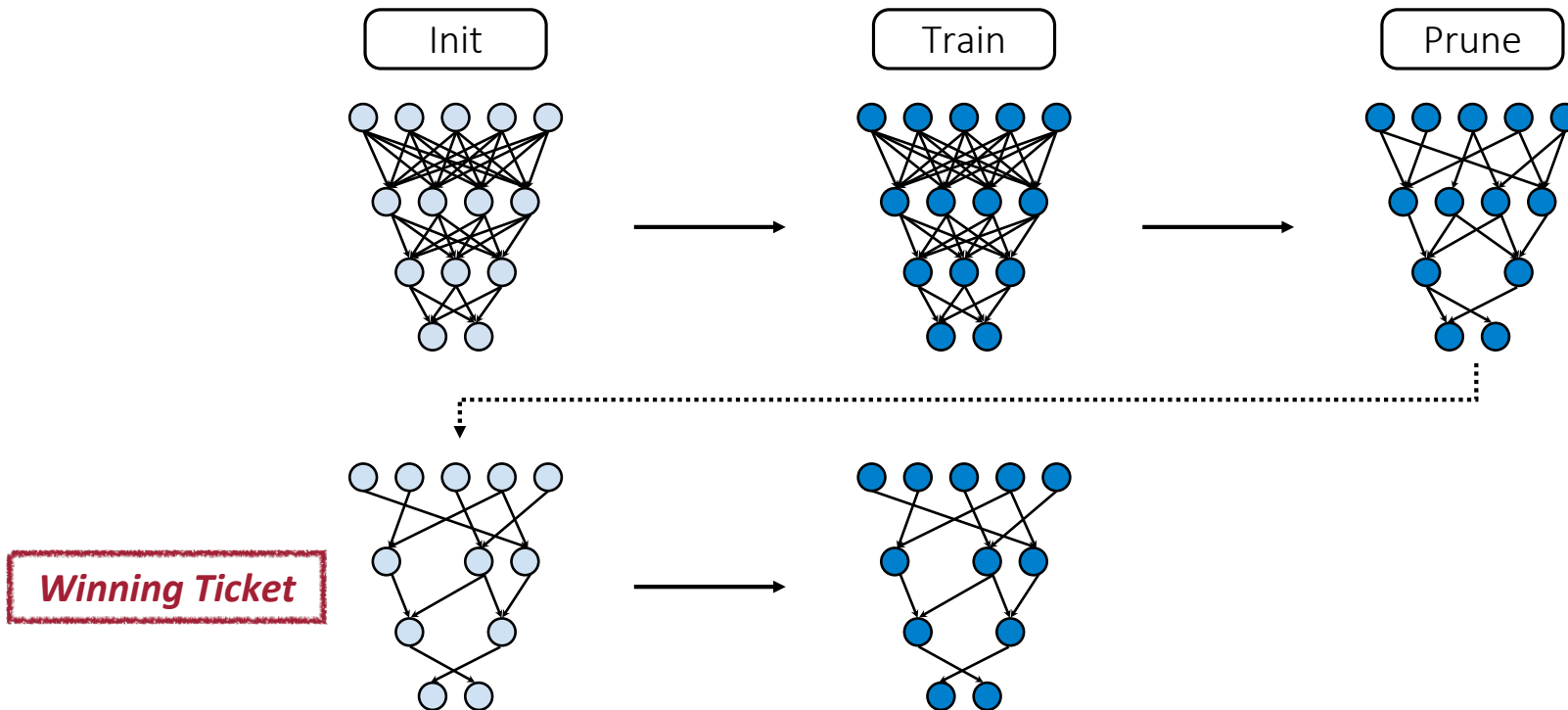
The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks [Frankle *et al.*, ICLR 2019]
<https://efficientml.ai>

Iterative Magnitude Pruning



The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks [Frankle *et al.*, ICLR 2019]
<https://efficientml.ai>

Iterative Magnitude Pruning

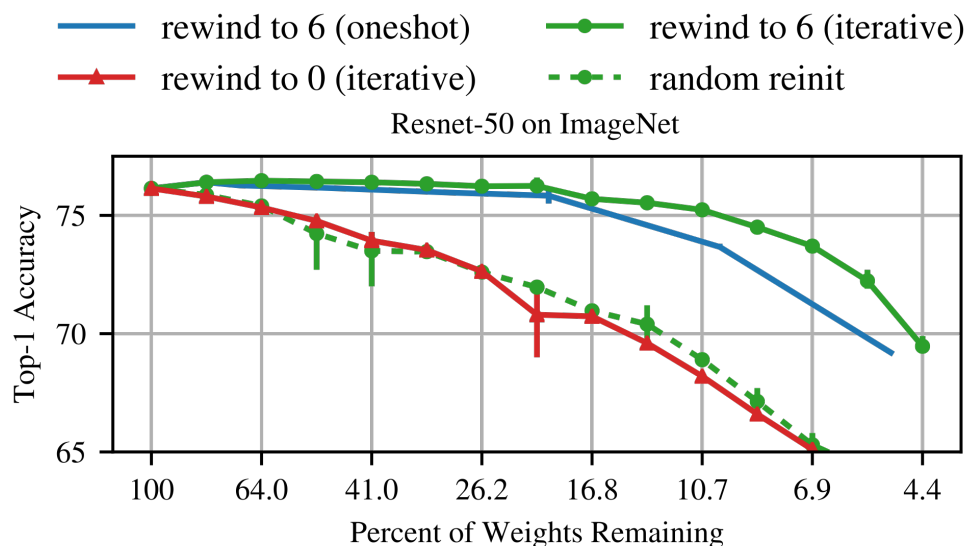


The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks [Frankle *et al.*, ICLR 2019]
<https://efficientml.ai>

Scaling Limitation

- Resetting the weights to the very initial value $\mathbf{W}_{t=0}$ works for small-scale tasks such as MNIST and CIFAR-10, and fails on deep networks.
- Instead, it is possible to robustly obtain pruned subnetworks by resetting the weights to the values after ***a small number of k training iterations***, that is

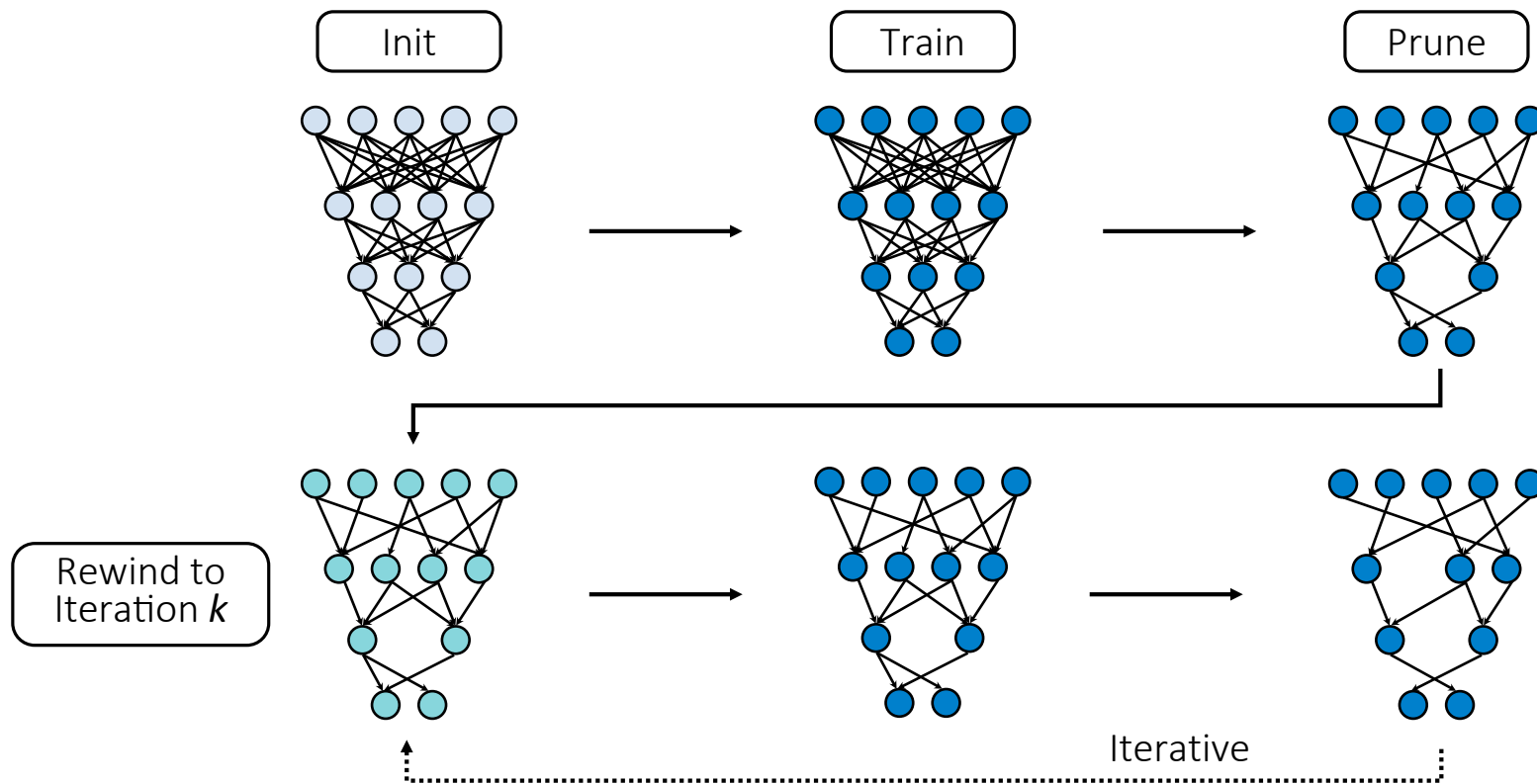
$$\mathbf{W}_{t=k}.$$



Total Training Epochs: $T = 90$
Rewinding to: $k = 6$

Stabilizing the Lottery Ticket Hypothesis [Frankle *et al.*, arXiv 2019]
<https://efficientml.ai>

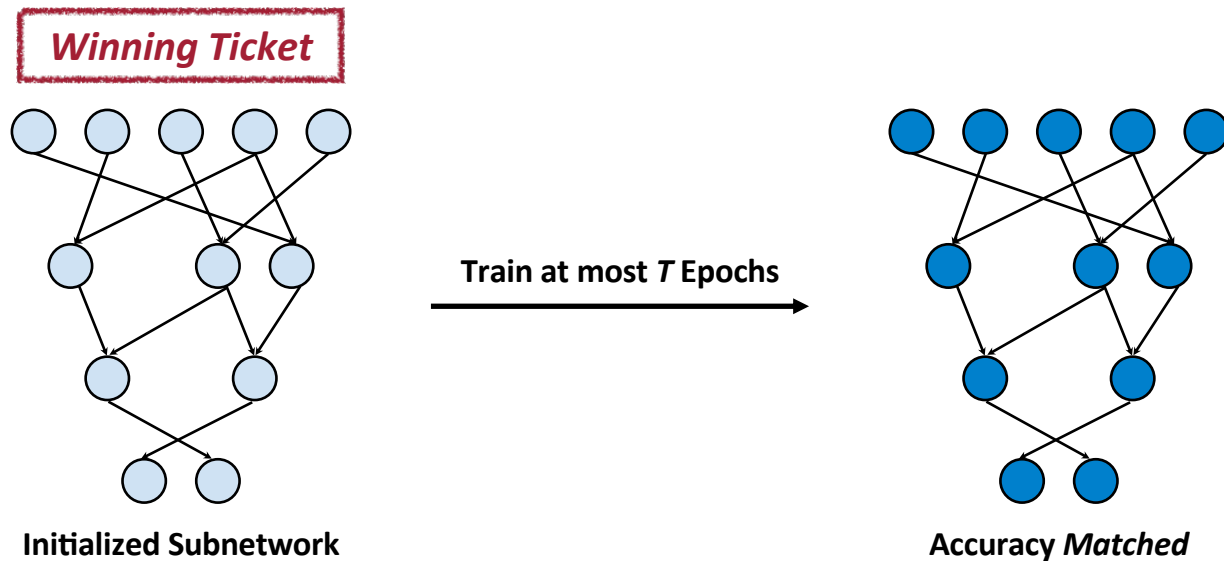
Iterative Magnitude Pruning with Rewinding



Stabilizing the Lottery Ticket Hypothesis [Frankle *et al.*, arXiv 2019]
<https://efficientml.ai>

The Lottery Ticket Hypothesis

*A randomly-initialized, dense neural network contains **a subnetwork** that is initialized such that—when **trained in isolation**—it can **match the test accuracy** of the original network after training for **at most the same number of iterations**.*
—The Lottery Ticket Hypothesis



The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks [Frankle *et al.*, ICLR 2019]
<https://efficientml.ai>

PyTorch Pruning Tutorial

[Tutorial Link](#)

 pruning_tutorial.ipynb

File Edit View Insert Runtime Tools Help [Cannot save changes](#)

+ Code + Text | Copy to Drive

[1] `%matplotlib inline`

✓ Pruning Tutorial

Author: Michela Paganini <<https://github.com/mickypaganini>>_

State-of-the-art deep learning techniques rely on over-parametrized models that are hard to deploy. On the contrary, biological neural networks are known to use efficient sparse connectivity. Identifying optimal techniques to compress models by reducing the number of parameters in them is important in order to reduce memory, battery, and hardware consumption without sacrificing accuracy, deploy lightweight models on device, and guarantee privacy with private on-device computation. On the research front, pruning is used to investigate the differences in learning dynamics between over-parametrized and under-parametrized networks, to study the role of lucky sparse subnetworks and initializations ("lottery tickets" <<https://arxiv.org/abs/1803.03635>>_) as a destructive neural architecture search technique, and more.

In this tutorial, you will learn how to use `torch.nn.utils.prune` to sparsify your neural networks, and how to extend it to implement your own custom pruning technique.

Requirements

```
"torch">=1.4.0a0+8e8a5e0"
```


Leveraging Sparsity on GPUs

A practical guide for model developers to exploit structured sparsity and hardware acceleration.

1. Types of Sparsity

- Unstructured: Random zeros – high compression, poor hardware acceleration.
- Structured (N:M): Regular pattern (e.g., 2:4) enabling GPU acceleration.
- Block/Channel: Zeros in whole channels or filters – common for CNNs.
- Hardware acceleration requires structured patterns that GPUs can exploit.

2. Producing a Sparse Model

- There are three main ways to introduce sparsity:
 1. Post-Training Pruning (PTP)
 2. Structured/N:M Pruning (Hardware-aware)
 3. Train-Aware Sparsity (RigL, Lottery Ticket, Movement Pruning)

```
import torch
import torch.nn.utils.prune as prune

model = MyModel()
prune.random_unstructured(model.fc1, name="weight", amount=0.5)
prune.remove(model.fc1, "weight") # make pruning permanent
```

3. Structured 2:4 Pruning Example

- To leverage GPU acceleration, prune weights according to the supported N:M pattern. For NVIDIA A100/H100, the 2:4 pattern is required (2 zeros per 4 weights).

```
from torch.ao.pruning import sparsifier

sparsifier = sparsifier.NMPruningSparsifier(model, n=2, m=4)
sparsifier.step() # apply structured pruning
```

4. Using TensorRT for Sparse Inference

- TensorRT 8+ automatically accelerates 2:4 structured sparsity.

Workflow:

1. Export dense model to ONNX.
2. Apply structured pruning (2:4).
3. Build TensorRT engine with sparsity enabled.

```
trtexec --onnx=model.onnx --sparsity=enable --  
saveEngine=model.plan
```

5. cuSPARSELt API Example

- For low-level CUDA developers, cuSPARSELt offers APIs for compression and sparse GEMM.
Available in CUDA 11.4+.

```
cusparseLtSpMM_compress(handle, descA, dense_weights,  
sparse_weights);  
cusparseLtMatmul(handle, opDesc, alpha, sparse_weights, input,  
beta, output);
```

6. PyTorch Integration

- PyTorch 2.x provides native CUDA backend support for 2:4 sparsity. Combine with torch.compile to preserve sparse kernels.

```
torch.nn.utils.prune.StructuredSparsity.apply(model.linear,  
n=2, m=4)  
torch.compile(model)  # TorchDynamo + Inductor supports sparse  
kernels
```

7. AMD ROCm Structured Sparsity

- AMD MI300 supports structured sparsity through ROCm's Matrix Cores. Use ROCm libraries such as MIOpen or hipSPARSE.

```
hipsparseSpMV(handle, alpha, matA, vecX, beta, vecY,  
rocparse_mv_alg_auto);
```


8. Measure and Validate

- After pruning, always measure speedup and accuracy:
 - Compare latency and throughput with/without sparsity.
 - Validate numerical correctness.

```
torch.allclose(output_dense, output_sparse, atol=1e-3)
```

9. Developer Checklist

Step-by-step for leveraging GPU sparsity:

1. Identify hardware sparsity pattern.
2. Apply structured pruning.
3. Fine-tune to recover accuracy.
4. Export to ONNX or TensorRT.
5. Benchmark speedup.
6. Integrate in CI/CD for automated sparse inference tests.

References

1. Learning Both Weights and Connections for Efficient Neural Network [Han et al., NeurIPS 2015]
2. Exploring the granularity of sparsity in convolutional neural networks [Mao et al., CVPR-W]
3. Learning Structured Sparsity in Deep Neural Networks [Wen et al., NeurIPS 2016]
4. Learning Efficient Convolutional Networks through Network Slimming [Liu et al., ICCV 2017]
5. A Systematic DNN Weight Pruning Framework using Alternating Direction Method of Multipliers [Zhang et al., ECCV 2018]
6. AMC: Automl for Model Compression and Acceleration on Mobile Devices [He et al., ECCV 2018]
7. Accelerating Inference with Sparsity Using the NVIDIA Ampere Architecture and NVIDIA TensorRT
8. EIE: Efficient Inference Engine on Compressed Deep Neural Network [Han et al., ISCA 2016]
9. ESE: Efficient Speech Recognition Engine with Sparse LSTM on FPGA [Han et al., FPGA 2017]
10. [Block Sparse Format](#) [NVIDIA, 2021]
11. Accelerating Sparse Deep Neural Networks [Mishra et al., arXiv 2021]

References

1. Model Compression and Hardware Acceleration for Neural Networks: A Comprehensive Survey [Deng et al., IEEE 2020]
2. Computing's Energy Problem (and What We Can Do About it) [Horowitz, M., IEEE ISSCC 2014]
3. Optimal Brain Damage [LeCun et al., NeurIPS 1989]
4. Learning Both Weights and Connections for Efficient Neural Network [Han et al., NeurIPS 2015]
5. Efficient Methods and Hardware for Deep Learning [Han S., Stanford University]
6. Peter Huttenlocher (1931–2013) [Walsh, C. A., Nature 2013]
7. Exploring the granularity of sparsity in convolutional neural networks [Mao et al., CVPR-W]
8. Accelerating Inference with Sparsity Using the NVIDIA Ampere Architecture and NVIDIA TensorRT
9. AMC: Automl for Model Compression and Acceleration on Mobile Devices [He et al., ECCV 2018]
10. Learning Structured Sparsity in Deep Neural Networks [Wen et al., NeurIPS 2016]
11. Learning Efficient Convolutional Networks through Network Slimming [Liu et al., ICCV 2017]
12. Pruning Convolutional Filters with First Order Taylor Series Ranking [Wang M.]
13. Importance Estimation for Neural Network Pruning [Molchanov et al., CVPR 2019]
14. Network Trimming: A Data-Driven Neuron Pruning Approach towards Efficient Deep Architectures [Hu et al., ArXiv 2017]
15. Pruning Convolutional Neural Networks for Resource Efficient Inference [Molchanov et al., ICLR 2017]
16. Channel Pruning for Accelerating Very Deep Neural Networks [He et al., ICCV 2017]
17. ThiNet: A Filter Level Pruning Method for Deep Neural Network Compression [Luo et al., ICCV 2017]

Acknowledgments:

Part of this material has been adapted from the MIT efficient Deep Learning class by Prof. Song Han, Quantization Neural Network white paper (<https://arxiv.org/pdf/2106.08295.pdf>), and Floran de Putter lecture

Class Attendance QR Code – April 20, 2026

